

GLM-5: from Vibe Coding to Agentic Engineering

GLM-5: 从 vibe 编码到代理工程

GLM-5 Team

GLM-5 团队

Zhipu AI & Tsinghua University

智普 AI 与清华大学

(For the complete list of authors, please refer to the Contribution section)

(完整作者名单请参见贡献部分)

Abstract

摘要

We present GLM-5, a next-generation foundation model designed to transition the paradigm of vibe coding to agentic engineering. Building upon the agentic, reasoning, and coding (ARC) capabilities of its predecessor, GLM-5 adopts DSA to significantly reduce training and inference costs while maintaining long-context fidelity. To advance model alignment and autonomy, we implement a new asynchronous reinforcement learning infrastructure that drastically improves post-training efficiency by decoupling generation from training. Furthermore, we propose novel asynchronous agent RL algorithms that further improve RL quality, enabling the model to learn from complex, long-horizon interactions more effectively. Through these innovations, GLM-5 achieves state-of-the-art performance on major open benchmarks. Most critically, GLM-5 demonstrates unprecedented capability in real-world coding tasks, surpassing previous baselines in handling end-to-end software engineering challenges. Code, models, and more information are available at <https://github.com/zai-org/GLM-5>.

我们推出 GLM-5，一款下一代基础模型，旨在将 vibe 编码的范式转变为自主工程。GLM-5 建立在前代模型的自主性、推理和编码（ARC）能力之上，采用 DSA（分布式策略架构）显著降低训练和推理成本，同时保持长上下文的准确性。为了推进模型对齐和自主性，我们实现了一种新的异步强化学习基础设施，通过将生成与训练解耦，大幅提高训练后的效率。此外，我们提出了新颖的异步智能体强化学习算法，进一步提升强化学习的质量，使模型能够更有效地从复杂、长周期的交互中学习。通过这些创新，GLM-5 在主要的开放基准测试中取得了最先进的性能。最关键的是，GLM-5 在现实世界的编码任务中展现出前所未有的能力，超越了以往的基线，在端到端软件工程挑战中表现出色。代码、模型和更多信息可在 <https://github.com/zai-org/GLM-5> 获取。

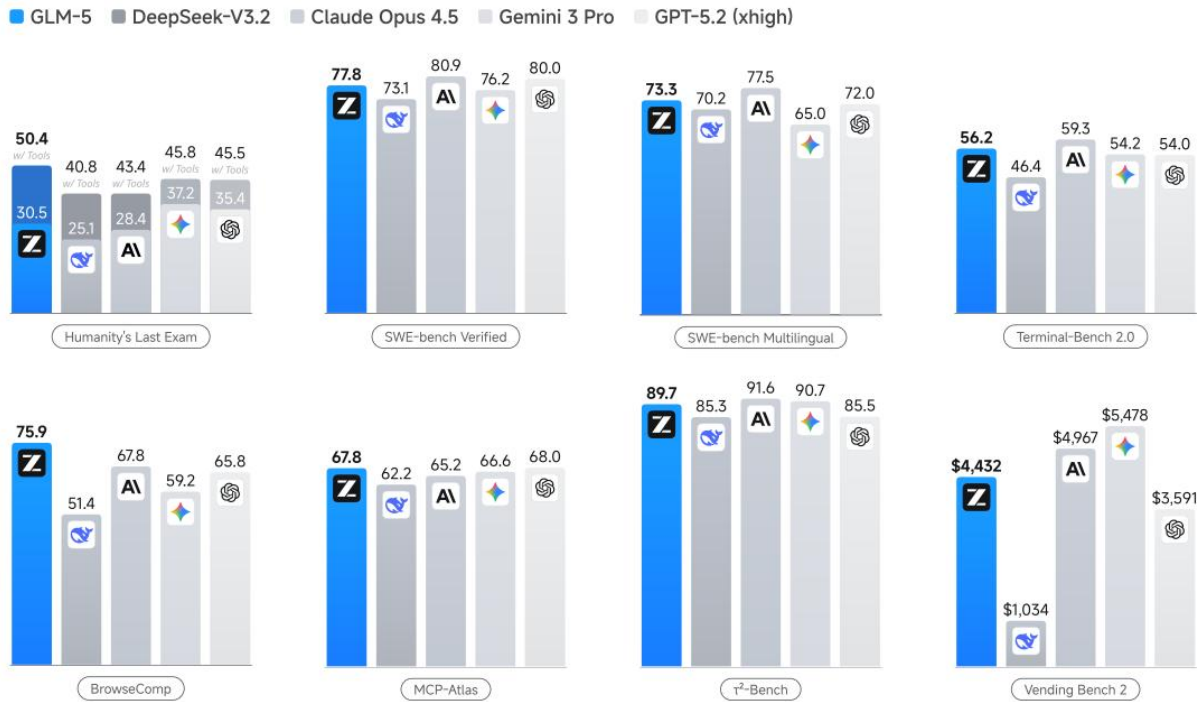


Figure 1: images/0c37c229aceafe0f194eff103444b564bb2a9f64344ca57e4eb21d487f120ca5.jpg

Figure 1: Results of GLM-5, DeepSeek-V3.2, Claude Opus 4.5, Gemini 3 Pro, and GPT-5.2 (xhigh) on 8 agentic, reasoning, and coding benchmarks: Humanity's Last Exam, SWE-bench Verified, SWE-bench Multilingual, Terminal-Bench 2.0, BrowseComp, MCP-Atlas, τ^2 -Bench, Vending Bench 2.

图 1: GLM-5、DeepSeek-V3.2、Claude Opus 4.5、Gemini 3 Pro 和 GPT-5.2 (xhigh) 在 8 个代理、推理和编码基准测试中的结果：人类最后一场考试、SWE-bench 验证版、SWE-bench 多语言版、Terminal-Bench 2.0、BrowseComp、MCP-Atlas、 τ^2 -Bench、Vending Bench 2。

Artificial Analysis Intelligence Index

人工智能分析指数

Artificial Analysis Intelligence Index v4.0 incorporates 10 evaluations: GDPval-AA, τ^2 -Bench Telecom, Terminal-Bench Hard, SciCode, AA-LCR, AA-Omniscience, IFBench, Humanity's Last Exam, GPQA Diamond, CritPt

Artificial Analysis Intelligence Index v4.0 包含 10 项评估：GDPval-AA, τ^2 -Bench Telecom, Terminal-Bench Hard, SciCode, AA-LCR, AA-Omniscience, IFBench, Humanity's Last Exam, GPQA Diamond, CritPt

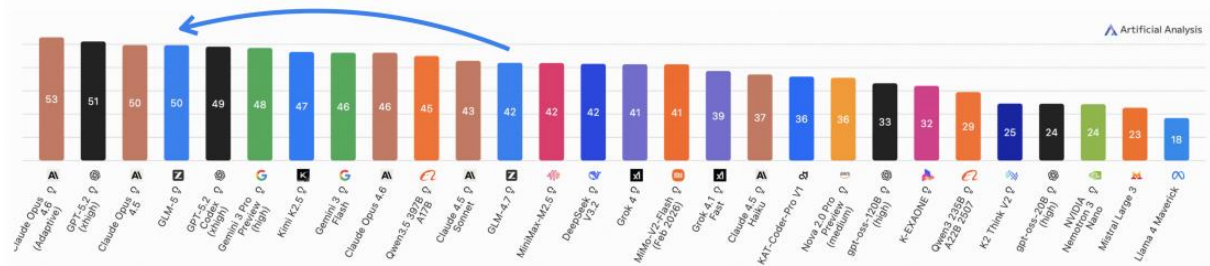


Figure 2: images/4df4b09b6e3e78bc3bbecc4909c5b8e35e7fd9be6f2dac14cb6c21fa437bf335.jpg

Figure 2: Artificial Analysis Intelligence Index v4.0 incorporates 10 evaluations: GDPval-AA, τ^2 -Bench Telecom, Terminal-Bench Hard, SciCode, AA-LCR, AA-Omniscience, IFBench, Humanity's Last Exam, GPQA Diamond, CritPt.

图 2: 人工智能分析指数 v4.0 包含 10 项评估: GDPval-AA, τ^2 -Bench 电信, 终端-Bench 硬件, SciCode, AA-LCR, AA-全知, IFBench, 人类最后的考试, GPQA 钻石, CritPt.

1 Introduction

1 引言

The pursuit of Artificial General Intelligence (AGI) requires not only scaling model parameters but also fundamentally rethinking the efficiency of intelligence and the architecture of autonomous improvement. With the release of GLM-4.5, we demonstrated that uniting Agentic, Reasoning, and Coding (ARC) capabilities into a single Model-of-Experts (MoE) architecture could yield state-of-the-art results across diverse benchmarks. However, as Large Language Models (LLMs) transition from passive knowledge repositories to active problem solvers, the dual challenges of computational cost and real-world adaptability—particularly in complex software engineering—have become the primary bottlenecks.

追求通用人工智能（AGI）不仅需要扩大模型参数，还需要从根本上重新思考智能效率和自主改进的架构。随着 GLM-4.5 的发布，我们展示了将自主性（Agentic）、推理（Reasoning）和编码（Coding，即 ARC）能力结合到单一专家模型（Model-of-Experts, MoE）架构中，可以在多种基准测试中取得最先进的结果。然而，随着大型语言模型（LLMs）从被动的知识存储库转变为积极的问题解决者，计算成本和现实世界适应性——尤其是在复杂的软件工程领域——的双重挑战已成为主要的瓶颈。

We present GLM-5, our next-generation flagship model designed to overcome these barriers. GLM-5 represents a paradigm shift in both performance and efficiency, achieving state-of-the-art status on major open leaderboards, including ArtificialAnalysis.ai, the LMArena Text, and the LMArena Code. More significantly, GLM-5 redefines the standard for real-world coding, demonstrating an unprecedented ability to handle complex, end-to-end software development tasks that go far beyond the scope of traditional static benchmarks like SWE-bench.

我们推出 GLM-5，这是我们下一代旗舰模型，旨在克服这些障碍。GLM-5 在性能和效率方面实现了范式转变，在包括 ArtificialAnalysis.ai、LMArena Text 和 LMArena Code 等主要开放排行榜上达到了最先进的水平。更重要的是，GLM-5 重新定义了现实世界编程的标准，展示了处理复杂、端到端软件开发任务的非凡能力，这些任务远超传统静态基准（如 SWE-bench）的范围。

Results. Figure 1 shows the results of GLM-5, GLM-4.7, Claude Opus 4.5, Gemini 3 Pro, and GPT-5.2 (xhigh) on 8 agentic, reasoning, and coding benchmarks: Humanity's Last Exam [34], SWE-bench Verified [19], SWE-bench Multilingual [53], Terminal-Bench 2.0 [45], BrowseComp [50], MCP-Atlas [6], τ^2 -Bench [55; 7], Vending Bench 2 [3]. On average, GLM-5 achieves about 20% improvement over our last version GLM-4.7, and is comparable to Claude Opus 4.5 and GPT-5.2 (xhigh), and

better than Gemini 3 Pro.

结果。图 1 展示了 GLM-5、GLM-4.7、Claude Opus 4.5、Gemini 3 Pro 和 GPT-5.2 (xhigh) 在 8 个涉及代理行为、推理和编程的基准测试中的表现：Humanity's Last Exam [34]、SWE-bench Verified [19]、SWE-bench Multilingual [53]、Terminal-Bench 2.0 [45]、BrowseComp [50]、MCP-Atlas [6]、 τ^2 -Bench [55; 7]、Vending Bench 2 [3]。总体而言，GLM-5 在我们的上一版本 GLM-4.7 的基础上实现了约 20% 的提升，并与 Claude Opus 4.5 和 GPT-5.2 (xhigh) 相当，优于 Gemini 3 Pro。

GLM-5 scores 50 on the Intelligence Index v4.0 and is the new open weights leader (Cf. Figure 2), up from GLM-4.7's score of 42 - an 8 point jump driven by improvements across agentic performance and knowledge/hallucination. This is the first time an open weights model has achieved a score of 50 on the Artificial Analysis Intelligence Index v4.0.

GLM-5 在 Intelligence Index v4.0 中得分 50，并成为新的开放权重领导者（参见图 2），相较于 GLM-4.7 的 42 分，提升了 8 分，这一进步得益于代理性能和知识/幻觉方面的改进。这是首个在 Artificial Analysis Intelligence Index v4.0 中达到 50 分的开放权重模型。

LMarena, initiated by UC Berkeley, is a transparent, shared space to evaluate and compare frontier AI capabilities by human judgment with millions of real tasks, including writing, coding, reasoning, designing, searching, and creating. The large volume of human interactions generates signals of real-world utility, making it different from the other static benchmarks. Figure 3 shows that GLM-5 again is the #1 open model in both Text Arena and Code Arena, and overall on par with Claude-Opus-4.5 and Gemini-3-pro.

LMarena 由加州大学伯克利分校发起，是一个透明、共享的空间，通过数百万个真实任务（包括写作、编程、推理、设计、搜索和创作）由人类判断来评估和比较前沿 AI 能力。大量的人机交互生成了现实世界效用的信号，使其与其他静态基准有所不同。图 3 显示，GLM-5 在文本领域和代码领域中再次成为排名第一的开源模型，整体表现与 Claude-Opus-4.5 和 Gemini-3-pro 相当。

Long-term coherence in agents becomes more and more important. Coding agents can now write code autonomously for hours, and the length and breadth of tasks AI models are able to complete are likely to increase. We use two benchmarks, Vending-Bench 2 and CC-Bench-V2, to evaluate how GLM-5 is able to complete long-horizon tasks. Vending-Bench 2 is a benchmark for measuring AI model performance in running a business over long time horizons. Models are tasked with running a simulated vending machine business over a year and are scored on their bank account balance at the end. Figure 4 (left) shows that GLM-5 ranks #1 among all open-source models, finishing with a final account balance of \$4,432. It approaches Claude Opus 4.5, demonstrating strong long-term

长期的连贯性对于代理来说变得越来越重要。编码代理现在可以自主编写代码数小时，AI 模型能够完成的任务长度和广度很可能还会增加。我们使用两个基准测试，Vending-Bench 2 和 CC-Bench-V2，来评估 GLM-5 完成长期任务的能力。Vending-Bench 2 是一个用于衡量 AI 模型在长时间范围内经营业务性能的基准测试。模型被要求经营一个模拟自动售货机业务一年，并根据最终银行账户余额进行评分。图 4 (左) 显示，GLM-5 在所有开源模型中排名第一，最终账户余额为 4,432 美元。它接近 Claude Opus 4.5，展示了强大的长期性能。

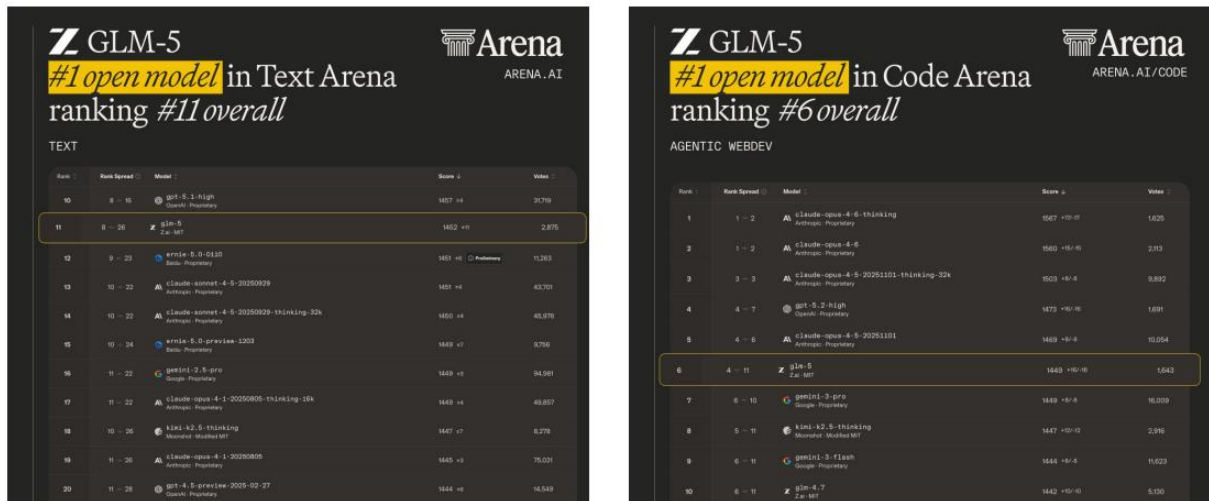


Figure 3: On LMArena, GLM-5 is the #1 open model in both Text Arena and Code Arena.

图 3：在 LMArena 上，GLM-5 在 Text Arena 和 Code Arena 中都是排名第一的开源模型。

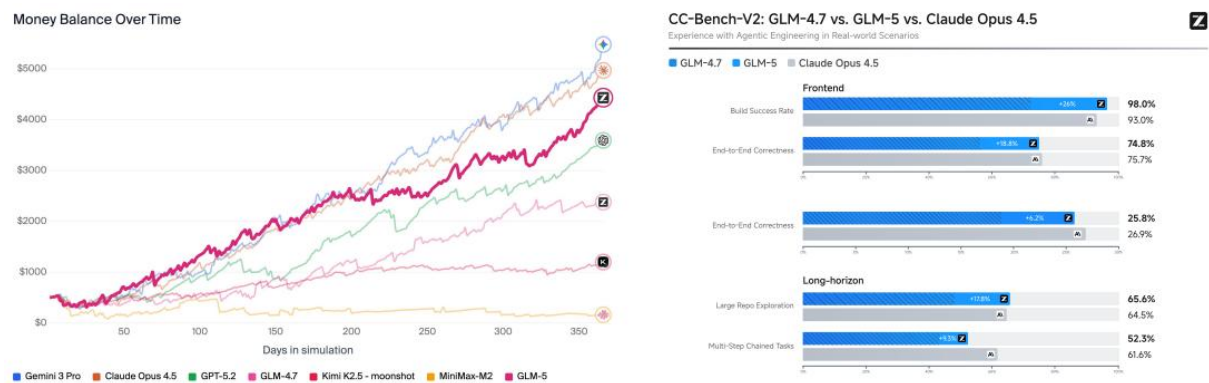


Figure 4: Results on several long-horizon tasks. Left: Vending-Bench 2; Right: CC-Bench-V2.

图 4：在多个长时间任务上的结果。左：Vending-Bench 2；右：CC-Bench-V2。

planning and resource management. Figure 4 (right) further shows results on our internal evaluation suite CC-Bench-V2. GLM-5 significantly outperforms GLM-4.7 across frontend, backend, and long-horizon tasks, narrowing the gap with Claude Opus 4.5.

规划与资源管理。图 4（右侧）进一步展示了我们在内部评估套件 CC-Bench-V2 上的结果。GLM-5 在前端、后端以及长周期任务中显著优于 GLM-4.7，缩小了与 Claude Opus 4.5 之间的差距。

Methods. Figure 5 shows the overall training pipeline of GLM-5. Our Base Model training began with a massive 27 trillion token corpus, prioritizing code and reasoning early on. We then employed a distinct Mid-training phase to progressively extend context length from 4K to 200K, focusing specifically on long-context agentic data to ensure stability in complex workflows. In Post-Training, we moved beyond standard SFT. We implemented a sequential Reinforcement Learning pipeline—starting with Reasoning RL, followed by Agentic RL, and finishing with General RL. Crucially, we utilized On-Policy Cross-Stage Distillation throughout this process to prevent catastrophic forgetting, ensuring the model retains its sharp reasoning edge while becoming a robust generalist. In summary, the leap in GLM-5's performance is driven by the following technical contributions:

方法。图 5 展示了 GLM-5 的整体训练流程。我们的基础模型训练最初基于一个庞大的 27 万亿个 token 的语料库，并优先处理代码和推理内容。随后，我们采用了一个独特的中期训练阶段，逐步将上下文长度从 4K 扩展到 200K，特别关注长上下文的代理数据，以确保在复杂工作流程中的稳定性。在后期训练阶段，我们超越了标准的监督微调（SFT）。我们实施了一个顺序的强化学习流程——首先进行推理强化学

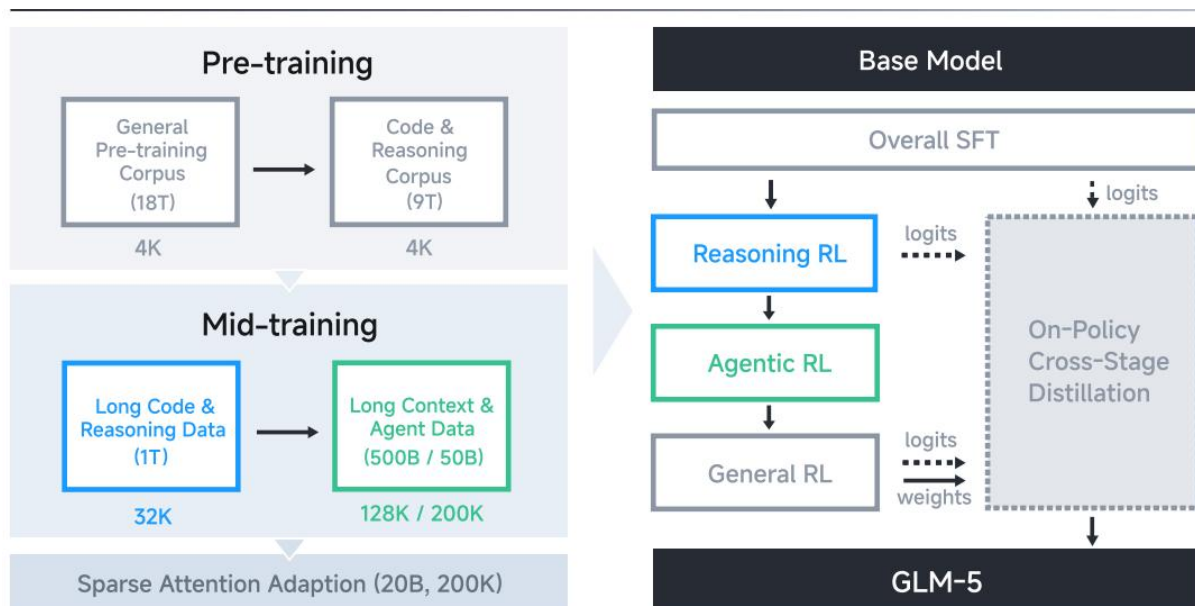
习，接着进行代理强化学习，最后进行通用强化学习。关键的是，在整个过程中，我们利用了基于策略的跨阶段蒸馏（On-Policy Cross-Stage Distillation），以防止灾难性遗忘，确保模型在保持其敏锐推理能力的同时，成为一个稳健的通用模型。总之，GLM-5 性能的提升主要得益于以下技术贡献：

First, we adopt DSA (DeepSeek Sparse Attention) [9], a novel architectural innovation that significantly reduces both training and inference costs. While GLM-4.5 improved efficiency through a standard MoE architecture, DSA allows GLM-5 to dynamically allocate attention resources based on token importance, drastically lowering the computational overhead without compromising long-context understanding or reasoning depth. With DSA, we scale the model parameters up to 744B and extend the training token budget to 28.5T tokens.

首先，我们采用了 DSA（DeepSeek 稀疏注意力）[9]，这是一种新颖的架构创新，显著降低了训练和推理的成本。虽然 GLM-4.5 通过标准的 MoE 架构提高了效率，但 DSA 使 GLM-5 能够根据 token 的重要性动态分配注意力资源，大幅降低计算开销，而不会影响长上下文理解和推理深度。借助 DSA，我们将模型参数扩展至 744B，并将训练 token 预算扩展至 28.5T tokens。

Second, we have engineered a new asynchronous reinforcement learning infrastructure. Building on the “slime” framework and the decoupled rollout engines initialized in GLM-4.5, our new infrastructure further decouples generation from training to maximize GPU utilization. This system allows for massive-scale exploration of agent trajectories without the synchronization bottlenecks that previously hampered iteration speed, significantly improving the efficiency of our RL post-training pipeline.

第二，我们构建了一个新的异步强化学习基础设施。基于“slime”框架和 GLM-4.5 中初始化的解耦 rollout 引擎，我们的新基础设施进一步将生成与训练解耦，以最大化 GPU 利用率。该系统使得在大规模范围内探索智能体轨迹成为可能，而无需受到之前限制迭代速度的同步瓶颈，显著提高了我们 RL 训练后的效率。



Sparse Attention Adaption (20B, 200K)

稀疏注意力适配 (20B, 200K)

Figure 5: Overall training pipeline of GLM-5.

图 5：GLM-5 的整体训练流程。

Third, we present novel asynchronous Agent RL algorithms designed to enhance the quality of autonomous decision-making. In GLM-4.5, we utilized iterative self-distillation and outcome supervision to train agents. For GLM-5, we have developed asynchronous algorithms that allow the model to learn from diverse, long-horizon interactions continuously. These algorithms are specif-

ically optimized to improve the model’s planning and self-correction capabilities in dynamic environments, directly contributing to our dominance in real-world coding scenarios.

第三，我们提出了新颖的异步智能体强化学习（Agent RL）算法，旨在提升自主决策的质量。在 GLM-4.5 中，我们利用迭代自蒸馏和结果监督来训练智能体。对于 GLM-5，我们开发了异步算法，使模型能够持续地从多样且长时效的交互中学习。这些算法特别优化了模型在动态环境中的规划和自我修正能力，直接助力我们在现实世界编码场景中的领先地位。

Last, one more technical contribution lies in the fact that, from the first day, GLM-5 is full-stack adapted to Chinese GPU ecosystems. We have successfully completed deep optimization—spanning from underlying kernels to upper-level inference frameworks—across seven mainstream domestic chip platforms, including Huawei Ascend, Moore Threads, Hygon, Cambricon, Kunlunxin, MetaX, and Enflame.

最后，还有一个技术贡献在于，从第一天起，GLM-5 就全面适配了中国 GPU 生态。我们成功完成了深度优化，覆盖了从底层内核到上层推理框架的多个层面，包括华为昇腾、摩尔线程、海光、寒武纪、昆仑芯、MetaX 和恩智浦等七种主流国产芯片平台。

With these advancements, GLM-5 stands not just as a more powerful model but as a more efficient and practical foundation for the next generation of AI agents. We release GLM-5 to the community to further advance the frontier of efficient, agentic general intelligence.

随着这些进展，GLM-5 不仅仅是一个更强大的模型，更是一个更高效且实用的下一代 AI 代理的基础。我们向社区发布 GLM-5，以进一步推动高效、自主通用智能的前沿。

2 Pre-Training

2 预训练

Similar to GLM-4.5, the base model of GLM-5 goes through two stages: pre-training for general language and coding capacity, and mid-training for agentic and long-context capacity. We extend the training token budget for all the training stages of GLM-5, totaling 28.5 trillion tokens for the base model.

与 GLM-4.5 类似，GLM-5 的基础模型也经历了两个阶段：首先是用于一般语言和编码能力的预训练，其次是用于代理能力和长上下文能力的中期训练。我们扩展了 GLM-5 所有训练阶段的训练令牌预算，总计达到 28.5 万亿个令牌。

2.1 Architecture

2.1 架构

Model size scaling. GLM-5 scales to 256 experts and reduces its layer count to 80 to minimize expert parallelism communication overhead. This results in a 744B parameter model (40B active parameters), doubling the total size of GLM-4.5, which utilized 355B total and 32B active parameters.

模型规模扩展。GLM-5 扩展到 256 个专家，并将层数减少到 80 层，以最小化专家并行通信的开销。这使得模型参数达到 744B（其中 40B 为活跃参数），总规模是 GLM-4.5 的两倍。GLM-4.5 的总参数为 355B，活跃参数为 32B。

Multi-latent Attention. By employing reduced key-value vectors, Multi-latent attention (MLA) [24] matches the effectiveness of Grouped-Query Attention (GQA) but offers superior GPU memory savings and faster processing for long-context sequences.

多潜注意力。通过使用缩减的键值向量，多潜注意力（MLA）[24] 在效果上与分组查询注意力（GQA）相当，但为长上下文序列提供了更好的 GPU 内存节省和更快的处理速度。

However, in our experiments with Muon optimizer, we find that MLA with a 576-dimension latent KV-cache cannot match the performance of GQA with 8 query groups (denoted as GQA-8, 2048-

然而，在我们使用 Muon 优化器进行的实验中，我们发现使用 576 维度的潜在 KV 缓存的 MLA 无法达到具有 8 个查询组的 GQA 的性能（记作 GQA-8, 2048-

Table 1: Evaluation results for GQA-8 and variants of MLA.

表 1: GQA-8 及其变体的评估结果。

Dataset	Hellaswag	MMLU	C-Eval	RACE	BBH	GSM8K	HumanEval
GQA-8	77.3	61.2	60.0	79.6	53.3	47.6	38.5
MLA	77.3	61.5	59.7	77.8	48.9	46.2	33.5
MLA + Muon Split	77.8	62.5	62.1	79.9	51.8	45.0	36.7
MLA-256 + Muon Split	77.4	62.0	59.9	79.6	51.3	47.5	36.6

数据集	Hellaswag	MMLU	C-Eval	RACE	BBH	GSM8K	HumanEval
GQA-8	77.3	61.2	60.0	79.6	53.3	47.6	38.5
MLA	77.3	61.5	59.7	77.8	48.9	46.2	33.5
MLA + Muon Split	77.8	62.5	62.1	79.9	51.8	45.0	36.7
MLA-256 + Muon Split	77.4	62.0	59.9	79.6	51.3	47.5	36.6

dimension KV-cache). To overcome the performance gap, we propose an adaptation to the recipe of Muon optimizer in GLM-4.5. In the original recipe, we apply matrix orthogonalization to the up-projection matrices W^{UQ} , W^{UK} , W^{UV} for multi-head queries, keys, and values. Instead, we split these matrices into smaller matrices for different heads and apply matrix orthogonalization to these independent matrices. The method, denoted as Muon Split, enables projection weights for different attention heads to update at different scales. As shown in Table 1, the method effectively improves the performance of MLA to match that of GQA-8. In practice, we also find that with Muon Split, the scale of attention logits of GLM-5 remains stable during pre-training without any clipping strategy.

维度 KV-cache)。为克服性能差距，我们提出对 GLM-4.5 中 Muon 优化器的配方进行适应。在原始配方中，我们对多头查询、键和值的 up-projection 矩阵 W^{UQ} , W^{UK} , W^{UV} 应用矩阵正交化。相反，我们将这些矩阵拆分为不同头的小矩阵，并对这些独立矩阵应用矩阵正交化。该方法称为 Muon Split，使得不同注意力头的投影权重能够在不同尺度上更新。如表 1 所示，该方法有效提升了 MLA 的性能，使其与 GQA-8 的性能相匹配。在实践中，我们还发现，在预训练过程中，使用 Muon Split 时，GLM-5 的注意力 logit 的尺度保持稳定，无需任何截断策略。

Another disadvantage of MLA is its high computational cost during decoding. In decoding, MLA performs a 576-dimensional dot product, higher than the 128-dimensional computation of GQA. While the number of attention heads in DeepSeek-V3 is selected according to the roofline of H800 [60], it is inappropriate for other hardware. Given the Multi-head Attention (MHA) style of MLA during training and prefilling, we increase the head dimension from 192 to 256 and decrease the number of attention heads by 1/3. This keeps the training computation and the number of parameters constant while decreasing the decoding computation. The variant, denoted as MLA-256 in Table 1, matches the performance of MLA under Muon Split.

MLA 的另一个缺点是在解码过程中计算成本较高。在解码过程中，MLA 进行的是 576 维的点积运算，这比 GQA 的 128 维计算更高。虽然 DeepSeek-V3 中的注意力头数量是根据 H800 的 roofline [60] 来选

择的，但这种设置对于其他硬件来说并不合适。鉴于 MLA 在训练和预填充过程中采用的是多头注意力 (MHA) 的风格，我们将头的维度从 192 增加到 256，并将注意力头的数量减少了三分之一。这样在保持训练计算量和参数数量不变的同时，降低了解码计算量。该变体在表 1 中标记为 MLA-256，在 Muon Split 下其性能与 MLA 相匹配。

Multi-token Prediction with Parameter Sharing.

多标记预测与参数共享。

Multi-token prediction (MTP) [13; 25] increases the performance of base models and acts as draft models for speculative decoding [20]. However, during training, to predict the next n tokens, n MTP layers are required. As a result, the memory usage of MTP parameters and the kv cache scales linearly with the number of speculative steps. Instead, DeepSeek-V3 is trained with a single MTP layer and predicts the next 2 tokens during inference. The

多标记预测 (MTP) [13; 25] 提高了基础模型的性能，并作为推测解码的草稿模型 [20]。然而，在训练过程中，为了预测接下来的 n 个标记，需要 n 个 MTP 层。因此，MTP 参数的内存使用和 kv 缓存会随着推测步骤的数量线性增长。相反，DeepSeek-V3 使用单个 MTP 层进行训练，并在推理过程中预测接下来的 2 个标记。

Table 2: Comparison of accept lengths of DeepSeek-V3.2 and GLM-5.

表 2: DeepSeek-V3.2 与 GLM-5 接受长度的对比

Model	Accept Length
DeepSeek-V3.2	2.55
GLM-5	2.76

模型	接受长度
DeepSeek-V3.2	2.55
GLM-5	2.76

training-inference discrepancy reduces the acceptance rate of the second token. Therefore, we propose sharing the parameters of 3 MTP layers during training. This keeps the memory cost of the draft model consistent with DeepSeek-V3 while increasing the acceptance rate. In Table 2, we show that the acceptance length of GLM-5 is longer than DeepSeek-V3.2, given the same number of speculative steps (4) in our private prompt set.

训练与推理之间的差异会降低第二个标记的接受率。因此，我们提出在训练过程中共享 3 个 MTP 层的参数。这使得草稿模型的内存成本与 DeepSeek-V3 保持一致，同时提高接受率。在表 2 中，我们展示了在我们的私有提示集中，给定相同的推测步骤数 (4)，GLM-5 的接受长度比 DeepSeek-V3.2 更长。

2.1.1 Continued Pre-Training with DeepSeek Sparse Attention (DSA)

2.1.1 使用 DeepSeek 稀疏注意力 (DSA) 进行持续预训练

Table 3: Comparison of long-context benchmarks between MLA and DSA base models.

表 3: MLA 和 DSA 基础模型在长上下文基准测试中的比较。

	MQ-NIAH-128k	MV-NIAH-128k	SQuAD-128k	HotpotQA-128k
MLA	100.0	95.5	79.7	66.3
DSA	100.0	97.0	86.0	63.0

	MQ-NIAH-128k	MV-NIAH-128k	SQuAD-128k	HotpotQA-128k
MLA	100.0	95.5	79.7	66.3
DSA	100.0	97.0	86.0	63.0

We use DSA in our training. The core philosophy of DSA [9] is to replace the traditional dense $O(L^2)$ attention—which becomes prohibitively expensive at 128K contexts—with a dynamic, fine-grained selection mechanism. Unlike fixed patterns (like sliding windows), DSA “looks” at the content to decide which tokens are important. What makes DSA particularly interesting from a researcher’s perspective is how it was introduced via Continued Pre-Training from a dense base model. This avoided the “astronomical” cost of training from scratch. The transition follows a two-stage “dense warm-up and sparse training adaptation” strategy. DeepSeek-V3.2-Exp maintains

我们在训练中使用 DSA。DSA [9] 的核心理念是用一种动态、精细的选择机制，取代传统的密集型 $O(L^2)$ 注意力机制——这种机制在 128K 个上下文中变得极其昂贵。与固定模式（如滑动窗口）不同，DSA 会“查看”内容，以决定哪些标记是重要的。从研究人员的角度来看，使 DSA 特别有趣的是它如何通过从密集基础模型进行持续预训练的方式引入。这避免了从头开始训练的“天文数字”成本。这种转变遵循一个两阶段的“密集预热和稀疏训练适应”策略。DeepSeek-V3.2-Exp 保持

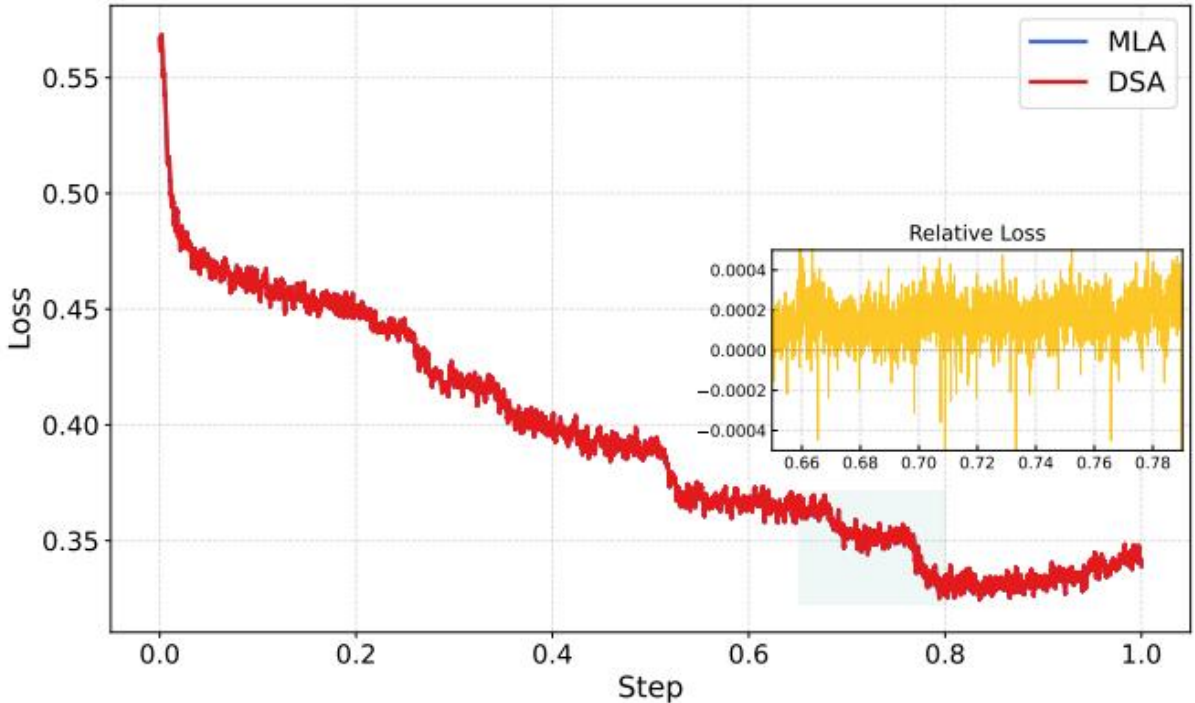


Figure 3: images/6fd479aefbc94174098147abdf1a58f61958b2f1178908fe9a9b771f3fa028c1.jpg

Figure 6: SFT loss curves comparison between MLA and DSA training. Results are smoothed by Running Average with a window size of 50.

图 6: MLA 与 DSA 训练的 SFT 损失曲线对比。结果通过窗口大小为 50 的运行平均值进行平滑处理。

the same benchmark performance as its dense predecessor, proving that 90% of attention entries in long contexts are indeed redundant. DSA reduces the attention computation by roughly 1.5-2 $\times$ for long sequences, which is very important for the reasoning-heavy agents we are building, being able to handle 128K contexts at half the GPU cost.

与密集型前代模型相同的基准性能，证明了在长上下文中 90% 的注意力项确实是冗余的。DSA 在处理长序列时，将注意力计算减少了大约 1.5 到 2 倍，这对于我们正在构建的以推理为主的智能体来说非常重要，能够在一半的 GPU 成本下处理 128K 个上下文。

The DSA training begins from the base model at the end of mid-training. The warm-up stage goes through 1000 steps with each step trained on 14 sequences of 202,752 tokens and a maximum learning rate of 5e-3. The sparse adaptation stage follows the training data and hyperparameters of mid-training and goes through 20B tokens. Although the training budget is much smaller than that of DeepSeek-V3.2 (943.7B tokens), we find that it is enough to adapt the DSA model to match the performance of the original MLA model. As shown in Table 3, the long-context performance of the DSA model is close to that of the MLA model. To further validate the effectiveness of DSA training, we fine-tune the DSA and MLA models with the same SFT data, respectively, and find that the two models tie in training loss and evaluation benchmarks.

DSA 训练从中期训练结束后的基础模型开始。预热阶段经过 1000 步，每一步训练 14 个序列，每个序列包含 202,752 个 token，最大学习率设为 5e-3。稀疏适应阶段遵循中期训练的训练数据和超参数，并经过 20B 个 token。尽管训练预算远小于 DeepSeek-V3.2 (943.7B 个 token)，但我们发现这足以将 DSA 模型适应到与原始 MLA 模型性能相当的水平。如表 3 所示，DSA 模型的长上下文性能接近 MLA 模型。为进一步验证 DSA 训练的有效性，我们分别使用相同的 SFT 数据对 DSA 和 MLA 模型进行微调，并发现两个模型在训练损失和评估基准上表现相当。

2.1.2 Ablation Study of Efficient Attention Variants

2.1.2 高效注意力变体的消融研究

Beyond DSA [26], we explore several alternative efficient attention mechanisms based on GLM-9B¹. The baseline employs group query attention across all 40 layers and has been fine-tuned with a 128K-token context window. We evaluate the following approaches:

超越 DSA [26]，我们基于 GLM-9B¹ 探索了几种替代的高效注意力机制。基线模型在所有 40 层中使用组查询注意力，并且已经通过 128K token 的上下文窗口进行了微调。我们评估了以下方法：

- Sliding Window Attention (SWA) Interleave: A fixed alternating pattern of full-attention and windowed-attention layers applied uniformly across the network.
- Gated DeltaNet (GDN) [54]: A linear attention variant that replaces the quadratic softmax attention computation with a gated linear recurrence, reducing the computational cost of attention from quadratic to linear in sequence length.
- 滑动窗口注意力 (SWA) 交错：一种固定的交替模式，将全注意力层和窗口注意力层按统一模式应用于整个网络。
- 门控 DeltaNet (GDN) [54]：一种线性注意力变体，用门控线性递归替代二次方 softmax 注意力计算，将注意力计算成本从序列长度的二次方降低到线性。

Building on these baselines, we propose two improvements:

基于这些基线，我们提出两个改进方法：

- SWA Pattern (Search-Based): Inspired by PostNAS [15], we introduce a search-based adaptation method that identifies the optimal subset of layers for SWA conversion while retaining full attention in the remaining layers. We employ a beam search strategy to determine the configuration that maximizes performance on long-context downstream tasks. To mitigate computational costs, we conduct the search exclusively at a 16K context length and generalize the resulting pattern to all other input lengths. Specifically, we use a beam size of 8, optimizing two layers per step; for GLM-9B (40 layers), the process converges in approximately 10 steps. At each step, candidate patterns are evaluated on the RULER benchmark [17] at 16K context length, and the top-8 candidates are retained for the subsequent step. The final derived pattern is SFSSFFSSSFFFFSFSFFFFFSFSFSFSFSFSFSFS, where S and F denote SWA and full-attention layers, respectively. As shown in Table 4, this search-based configuration significantly outperforms the fixed interleaved approach. Notably, despite being optimized only at 16K,

- SWA 模式（基于搜索的模式）：受 PostNAS [15] 的启发，我们引入了一种基于搜索的适应方法，该方法在进行 SWA 转换时识别出最优的层子集，同时保留剩余层的完整注意力。我们采用一种束搜索策略来确定在长上下文下游任务中性能最佳的配置。为了降低计算成本，我们仅在 16K 上下文长度上进行搜索，并将得到的模式推广到所有其他输入长度。具体来说，我们使用一个束大小为 8 的策略，每一步优化两层；对于 GLM-9B（40 层）来说，该过程大约在 10 步内收敛。在每一步中，候选模式在 16K 上下文长度的 RULER 基准测试 [17] 上进行评估，并保留排名前 8 的候选模式用于下一步。最终得到的模式为 SFSSFFSSSFFFFSFSFFFFFSFSFSFSFSFSFSFS，其中 S 和 F 分别表示 SWA 层和完整注意力层。如表 4 所示，这种基于搜索的配置显著优于固定的交错方法。值得注意的是，尽管该模式仅在 16K 上下文长度上进行优化，

the pattern exhibits robust length generalization, maintaining effective across all tested context lengths.

该模式表现出强大的长度泛化能力，在所有测试的上下文长度中均保持有效。

- SimpleGDN: A minimalist linearization strategy designed for maximal reuse of pre-trained weights, improving upon GDN for continual-training adaptation. We remove the Conv1d and explicit gating modules entirely and instead directly map the pre-trained Query, Key, and Value projection weights into the linear recurrence formulation. This simplification eliminates the need for additional parameters while preserving the efficiency benefits of linear attention.

- SimpleGDN：一种极简的线性化策略，旨在最大限度地重用预训练的权重，改进了 GDN 在持续训练适应中的表现。我们完全移除了 Conv1d 和显式的门控模块，而是直接将预训练的 Query、Key 和 Value 投影权重映射到线性递归公式中。这种简化消除了对额外参数的需求，同时保留了线性注意力的效率优势。

Table 4: RULER benchmark results for the GLM-9B baseline and two SWA variants without any additional training. Both SWA methods use a 1:1 ratio of full-attention to SWA layers with a 4096-token window size. The search-based SWA pattern is discovered once at 16k context length and applied uniformly across all input lengths.

表 4：GLM-9B 基线模型和两种无需额外训练的 SWA 变体在 RULER 基准测试中的结果。两种 SWA 方法均采用全注意力与 SWA 层 1:1 的比例，窗口大小为 4096 个标记。基于搜索的 SWA 模式在 16k 上下文长度处被发现，并在所有输入长度上统一应用。

	4K	8K	16K	32K	64K	128K
GLM-9B (Full Attn)	95.19	93.67	92.01	91.09	85.35	75.28
SWA Interleave	94.87	54.02	25.89	12.61	8.32	6.51
SWA Pattern	95.78	92.54	88.92	82.52	70.23	53.95

	4K	8K	16K	32K	64K	128K
GLM-9B (Full Attn)	95.19	93.67	92.01	91.09	85.35	75.28

SWA Interleave	94.87	54.02	25.89	12.61	8.32	6.51
SWA Pattern	95.78	92.54	88.92	82.52	70.23	53.95

We evaluate all methods on four long-context benchmarks: RULER [17], MRCR ², HELMET-ICL [56], and RepoQA [27]. Results are summarized in Table 5. We continually train each method on 190B tokens with a 64K context length, maintaining a 1:1 ratio between efficient attention layers and full attention layers. For the GDN and SimpleGDN methods, we follow the Jet-Nemotron [15] pipeline.

我们在四个长上下文基准测试上评估所有方法：RULER [17]、MRCR ²、HELMET-ICL [56] 和 RepoQA [27]。结果汇总在表 5 中。我们持续使用 190B 个 token 进行每种方法的训练，上下文长度为 64K，保持高效注意力层和完整注意力层之间的比例为 1:1。对于 GDN 和 SimpleGDN 方法，我们遵循 Jet-Nemotron [15] 的流程。

Table 5: Long-context benchmark results. All efficient attention variants are continual-trained from the GLM-9B full-attention baseline. SWA pattern denotes search-based layer selection; SWA interleave denotes the fixed alternating pattern. $\Delta@64K$ and $\Delta@128K$ show the difference relative to the full-attention baseline at 64K and 128K context lengths, respectively.

表 5：长上下文基准结果。所有高效的注意力变体均从 GLM-9B 全注意力基线进行持续训练。SWA 模式表示基于搜索的层选择；SWA 交错表示固定的交替模式。 $\Delta@64K$ 和 $\Delta@128K$ 分别表示在 64K 和 128K 上下文长度下相对于全注意力基线的差异。

	RULER (64K/128K)	MRCR (64K/128K)	HELMET-ICL (64K/128K)	RepoQA (64K/128K)
GLM-9B	85.35/75.28	36.53/35.39	77.68/77.36	69.00/65.83
SWA Interleave	65.94/44.93 (↓ 19.41/ ↓ 30.35)	30.03/28.83 (↓ ↓ 6.50/ ↓ 6.56)	75.96/63.52 (↓ 1.72/ ↓ 13.84)	50.33/39.33 (↓ 18.67/ ↓ 26.50)
SWA Pattern	83.72/69.59 (↓ ↓ 1.63/ ↓ 5.69)	35.02/33.58 (↓ ↓ 1.51/ ↓ 1.81)	76.48/74.60 (↓ ↓ 1.20/ ↓ 2.76)	62.33/51.17 (↓ 6.67/ ↓ 14.66)
GDN	76.76/64.00 (↓ 8.59/ ↓ 11.28)	31.72/30.22 (↓ ↓ 4.81/ ↓ 5.17)	76.88/74.84 (↓ ↓ 0.80/ ↓ 2.52)	65.50/56.17 (↓ ↓ 3.50/ ↓ 9.66)
SimpleGDN	81.76/67.03 (↓ ↓ 3.59/ ↓ 8.25)	33.03/31.27 (↓ ↓ 3.50/ ↓ 4.12)	79.80/81.84 (↓ ↑ 2.12/ ↑ 4.48)	65.50/58.50 (↓ ↓ 3.50/ ↓ 7.33)

	RULER (64K/128K)	MRCR (64K/128K)	头盔-ICL (64K/128K)	RepoQA (64K/128K)
GLM-9B	85.35/75.28	36.53/35.39	77.68/77.36	69.00/65.83
SWA Interleave	65.94/44.93 (↓ 19.41/ ↓ 30.35)	30.03/28.83 (↓ ↓ 6.50/ ↓ 6.56)	75.96/63.52 (↓ 1.72/ ↓ 13.84)	50.33/39.33 (↓ 18.67/ ↓ 26.50)
SWA 模式	83.72/69.59 (↓ ↓ 1.63/ ↓ 5.69)	35.02/33.58 (↓ ↓ 1.51/ ↓ 1.81)	76.48/74.60 (↓ ↓ 1.20/ ↓ 2.76)	62.33/51.17 (↓ 6.67/ ↓ 14.66)

GDN	76.76/64.00 (↓ 8.59/ ↓ 11.28)	31.72/30.22 (↓ 4.81/ ↓ 5.17)	76.88/74.84 (↓ 0.80/ ↓ 2.52)	65.50/56.17 (↓ 3.50/ ↓ 9.66)
简单 GDN	81.76/67.03 (↓ 3.59/ ↓ 8.25)	33.03/31.27 (↓ 3.50/ ↓ 4.12)	79.80/81.84 (↑ 2.12/ ↑ 4.48)	65.50/58.50 (↓ 3.50/ ↓ 7.33)

The results in Table 5 reveal a clear trade-off hierarchy among efficient attention methods. Naively interleaved sliding window attention (SWA) causes catastrophic degradation on long-context tasks (e.g., -30.35 on RULER@128K), while search-based layer selection substantially narrows this gap by preserving full attention where it matters most. Linear attention variants such as GDN further improve quality but at the cost of additional parameters; SimpleGDN strikes the best balance by maximally reusing pre-trained weights. Nevertheless, all of these methods incur an inherent accuracy gap on fine-grained retrieval tasks—up to 5.69 points on RULER@128K and 7.33 on RepoQA@128K—due to the unavoidable information loss introduced by efficient attention mechanisms during continual-training adaptation, even when half of the layers retain full attention. In contrast, DSA is lossless by construction: its lightning indexer achieves token-level sparsity without discarding any long-range dependencies, enabling application to all layers with no quality degradation.

表 5 中的结果揭示了高效注意力方法之间明显的权衡层次结构。简单交错的滑动窗口注意力 (SWA) 在长上下文任务中会导致灾难性的性能下降 (例如, 在 RULER@128K 上下降-30.35), 而基于搜索的层选择通过在最关键的地方保留完整注意力, 显著缩小了这一差距。如 GDN 等线性注意力变体进一步提高了质量, 但代价是增加了额外的参数; SimpleGDN 通过最大程度地复用预训练权重, 达到了最佳的平衡。然而, 所有这些方法在细粒度检索任务中都会导致固有的准确率差距——在 RULER@128K 上高达 5.69 点, 在 RepoQA@128K 上高达 7.33 点——这是由于在持续训练适应过程中, 高效注意力机制不可避免地引入了信息损失, 即使有一半的层保留完整注意力。相比之下, DSA 在设计上是无损的: 其闪电索引器在不丢弃任何长距离依赖关系的情况下实现了逐标记的稀疏性, 使得其可以应用于所有层而不会导致质量下降。

To verify this, we conduct a small-scale DSA experiment on GLM-4.7-Flash³ with multi-latent attention. Following the standard DSA recipe, training proceeds in two stages: (i) a warmup phase that trains only the indexer for 1,000 steps (batch size 16) while keeping all base-model weights frozen, followed by (ii) a joint-training phase in which both the model and the indexer are co-trained on 150B tokens. Table 6 summarizes the results on RULER across context lengths from 4K to 128K. Even the warmup-only variant (GLM-4.7-Flash + DSA warmup) already preserves the vast majority of baseline performance; the drop is modest and concentrated at the longest context window (128K: 79.21 $\square$ 71.35), while shorter contexts remain virtually unaffected. After the full 150B-token

为了验证这一点, 我们在 GLM-4.7-Flash³ 上进行了一项小规模 DSA 实验, 使用多潜在注意力机制。按照标准的 DSA 流程, 训练分为两个阶段: (i) 预热阶段, 仅训练索引器, 持续 1,000 步 (批次大小为 16), 同时保持所有基础模型权重冻结; 随后是 (ii) 联合训练阶段, 模型和索引器共同在 150B 个 token 上进行训练。表 6 总结了在不同上下文长度 (从 4K 到 128K) 上的 RULER 结果。即使仅预热的变体 (GLM-4.7-Flash + DSA 预热) 也已经保留了基线性能的绝大部分; 性能下降幅度较小, 主要集中在最长上下文窗口 (128K: 79.21 $\square$ 71.35), 而较短的上下文几乎不受影响。在完成完整的 150B token 训练后

Table 6: RULER benchmark results for the GLM-4.7-Flash with DSA. The warmup-only variant trains only the indexer while keeping the base model frozen, the full DSA variant jointly trains both for 150B tokens.

表 6: GLM-4.7-Flash 在 RULER 基准上的结果, 使用 DSA。仅预热的变体仅训练索引器, 同时保持基础模型冻结, 完整的 DSA 变体则联合训练两者, 共训练 1500 亿个 token。

	4K	8K	16K	32K	64K	128K
GLM-4.7-Flash	97.44	96.72	95.83	92.96	85.34	79.21

GLM-4.7-Flash + DSA warmup	97.51	96.54	95.40	90.09	84.05	71.35
GLM-4.7-Flash + DSA	96.77	96.25	96.69	93.45	87.06	78.86

	4K	8K	16K	32K	64K	128K
GLM-4.7-Flash	97.44	96.72	95.83	92.96	85.34	79.21
GLM-4.7-Flash + DSA warmup	97.51	96.54	95.40	90.09	84.05	71.35
GLM-4.7-Flash + DSA	96.77	96.25	96.69	93.45	87.06	78.86

joint-training phase, GLM-4.7-Flash + DSA closes nearly all of this residual gap: it surpasses the baseline at 16K (+0.86), 32K (+0.49), and 64K (+1.72), while incurring only a 0.35-point deficit at 128K.

联合训练阶段，GLM-4.7-Flash + DSA 几乎填补了所有剩余的差距：它在 16K (+0.86)、32K (+0.49) 和 64K (+1.72) 时超越了基线，而在 128K 时仅落后 0.35 分。

2.2 Pre-training Data

2.2 预训练数据

Web. Building upon the GLM-4.5 data pipeline, we refined our selection criteria for massive web datasets. We introduced another DCLM [21] classifier based on sentence embeddings to identify and aggregate additional high-quality data beyond standard classifiers. To address the challenge of long-tail knowledge, we utilized a World Knowledge classifier—optimized via Wikipedia entries and LLM-labeled data—to distill valuable information from otherwise medium-low-quality data.

Web. 基于 GLM-4.5 数据管道，我们优化了对大规模网络数据集的选择标准。我们引入了另一个基于句子嵌入的 DCLM [21] 分类器，以识别并整合超出标准分类器之外的高质量数据。为了解决长尾知识的挑战，我们使用了世界知识分类器——通过维基百科条目和 LLM 标注数据进行优化——从原本中低质量的数据中提炼出有价值的信息。

Code. We expand the code pre-training corpus with refreshed snapshots from major code hosting platforms and a larger collection of code-containing web pages, resulting in a 28% increase in fuzzily deduplicated unique tokens. To improve corpus integrity and reduce noise, we fix metadata alignment issues in Software Heritage code files and adopt a more accurate language classification pipeline. We follow GLM-4.5’s quality-aware sampling strategy for source code and code-related web documents. In addition, we train dedicated classifiers for a broader set of low-resource programming languages (e.g., Scala, Swift, Lua, etc.), improving sampling quality for these languages.

代码。 我们通过从主要代码托管平台获取更新的快照以及更大规模包含代码的网页集合，扩展了代码预训练语料库，从而实现了模糊去重后唯一标记数量的 28% 增长。为了提高语料库的完整性并减少噪声，我们修复了 Software Heritage 代码文件中的元数据对齐问题，并采用了更精确的语言分类流程。我们遵循 GLM-4.5 的质量感知采样策略，用于源代码及相关的网页文档。此外，我们还训练了专门的分类器，以支持更广泛的低资源编程语言（例如 Scala、Swift、Lua 等），从而提升这些语言的采样质量。

Math & Science. We collect high-quality math & science data from webpages, books, and papers to further increase the reasoning abilities. Specifically, the content extraction pipelines for webpages and PDF parsing mechanisms for books and papers are refined to increase data quality. We adopt large language models to score candidate documents and only retain the most educational content. For long-context documents, we develop a chunk-and-aggregate scoring algorithm to

increase scoring accuracy. Filtering pipelines are conducted to strictly avoid the use of synthetic, AI-generated, or template-based data.

数学与科学。我们从网页、书籍和论文中收集高质量的数学与科学数据，以进一步提升推理能力。具体而言，我们优化了网页内容提取流程和书籍与论文的 PDF 解析机制，以提高数据质量。我们采用大型语言模型对候选文档进行评分，并只保留最具教育意义的内容。对于长文本文档，我们开发了一种分块与聚合评分算法，以提高评分的准确性。我们还实施了过滤流程，严格避免使用合成数据、AI 生成数据或基于模板的数据。

2.3 Mid-Training

2.3 中期训练

Building upon the mid-training framework introduced in GLM-4.5, we scale up both the training volume and the maximum context length in GLM-5 to further strengthen the model’s reasoning, long-context, and agentic capabilities.

在 GLM-4.5 中引入的中期训练框架的基础上，我们在 GLM-5 中进一步扩大了训练量和最大上下文长度，以进一步增强模型的推理能力、长上下文处理能力和自主性能力。

Extended context and training scale. We progressively extend the context window across three stages: 32K (1T tokens), 128K (500B tokens), and 200K (50B tokens). Compared to the 128K maximum in GLM-4.5, the additional 200K stage substantially improves the model’s ability to process ultra-long documents and complex multi-file codebases. Long documents and synthetic agent trajectories are up-sampled at the later stages accordingly.

扩展的上下文和训练规模。我们逐步扩展上下文窗口，分为三个阶段：32K（1T tokens）、128K（500B tokens）和 200K（50B tokens）。与 GLM-4.5 中的 128K 最大值相比，额外的 200K 阶段显著提升了模型处理超长文档和复杂多文件代码库的能力。相应地，在后期阶段，长文档和合成代理轨迹会被进行上采样。

Software engineering data. We retain the paradigm of concatenating repo-level code files, commit diffs, GitHub issues, pull requests, and relevant source files into unified training sequences. In GLM-5, we relax the repository-level filtering criteria to broaden the pool of eligible repositories, yielding approximately 10 million issue-PR pairs, while strengthening quality filtering at the individual issue level to reduce noise. We also retrieve a larger set of relevant files for each issue-PR pair, resulting in richer development contexts and broader coverage of real-world software engineering scenarios. After filtering, the issue-PR portion of the dataset comprises approximately 160B unique tokens.

软件工程数据。我们保留将仓库级别的代码文件、提交差异、GitHub 问题、拉取请求以及相关源文件连接成统一训练序列的范式。在 GLM-5 中，我们放宽了仓库级别的筛选标准，以扩大合格仓库的池子，从而获得大约 1000 万个问题-拉取请求对，同时加强了在单个问题层面的质量筛选，以减少噪声。我们还为每个问题-拉取请求对检索了更大数量的相关文件，从而获得了更丰富的开发上下文，并覆盖了更多现实中的软件工程场景。经过筛选后，数据集中的问题-拉取请求部分包含大约 1600 亿个唯一标记。

Long-context data. Our long-context training set comprises both natural and synthetic data. Natural data is curated from books, academic papers, and documents from general pre-training corpora employing multi-stage filtering (PPL, deduplication, length) and upsampling knowledge-intensive domains. In synthetic data construction, inspired by NextLong[11] and EntropyLong[18], we employed diverse techniques to build long-range dependencies. Highly similar texts were aggregated via interleaved packing to produce sequences, aiming to mitigate the lost-in-the-middle phenomenon and improve performance across a range of long-context tasks. At the 200K stage, we additionally incorporated a small proportion of MRCR-like data, with multiple variants designed to extend OpenAI’s original paradigm, to strengthen recall in extended multi-turn dialogues. Empirically, we find that increasing data diversity progressively enhances the model’s long-context performance; notably, a subsequent 200K mid-training stage, building upon the initial 128K phase, further bolstered the model’s performance even within the

128K context window.

长上下文数据。我们的长上下文训练集包括自然数据和合成数据。自然数据是从书籍、学术论文和通用预训练语料库中的文档中整理而来的，采用多阶段过滤（PPL、去重、长度）和对知识密集型领域进行上采样。在合成数据构建过程中，受到 NextLong[11] 和 EntropyLong[18] 的启发，我们采用多种技术来构建长距离依赖关系。通过交错打包方式聚合高度相似的文本以生成序列，旨在缓解中间丢失现象，并提高在各种长上下文任务中的性能。在 200K 阶段，我们还加入了一小部分类似 MRCR 的数据，这些数据有多种变体，旨在扩展 OpenAI 的原始范式，以增强扩展多轮对话中的召回能力。实证发现，增加数据多样性逐步提升了模型的长上下文性能；值得注意的是，在初始 128K 阶段的基础上进行的后续 200K 中期训练阶段，进一步增强了模型在 128K 上下文窗口内的性能。

2.4 Training Infrastructure

2.4 训练基础设施

2.4.1 Memory Efficiency

2.4.1 内存效率

Flexible MTP placement. Under interleaved pipeline parallelism [31], model components are flexibly assigned to stages. The MTP module spans embedding, transformer, and output components. It incurs substantially higher memory usage than other modules, leading to stage-level imbalance. We co-locate the MTP output layer with the main output layer on the final stage to enable parameter sharing, while placing its embedding and transformer components on the preceding stage. This reduces memory pressure on the final stage and improves balance across pipeline ranks.

灵活的 MTP 放置。在交错的流水线并行 [31] 中，模型组件可以灵活地分配到各个阶段。MTP 模块覆盖了嵌入层、Transformer 层和输出层组件。它比其他模块消耗更多的内存，导致阶段间的不平衡。我们将 MTP 的输出层与主输出层共置在最后一个阶段，以实现参数共享，同时将它的嵌入层和 Transformer 组件放置在前一个阶段。这可以降低最后一个阶段的内存压力，并提高流水线各阶段之间的平衡。

Pipeline ZeRO2 gradient sharding. Each pipeline rank maintains multiple stages [31], and naively each stage requires a full gradient buffer for accumulation and optimizer updates. Inspired by ZeRO2 [38], we shard gradients across data-parallel ranks so that each stage stores only a $1/dp$ fraction of the full gradients. In addition, we retain full accumulation buffers for only two stages at a time and reuse them via double buffering. While one stage buffer accumulates gradients over consecutive microbatches, gradient synchronization for the previous stage buffer is performed in parallel. This reduces persistent gradient memory to per-stage sharded buffers plus only two full buffers for rolling accumulation, without additional synchronization overhead in practice.

Pipeline ZeRO2 梯度分片。每个流水线等级维护多个阶段 [31]，并且每个阶段需要完整的梯度缓冲区来进行累积和优化器更新。受 ZeRO2 [38] 的启发，我们将在数据并行等级之间分片梯度，使得每个阶段仅存储完整梯度的 $1/dp$ 部分。此外，我们仅保留两个阶段的完整累积缓冲区，并通过双缓冲技术重复利用它们。当一个阶段缓冲区在连续的微批次上累积梯度时，前一个阶段缓冲区的梯度同步操作则在后台并行执行。这将持久梯度内存减少到每个阶段分片的缓冲区，加上仅用于滚动累积的两个完整缓冲区，而实际操作中无需额外的同步开销。

Zero-redundant communication for the Muon distributed optimizer. Naive Muon implementations all-gather full model parameters on each data-parallel rank, causing transient memory spikes and redundant communication. We restrict all-gather to parameter shards owned by each rank and overlap local computation with shard communication. This eliminates redundant communication and significantly reduces optimizer-related peak memory overhead.

面向 Muon 分布式优化器的零冗余通信。传统的 Muon 实现方式在每个数据并行等级上都会收集完整的

模型参数，导致临时内存峰值和冗余通信。我们限制收集操作仅针对每个等级拥有的参数分片，并将本地计算与分片通信重叠。这消除了冗余通信，并显著降低了与优化器相关的峰值内存开销。

Pipeline activation offloading. During pipeline warmup, forward execution advances ahead of backpropagation, prolonging the lifetime of intermediate activations. We offload the activations to host memory after forward execution and reload them prior to backward execution [58]. Offloading is applied at layer granularity to further reduce peak memory usage. Combined with fine-grained recomputation, this largely eliminates the need to keep activations resident in GPU memory. Offload and reload are scheduled to overlap with computation while avoiding contention with peer-to-peer communication and MoE token routing (dispatch and combination). This substantially reduces the activation memory footprint with near-zero overhead.

流水线激活卸载。在流水线预热阶段，前向执行会领先于反向传播，从而延长中间激活的生命周期。我们在前向执行后将激活卸载到主机内存，并在反向执行前重新加载它们 [58]。以层为粒度进行卸载，可以进一步减少峰值内存使用。结合细粒度重计算，这在很大程度上消除了需要在 GPU 内存中保留激活的必要性。卸载和重新加载的调度与计算重叠，同时避免与点对点通信和 MoE token 路由（分发和组合）发生竞争。这大大减少了激活内存的占用，且开销接近于零。

Sequence-chunked output projection for peak memory reduction. Output projection and cross-entropy loss incur transient memory overhead from storing activations for backpropagation and promoting them to higher precision during loss computation. To reduce this overhead, we partition the input sequence into smaller chunks and compute projection and loss independently on each chunk, completing forward and backward passes and releasing activations before moving on. As a result, peak memory usage decreases as the number of chunks increases. With an appropriate chunk count, this approach alleviates output-layer memory pressure while maintaining performance comparable to unchunked execution.

分块输出投影以减少峰值内存使用。输出投影和交叉熵损失在计算过程中会因存储用于反向传播的激活值以及在损失计算期间将其提升到更高精度而产生暂时性的内存开销。为了减少这种开销，我们将输入序列划分为更小的块，并在每个块上独立地计算投影和损失，完成前向和反向传播并释放激活值后再继续处理下一个块。因此，随着分块数量的增加，峰值内存使用量会减少。通过适当的分块数量，这种方法可以在保持与未分块执行相当的性能的同时，缓解输出层的内存压力。

2.4.2 Parallelism Efficiency

2.4.2 并行效率

Efficient deferred weight gradient computation. To reduce pipeline bubbles, we defer some weight gradient computation of the critical path [37]. Fine-grained deferral with optimized storage and communication overlap improves throughput while keeping memory overhead bounded.

高效延迟权重梯度计算。为了减少流水线气泡，我们延迟了关键路径 [37] 的部分权重梯度计算。通过细粒度的延迟并优化存储和通信重叠，可以在保持内存开销受限的同时提高吞吐量。

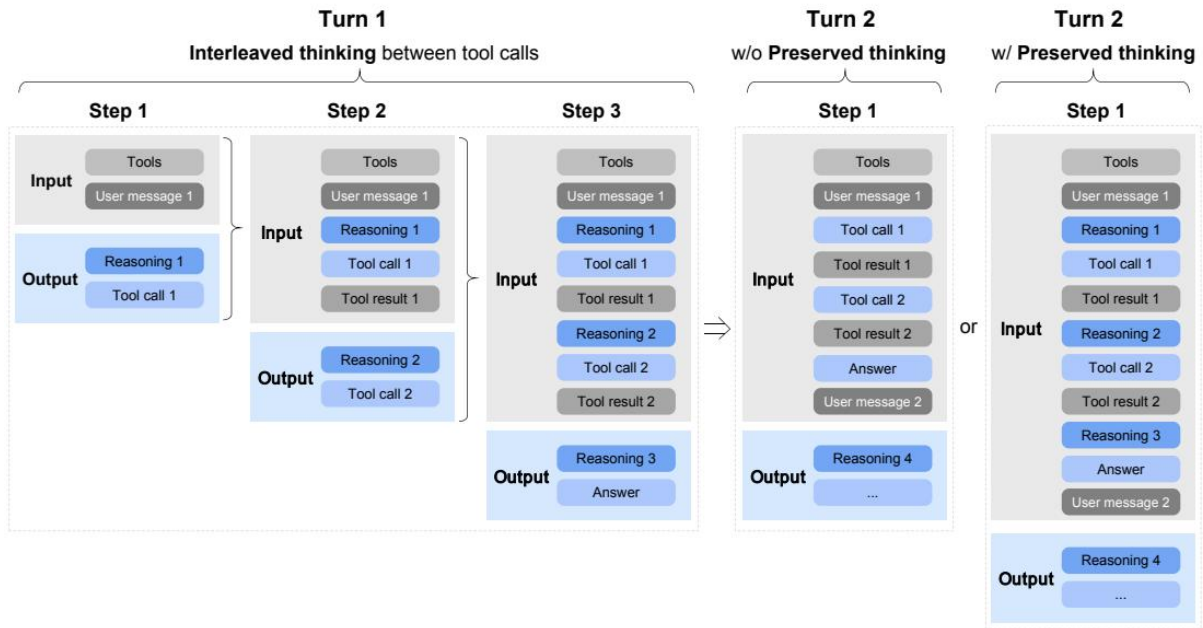


Figure 4: images/a6a72fe4e3538560647d6485af71247eda26c488f7d9a207c957ebf2bab02ca5.jpg

Figure 7: Illustration of Interleaved Thinking and Preserved Thinking.

图 7：交错思考与保留思考的示意图。

Efficient long-sequence training. Longer sequences exacerbate load imbalance across data parallel and pipeline parallel groups. We address this through workload-aware sequence reordering, dynamic redistribution of attention computation, and flexible partitioning of data parallel ranks into context-parallel groups of varying sizes [12; 47]. A hierarchical all-to-all overlaps intra-node and inter-node communication for QKV tensors to reduce latency.

高效长序列训练。更长的序列会加剧数据并行和流水线并行组之间的负载不平衡。我们通过负载感知的序列重排序、注意力计算的动态重新分配，以及将数据并行等级灵活划分为不同大小的上下文并行组来解决这一问题 [12; 47]。一种层次化的全互连方式重叠了节点内和节点间的 QKV 张量通信，以减少延迟。

2.4.3 INT4 Quantization-aware training

2.4.3 INT4 量化感知训练

To provide better accuracy at low-precision, we apply INT4 QAT in the SFT stage. Moreover, to further mitigate the training time overhead, we have developed a quantization kernel applicable to both training and offline weight quantization, which ensures bitwise-identical behavior between training and inference.

为了在低精度下提供更好的准确性，我们在 SFT 阶段应用了 INT4 QAT。此外，为了进一步减轻训练时间的开销，我们开发了一种量化内核，适用于训练和离线权重量化，确保训练和推理之间的位级一致性。

3 Post-Training

3 后训练

The post-training phase of GLM-5 aims to transform the base model into a highly capable assistant with robust reasoning, coding, and agentic abilities. As illustrated in Figure 5, our pipeline follows a progressive alignment strategy: starting with multi-task Supervised Fine-Tuning (SFT) that introduces sophisticated interleaved thinking modes, followed by specialized Reinforcement Learning (RL) stages for reasoning and agentic tasks, and concluding with a general RL stage for human-style alignment. By leveraging on-policy cross-stage distillation as the final refinement, GLM-5 effectively mitigates capability regression while harnessing the performance gains from each training stage.

GLM-5 的训练后阶段旨在将基础模型转变为具备强大推理、编码和自主能力的高效助手。如图 5 所示，我们的流程采用渐进式对齐策略：首先通过多任务监督微调（SFT）引入复杂的交错思考模式，随后进行专门针对推理和自主任务的强化学习（RL）阶段，最后通过一般性的 RL 阶段实现与人类风格的对齐。通过利用基于策略的跨阶段蒸馏作为最终的精细调整，GLM-5 有效缓解了能力退化，同时充分利用了每个训练阶段带来的性能提升。

3.1 Supervised Fine-Tuning

3.1 监督微调

Compared with GLM-4.5, GLM-5 significantly expands the scale of Agent and Coding data during the SFT stage. The SFT corpus of GLM-5 covers three major categories:

与 GLM-4.5 相比，GLM-5 在 SFT 阶段显著扩大了 Agent 和 Coding 数据的规模。GLM-5 的 SFT 语料库涵盖三大类：

- General Chat: question answering, writing, role-playing, translation, multi-turn dialogue, and long-context interactions;
- Reasoning: mathematical, programming, and scientific reasoning;
- Coding & Agent: frontend and backend engineering code, tool calling, coding agents, search agents, and general-purpose agents.
- 通用聊天：问题回答、写作、角色扮演、翻译、多轮对话以及长上下文交互；
- 推理：数学、编程和科学推理；
- 编码与代理：前端和后端工程代码、工具调用、编码代理、搜索代理以及通用代理。

Additionally, GLM-5 extends the maximum context length to 202,752 tokens during SFT. Along with an updated chat template, the model supports three distinct thinking characteristics (see Figure 7), including:

此外，GLM-5 在 SFT 过程中将最大上下文长度扩展至 202,752 个标记。随着更新的对话模板，该模型支持三种不同的思维特征（见图 7），包括：

- Interleaved Thinking: the model thinks before every response and tool call, improving instruction following and the quality of generation⁴.

- Preserved Thinking: in coding agent scenarios, the model automatically retains all thinking blocks across multi-turn conversations, reusing existing reasoning instead of re-deriving it from scratch. This reduces information loss and inconsistencies, and is well-suited for long-horizon, complex tasks⁵.
- Turn-level Thinking: the model supports per-turn control over reasoning within a session—disable thinking for lightweight requests to reduce latency/cost, enable it for complex tasks to improve accuracy and stability.
- 交错思考：模型在每次回复和工具调用前进行思考，提高指令遵循能力和生成质量⁴。
- 保留思考：在编码代理场景中，模型会自动保留多轮对话中的所有思考块，复用已有推理过程，而不是从头开始重新推导。这可以减少信息丢失和不一致性，非常适合长期、复杂的任务⁵。
- 轮次级思考：模型支持在会话中对每一轮推理进行控制——对于轻量级请求可以禁用思考以降低延迟和成本，对于复杂任务则启用思考以提高准确性和稳定性。

By thinking between actions and maintaining consistency across turns, GLM-5 achieves more stable and controllable behavior on complex tasks.

通过在行动之间进行思考并在回合间保持一致性，GLM-5 在复杂任务上实现了更加稳定和可控的行为。

For General Chat, we optimize the response style to be more logical and concise compared to GLM-4.5. For role-playing tasks, we collect and construct a broader and more diverse dataset covering multiple languages and role configurations. In particular, we define several evaluation dimensions—including instruction following, linguistic expressiveness, creativity, logical coherence, and long-dialogue consistency—and apply both automatic and human filtering to curate and refine the data.

对于通用聊天，我们优化了响应风格，使其比 GLM-4.5 更加逻辑清晰和简洁。对于角色扮演任务，我们收集并构建了一个涵盖多种语言和角色配置的更广泛、更多样化的数据集。特别是，我们定义了几个评估维度，包括遵循指令、语言表达性、创造力、逻辑一致性以及长对话的一致性，并通过自动和人工过滤来筛选和优化数据。

For Reasoning tasks, we further enhance the depth of the model’s reasoning. Specifically, for logical reasoning, we construct verifiable problems and synthesize high-quality data using rejection sampling. For mathematical and scientific problems, a difficulty-based filtering process is applied, retaining only problems that are challenging for the GLM-4.7 model.

对于推理任务，我们进一步增强了模型推理的深度。具体来说，对于逻辑推理，我们构建可验证的问题，并使用拒绝采样方法合成高质量的数据。对于数学和科学问题，我们采用基于难度的筛选过程，仅保留对 GLM-4.7 模型具有挑战性的问题。

For Coding and Agent tasks, compared to GLM-4.5, GLM-5 constructs a large number of execution environments to obtain high-quality trajectories, with particular emphasis on real-world scenarios and long-horizon tasks. We further improve the SFT data using expert reinforcement learning and rejection sampling. Erroneous segments within trajectories are retained but masked out in the loss function, allowing the model to learn error correction behaviors without reinforcing incorrect actions.

对于编码和代理任务，与 GLM-4.5 相比，GLM-5 构建了大量执行环境以获取高质量的轨迹，特别强调现实世界场景和长周期任务。我们进一步使用专家强化学习和拒绝采样改进了 SFT 数据。轨迹中的错误片段会被保留，但在损失函数中被屏蔽，从而使模型能够在不强化错误行为的情况下学习错误纠正行为。

3.2 Reasoning RL

3.2 基于推理的强化学习

RL algorithm backbone. Our RL algorithm builds upon GRPO [40] and incorporates the IcePop technique [61] to mitigate the training-inference mismatch, i.e., the discrepancy between the inference distribution and the training distribution during RL optimization. We explicitly distinguish between the training policy π^{train} , used for gradient updates, and the inference policy π^{infer} , used for trajectory sampling. Compared to the original IcePop formulation, we remove the KL regularization term to accelerate RL improvement. The final optimization loss is:

强化学习算法框架。我们的强化学习算法基于 GRPO [40]，并结合了 IcePop 技术 [61] 来缓解训练与推理之间的不匹配问题，即在强化学习优化过程中，推理分布与训练分布之间的差异。我们明确区分了用于梯度更新的训练策略 π^{train} 和用于轨迹采样的推理策略 π^{infer} 。与原始的 IcePop 表达式相比，我们去除了 KL 正则化项以加快强化学习的改进。最终的优化损失函数为：

$$\mathcal{L}(\theta) = -\mathbb{E}_{x \sim \mathcal{D}, \{y_i\}_{i=1}^G \sim \pi_{\theta_{old}}^{infer}(\cdot|x)} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \text{pop}(\rho_{i,t}, 1/\beta, \beta) \cdot \min \left(r_{i,t} \hat{A}_{i,t}, \text{clip}(r_{i,t}, 1 - \epsilon_{low}, 1 + \epsilon_{high}) \hat{A}_{i,t} \right) \right], \quad (1)$$

where the training-inference mismatch ratio is defined as

训练与推理不匹配比率的定义为

$$\rho_{i,t} = \frac{\pi_{\theta_{old}}^{train}(y_{i,t} | x, y_{i,<t})}{\pi_{\theta_{old}}^{infer}(y_{i,t} | x, y_{i,<t})}.$$

The operator $\text{pop}(\cdot)$ suppresses samples whose mismatch ratio deviates excessively:

操作符 $\text{pop}(\cdot)$ 抑制那些不匹配比率偏离过大的样本：

$$\text{pop}(\rho_{i,t}, 1/\beta, \beta) = \begin{cases} \rho_{i,t}, & 1/\beta \leq \rho_{i,t} \leq \beta, \\ 0, & \text{otherwise.} \end{cases}$$

The PPO-style importance ratio and the group-normalized advantage follow the original GRPO definition:

PPO 风格的重要性比率和组归一化优势遵循原始 GRPO 定义：

$$r_{i,t} = \frac{\pi_{\theta}^{train}(y_{i,t} | x, y_{i,<t})}{\pi_{\theta_{old}}^{train}(y_{i,t} | x, y_{i,<t})}, \quad \hat{A}_{i,t} = \frac{R_i - \text{mean}(R_1, \dots, R_G)}{\text{std}(R_1, \dots, R_G)}.$$

During training, we set hyperparameters $\beta = 2$, $\epsilon_{low} = 0.2$, $\epsilon_{high} = 0.28$. Training is performed entirely on-policy with a group size of 32 and a batch size of 32.

在训练过程中，我们设置了超参数 $\beta = 2$, $\epsilon_{low} = 0.2$, $\epsilon_{high} = 0.28$ 。训练完全基于策略进行，组大小为 32，批大小为 32。

DSA RL insights. We conduct a very large-scale RL training on a model based on the DSA architecture. Compared with MLA, DSA introduces an additional indexer that retrieves the top-k most relevant key-value entries and computes attention sparsely over the retrieved subset. The retrieved top-k results are critical for RL stability. This is analogous to how MoE models use routing replay [62] to preserve the activated top-k experts to ensure training-inference consistency. However, directly adapting this strategy to indexer replay, i.e., storing the indexer’s top-k indices at every token position is clearly impractical, since the $k = 2048$ used by the indexer is much larger than the k typically used in MoE, and storing all these indices would incur enormous storage costs as well as significant communication overhead between the training engine and the inference engine.

DSA RL 见解。我们在基于 DSA 架构的模型上进行非常大规模的强化学习训练。与 MLA 相比，DSA 引入了一个额外的索引器，该索引器检索出最相关的前 k 个键值条目，并在检索到的子集上稀疏地计算注意力。检索到的前 k 个结果对于强化学习的稳定性至关重要。这类似于 MoE 模型使用路由回放 [62] 来保留激活的前 k 个专家，以确保训练和推理的一致性。然而，直接将这种策略应用于索引器回放，即在每个 token 位置存储索引器的前 k 个索引显然是不切实际的，因为索引器使用的 $k = 2048$ 明显大于 MoE 通常使用的 k ，存储所有这些索引将导致巨大的存储成本，以及训练引擎和推理引擎之间显著的通信开销。

We find that adopting a deterministic top-k operator effectively resolves the training-inference mismatch in DSA indexer token selection. Compared with the non-deterministic CUDA-based top-k implementation used in SGLang’s DSA Indexer, directly using the naive torch.topk is slightly slower but deterministic. It produces more consistent outputs and yields substantial RL gains. In contrast, other non-deterministic top-k operators (e.g., CUDA or TileLang implementations) caused drastic performance degradation during RL after only a few steps, accompanied by a sharp drop in entropy. Therefore, throughout our RL stages, we use torch.topk as the default top-k operator in the DSA Indexer in our training engine. We also freeze the indexer parameters by default during RL to accelerate training and prevent unstable learning in the indexer.

我们发现采用确定性的 top-k 运算符可以有效解决 DSA 索引器中训练与推理不一致的问题。与 SGLang 的 DSA 索引器中使用的非确定性 CUDA 基础 top-k 实现相比，直接使用简单的 torch.topk 虽然稍慢，但具有确定性。它会产生更一致的输出，并带来显著的强化学习收益。相比之下，其他非确定性的 top-k 运算符（例如 CUDA 或 TileLang 实现）在仅经过几步强化学习后就会导致性能急剧下降，并伴随着熵值的显著下降。因此，在我们的强化学习阶段中，我们默认在 DSA 索引器中使用 torch.topk 作为 top-k 运算符。我们还默认在强化学习过程中冻结索引器参数，以加速训练并防止索引器中的不稳定学习。

Mixed domain reasoning RL. In the Reasoning RL stage, we perform mixed RL training over four domains: mathematics, science, code, and tool-integrated reasoning (TIR). For mathematics and science, we curate data from both open-source datasets [10; 30] and co-developed collections with external annotation vendors. We further apply difficulty filtering to focus training on problems that GLM-4.7 solves correctly only rarely or fails consistently, while remaining solvable by stronger teacher models (e.g., GPT-5.2 xhigh and Gemini 3 Pro Preview). For code, we cover both competitive programming style tasks and scientific coding tasks. The former is primarily sourced from Codeforces and representative datasets such as TACO [23] and SYNTHETIC-2-RL [35], while the latter is constructed from internal problem pools by decomposing questions into the minimal code implementations required for correct solutions. For TIR, we reuse the more challenging subset of mathematics and science RL data, and additionally co-build STEM questions with annotation vendors that are explicitly designed to be answered with external tools. During RL training, we assign domain and source-specific judge models or evaluation systems to produce binary outcome rewards. We keep the overall mixture roughly balanced across the four domains, and consistently observe stable and significant gains in each domain under the mixed RL setting.

混合领域推理强化学习 (RL)。在推理强化学习阶段，我们在四个领域上进行混合 RL 训练：数学、科学、代码和工具集成推理 (TIR)。对于数学和科学领域，我们从开源数据集 [10; 30] 以及外部标注供应商共同开发的数据集中精选数据。我们进一步应用难度过滤，专注于 GLM-4.7 仅偶尔正确解决或始终失败的问题，而这些问题仍能被更强的教师模型（如 GPT-5.2 xhigh 和 Gemini 3 Pro Preview）解决。对于代码领域，我们涵盖竞赛编程风格任务和科学编码任务。前者主要来源于 Codeforces 和代表性的数据集如 TACO [23] 和 SYNTHETIC-2-RL [35]，而后者则是通过将问题分解为正确解答所需的最小代码实现，从

内部问题池中构建而来。对于 TIR，我们重用数学和科学 RL 数据中更具挑战性的子集，并且还与标注供应商共同构建专门设计用于借助外部工具回答的 STEM 问题。在 RL 训练过程中，我们为每个领域和来源分配特定的评判模型或评估系统以生成二元结果奖励。我们保持四个领域整体混合比例大致均衡，并且在混合 RL 设置下，每个领域都持续观察到稳定且显著的提升。

3.3 Agentic RL

3.3 代理强化学习

To facilitate agentic performance of GLM-5, we develop a fully asynchronous and decoupled RL framework and optimize GLM-5 in coding and search agent tasks. Naive synchronous RL suffers from severe GPU idle time during long-horizon agent rollouts. By decoupling inference and training engines via a central Multi-Task Rollout Orchestrator, we achieve high-throughput joint training across diverse agentic workloads.

为了促进 GLM-5 的自主性能，我们开发了一个完全异步且解耦的强化学习框架，并在编码和搜索代理任务中优化 GLM-5。传统的同步强化学习在长周期代理运行过程中会遇到严重的 GPU 空闲时间问题。通过借助一个中心化的多任务运行协调器来解耦推理和训练引擎，我们实现了在多样化的自主任务负载下的高吞吐量联合训练。

To maintain training stability under asynchronous off-policy conditions, we introduce two key mechanisms. First, a Token-in-Token-out (TITO) gateway eliminates re-tokenization mismatches by

为了在异步离线策略条件下保持训练稳定性，我们引入了两种关键机制。首先，一个 Token-in-Token-out (TITO) 网关通过消除重新标记不匹配问题来

preserving exact action-level correspondence. Second, we employ a Direct Double-sided Importance Sampling, which applies a token-level clipping mechanism ($[1 - \epsilon_\ell, 1 + \epsilon_h]$) to rollout log-probabilities, while efficiently controlling off-policy bias without tracking historical policy checkpoints. We also employ a DP-aware routing to maximize KV-cache reuse during long-context inference for large-scale MoE models for speed up. To scaling agentic environments, we scale verifiable training environments across three domains: over 10K real-world Software Engineering (SWE), terminal tasks, and high-difficulty multi-hop search tasks. More details about agentic RL can be found in the subsequent Section 4.

保持精确的行动级对应关系。其次，我们采用直接双面重要性抽样（Direct Double-sided Importance Sampling），该方法在展开对数概率时应用了基于 token 的裁剪机制 ($[1 - \epsilon_\ell, 1 + \epsilon_h]$)，同时高效地控制离策略偏差，而无需跟踪历史策略检查点。我们还采用了一种 DP 感知的路由机制，以在大规模 MoE 模型的长上下文推理过程中最大化 KV 缓存的重用，从而加快推理速度。为了扩展代理环境，我们在三个领域中扩展可验证的训练环境：超过 10,000 个现实世界的软件工程（SWE）、终端任务以及高难度的多跳搜索任务。有关代理强化学习的更多细节，请参见后续的第 4 节。

3.4 General RL

3.4 普通强化学习

Multi-dimensional optimization objectives. We decompose the optimization objectives of General RL into three complementary dimensions: foundational correctness, emotional intelligence, and task-specific quality.

多维优化目标。我们将通用强化学习（General RL）的优化目标分解为三个互补的维度：基础正确性、情感智能和任务特定质量。

The foundational correctness dimension serves as the bedrock of response quality. It targets

a broad spectrum of error types that undermine the usability of model outputs, including instruction-following failures, logical inconsistencies, factual inaccuracies, knowledge hallucinations, and language disfluencies. The goal is to minimize the error rate so that responses reach a usable baseline. We consider this a prerequisite for all subsequent optimization: a response containing factual errors or misinterpreting the user’s intent can actively mislead the user, no matter how polished it may appear.

基础正确性维度是响应质量的基石。它针对一系列会削弱模型输出可用性的错误类型，包括指令执行失败、逻辑不一致、事实错误、知识幻觉和语言不流畅等问题。我们的目标是尽可能降低错误率，使响应达到可使用的基准水平。我们认为这是所有后续优化的前提条件：无论响应多么完美，包含事实错误或误解用户意图的响应都可能主动误导用户。

The emotional intelligence dimension optimizes user experience beyond core correctness. It aims to produce responses that are empathetic, insightful, and stylistically close to natural human communication, making interactions with the model feel more natural and engaging.

情感智能维度超越核心正确性，优化用户体验。其目标是生成富有同理心、有洞察力且在风格上接近自然人类交流的回复，使与模型的互动更加自然和引人入胜。

The task-specific quality dimension targets fine-grained optimization across various specific tasks. Building on the usability established by foundational correctness, it aims to elevate responses from merely correct to genuinely high-quality within each task category. This dimension covers a wide range of tasks, including writing, text processing, subjective and objective question answering, role-playing, and translation. Each task domain demands distinct reward signals, necessitating a hybrid reward system.

任务特定的质量维度旨在在各种具体任务中实现细粒度的优化。在基础正确性建立的可用性基础上，该维度旨在将响应从仅仅正确提升到每个任务类别中的真正高质量。这一维度涵盖了广泛的任务，包括写作、文本处理、主观和客观问题回答、角色扮演和翻译。每个任务领域都需要不同的奖励信号，因此需要一种混合奖励系统。

Hybrid reward system. To supervise the diverse objectives above, we build a hybrid reward system that integrates three complementary types of reward signals: rule-based reward functions, outcome reward models (ORMs), and generative reward models (GRMs). Each has distinct strengths and weaknesses, and their combination is key to a stable, efficient, and scalable General RL training process.

混合奖励系统。为了监督上述多样化的目标，我们构建了一个混合奖励系统，整合了三种互补类型的奖励信号：基于规则的奖励函数、结果奖励模型（ORMs）和生成性奖励模型（GRMs）。每种都有其独特的优缺点，它们的结合对于实现稳定、高效且可扩展的通用强化学习（General RL）训练过程至关重要。

Rule-based rewards provide precise and interpretable signals, but are limited to aspects expressible as deterministic rules. ORM’s offer low-variance signals and high training efficiency, but are more susceptible to reward hacking, where the policy exploits superficial patterns rather than genuinely improving core capability. GRMs leverage language models to produce scalar or structured evaluations and are more robust to such exploitation, but tend to exhibit higher variance. By blending these three signal types, we obtain a reward system that balances precision, efficiency, and robustness, mitigating the weaknesses of any single component.

基于规则的奖励机制提供了精确且可解释的信号，但其适用范围仅限于能够表示为确定性规则的方面。ORMs（对象奖励模型）提供了低方差的信号和高效的训练效果，但更容易受到奖励黑客攻击，即策略利用表面模式而非真正提升核心能力。GRMs（生成式奖励模型）利用语言模型生成标量或结构化的评估，从而更不容易受到此类利用，但往往表现出更高的方差。通过融合这三种信号类型，我们获得了一个在精确性、效率和鲁棒性之间取得平衡的奖励系统，从而缓解了单一组件的局限性。

Human-in-the-loop style alignment. A distinctive aspect of our General RL pipeline is the explicit incorporation of high-quality human-authored responses. Rather than relying solely on model-generated responses, we introduce expert human responses as stylistic and qualitative anchors. This is motivated by the observation that purely model-generated optimization tends to converge

toward recognizably “model-like” patterns—often verbose, formulaic, or lacking the nuance of skilled human writing. By exposing the model to human-written exemplars, we encourage it to adopt more natural, human-aligned response patterns.

人参与的循环风格对齐。我们通用强化学习（General RL）流程的一个显著特点是明确纳入高质量的人类作者回复。我们不仅仅依赖模型生成的回复，而是引入专家人类回复作为风格和质量的基准。这是基于这样一个观察：纯粹由模型生成的优化往往趋向于明显“模型式”的模式——通常冗长、公式化，或缺乏熟练人类写作的细腻之处。通过向模型展示人类撰写的示例，我们鼓励其采用更加自然、与人类对齐的回复模式。

3.5 On-Policy Cross-Stage Distillation

3.5 策略一致的跨阶段知识蒸馏

In our multi-stage RL pipeline, sequentially optimizing for distinct objectives can lead to the cumulative degradation of previously acquired capabilities. To mitigate this issue, we perform on-policy cross-stage distillation as the final stage, adopting an on-policy distillation algorithm [14; 52; 51; 28] to swiftly recover the skills acquired in earlier SFT and RL stages (Reasoning RL and General RL).

在我们的多阶段强化学习（RL）流水线中，按顺序优化不同的目标可能导致之前获得的能力出现累积性退化。为了解决这个问题，我们在最后阶段执行基于策略的跨阶段蒸馏，采用一种基于策略的蒸馏算法 [14; 52; 51; 28]，以快速恢复在早期 SFT 和 RL 阶段（推理 RL 和通用 RL）中获得的技能。

Specifically, the final checkpoints from the preceding training stages serve as teacher models, where the training prompts are sampled from the corresponding teachers’ RL training sets and mixed in appropriate proportions. The training loss can be obtained by replacing the advantage term in Eq. 1 with the following formula (‘sg’ stands for the stop gradient operation, e.g., `.detach()`):

具体而言，前一训练阶段的最终检查点作为教师模型，其中训练提示是从相应教师的强化学习训练集中采样的，并以适当的比例进行混合。可以通过将公式 1 中的优势项替换为以下公式来获得训练损失（‘sg’表示停止梯度操作，例如`.detach()`）：

$$\hat{A}_{i,t} = \text{sg} \left[\log \frac{\pi_{\theta_{\text{teacher}}}^{\text{infer}}(y_{i,t} \mid x, y_{i,<t})}{\pi_{\theta}^{\text{train}}(y_{i,t} \mid x, y_{i,<t})} \right]. \quad (2)$$

Currently, we utilize the inference engine to fetch teachers’ logits. In the future, we plan to migrate the inference backend to the training engine and uniformly adopt the Multi-Query Attention (MQA) mode of MLA for inference ($\pi_{\theta_{\text{teacher}}}^{\text{infer}} \rightarrow \pi_{\theta_{\text{teacher}}}^{\text{train}}$). During training, the group size in the GRPO algorithm is configured to 1 to increase data throughput, and the batch size is set to 1024. This is feasible at this stage because it is no longer necessary to maintain a large group of samples per prompt to estimate advantages; the advantage is computed directly from the gap with the teacher models instead.

目前，我们使用推理引擎来获取教师模型的 logits。未来，我们计划将推理后端迁移到训练引擎，并统一采用 MLA 的多查询注意力（MQA）模式进行推理（ $\pi_{\theta_{\text{teacher}}}^{\text{infer}} \rightarrow \pi_{\theta_{\text{teacher}}}^{\text{train}}$ ）。在训练过程中，GRPO 算法中的组大小被配置为 1 以提高数据吞吐量，批大小设置为 1024。现阶段这样做是可行的，因为不再需要为每个提示维护大量样本以估计优势；优势是直接通过与教师模型之间的差距来计算的。

3.6 RL Training Infrastructure: The slime Framework

3.6 RL 训练基础设施：slime 框架

We continue to use slime as the unified post-training infrastructure for GLM-5, enabling end-to-end reinforcement learning (RL) at scale. Rather than introducing new system components, GLM-5 fully leverages slime’s capabilities to (1) broaden task coverage via free-form rollout customization and a server-based execution model, (2) substantially increase throughput via mixed-precision training/rollouts together with MTP and Prefill-Decode (PD) disaggregation—particularly for multi-turn RL workloads, and (3) improve robustness through heartbeat-driven rollout fault tolerance and router-level server lifecycle management.

我们继续使用 slime 作为 GLM-5 的统一微调基础设施，从而实现大规模端到端强化学习（RL）。与引入新的系统组件不同，GLM-5 充分利用 slime 的能力，通过以下三个方面实现提升：(1) 通过自由形式的 rollout 自定义和基于服务器的执行模型，扩大任务覆盖范围；(2) 通过混合精度训练/rollout 以及 MTP 和 Prefill-Decode (PD) 解耦，显著提高吞吐量——特别是对于多轮 RL 工作负载；(3) 通过心跳驱动的 rollout 故障容错和路由器级别的服务器生命周期管理，提高鲁棒性。

3.6.1 Scaling Out: Flexible Training via Highly Customizable Rollouts

3.6.1 拓展扩展：通过高度可定制的发布实现灵活训练

GLM-5’s post-training spans a diverse spectrum of objectives. To support this diversity without task-specific forks, GLM-5 leverages slime’s highly customizable rollout interface together with its server-based rollout execution.

GLM-5 的微调覆盖了多样化的目标。为了在不进行任务特定分支的情况下支持这种多样性，GLM-5 利用 slime 高度可定制的 rollout 接口，以及基于服务器的 rollout 执行。

Highly customizable rollouts. slime provides a flexible interface for implementing task-specific rollout logic—including multi-turn interaction loops, tool invocation, environment feedback handling, and verifier-guided branching—without modifying the underlying infrastructure. GLM-5 leverages this capability to support a broad range of domains and training paradigms, including but not limited to reasoning RL, general RL, agentic RL, and on-policy distillation, all within a unified training stack.

高度可定制的部署。slime 提供了一个灵活的接口，用于实现任务特定的部署逻辑——包括多轮交互循环、工具调用、环境反馈处理以及由验证器引导的分支逻辑——而无需修改底层基础设施。GLM-5 利用这一能力来支持多种领域和训练范式，包括但不限于推理强化学习、通用强化学习、自主强化学习和在线策略蒸馏，所有这些都在一个统一的训练栈中实现。

Server-based rollouts via HTTP APIs. slime exposes its rollout servers and inference router through standard HTTP APIs, allowing users to interact with slime’s serving layer in the same way as a conventional inference engine. This decouples rollout logic from the training process boundary: external agent frameworks and environments can call the server/router endpoints directly, while the optimization backend remains unchanged for both short-horizon single-turn training and long-horizon multi-turn trajectories.

通过 HTTP APIs 进行基于服务器的发布。slime 通过标准 HTTP APIs 暴露其发布服务器和推理路由器，使用户能够以与传统推理引擎相同的方式与 slime 的服务层进行交互。这将发布逻辑与训练过程的边界解耦：外部代理框架和环境可以直接调用服务器/路由器的端点，而优化后端在短时间范围的单次训练和长时间范围的多轮轨迹中保持不变。

3.6.2 Scaling Up: Tail-Latency Optimization for RL Rollouts

3.6.2 扩展规模：强化学习 rollout 的尾延迟优化

For RL rollouts, the optimization target is not aggregate throughput but end-to-end latency, dominated by the slowest (long-tail) sample in each step. In practice, a single straggling trajectory can stall synchronization points (e.g., batch completion, buffer readiness, trainer updates) and directly determine wall-clock progress. GLM-5 therefore fully leverages slime’s latency-oriented serving and scheduling mechanisms to minimize both median latency and, more importantly, tail latency.

对于强化学习 (RL) 的 rollout，优化目标不是聚合吞吐量，而是端到端延迟，这由每一步中最慢（长尾）的样本主导。在实践中，一条拖后腿的轨迹可能会导致同步点（例如，批量完成、缓冲区就绪、训练器更新）停滞，并直接决定实际时间进度。因此，GLM-5 完全利用 slime 的面向延迟的服务器和调度机制，以最小化中位延迟，更重要的是，最小化尾延迟。

No-queue serving via multi-node inference with DP-attention for MLA. To avoid queueing delays, rollout requests must be served promptly even under bursty traffic, which requires substantial KV-cache capacity. GLM-5 adopts a multi-node inference deployment (e.g., EP64 and DP64 over 8 nodes) to provision sufficient distributed KV-cache. DP-attention is primarily introduced to prevent copying KV across different ranks.

通过多节点推理和 DP-attention 实现无队列服务，用于 MLA。为了避免排队延迟，在突发流量下也必须及时处理请求，这需要大量的 KV 缓存容量。GLM-5 采用多节点推理部署（例如，在 8 个节点上使用 EP64 和 DP64）来提供足够的分布式 KV 缓存。DP-attention 主要引入是为了防止在不同节点之间复制 KV。

Tail-latency reduction with FP8 rollouts and MTP. GLM-5 uses FP8 for rollout inference to reduce per-token latency and shorten the completion time of long trajectories. In addition, GLM-5 leverages slime’s support for Multi-Token Prediction (MTP), which is especially effective under the

使用 FP8 发布和 MTP 进行尾延迟减少。GLM-5 使用 FP8 进行发布推理以减少每标记延迟，并缩短长轨迹的完成时间。此外，GLM-5 利用 slime 对多标记预测（MTP）的支持，这在以下情况下特别有效

small-batch decoding regime typical in RL rollouts. Since tail latency is often driven by small-BS stragglers (e.g., rare long contexts, complex multi-turn reasoning, tool-heavy traces), MTP provides disproportionately large benefits on the long tail, improving the time-to-completion of the slowest sample and thus reducing step-level stall time.

小批量解码机制在强化学习的 rollout 中很常见。由于尾部延迟通常由小 BS 的落后项（例如，罕见的长上下文、复杂的多轮推理、工具密集型轨迹）所驱动，MTP 在长尾部分提供了不成比例的显著优势，提高了最慢样本的完成时间，从而减少了步骤级别的停滞时间。

PD disaggregation to prevent prefill-decode interference in multi-turn RL. In multi-turn settings, long-prefix prefills are frequent (conversation history, tool traces, code context). Under DP-attention, mixing prefill and decode on the same serving resources can create severe interference: a heavy prefill can preempt or disrupt ongoing decodes on the server, preventing other samples from making continuous progress and sharply worsening tail latency. GLM-5, therefore, leverages slime’s Prefill-Decode (PD) disaggregation. By running prefills and decodes on dedicated resources, decodes remain stable and uninterrupted, enabling long-horizon samples to progress continuously and significantly improving tail behavior in multi-turn agentic RL.

PD disaggregation to prevent prefill-decode interference in multi-turn RL. In multi-turn settings, long-prefix prefills are frequent (conversation history, tool traces, code context). Under DP-attention, mixing prefill and decode on the same serving resources can create severe interference: a heavy prefill can preempt or disrupt ongoing decodes on the server, preventing other samples from making continuous progress and sharply worsening tail latency. GLM-5, therefore, leverages slime’s Prefill-Decode (PD) disaggregation. By running prefills and decodes on dedicated resources, decodes remain stable and uninterrupted, enabling long-horizon samples

to progress continuously and significantly improving tail behavior in multi-turn agentic RL.

3.6.3 Rollout Robustness: Heartbeat-Driven Fault Tolerance

3.6.3 发布鲁棒性：基于心跳的容错机制

At scale, transient failures (e.g., individual server crashes, network issues, or performance degradation) are inevitable. GLM-5 leverages slime’s heartbeat-driven fault-tolerance to ensure training continuity under such events: rollout servers periodically emit heartbeats monitored by the orchestration layer, and unhealthy servers are proactively terminated and deregistered from the inference router. As a result, retries are automatically routed away from failed or degraded servers to healthy ones, preventing single-server incidents from interrupting rollouts and preserving uninterrupted end-to-end RL training.

在大规模部署中，瞬时故障（例如单个服务器崩溃、网络问题或性能下降）是不可避免的。GLM-5 利用 slime 的基于心跳的容错机制，在此类事件下确保训练的连续性：rollout 服务器定期发送心跳信号，由编排层监控，不健康的服务器会被主动终止并从推理路由器中注销。因此，重试会自动路由到健康服务器，从而防止单个服务器的故障或性能下降中断 rollout 过程，并保持端到端强化学习训练的连续性。

4 Agentic Engineering

4 代理工程

We describe the transition from vibe coding (human prompting) to agentic engineering. In vibe coding, a human prompts an AI model to write code. In agentic engineering, AI agents write the code themselves. They plan, implement, and iterate. To support these long-horizon tasks, GLM-5 utilizes a fully asynchronous and decoupled RL framework to significantly boost GPU utilization by reducing idle time during agent rollouts. To scaling agent environments, we have developed environment-building pipelines. For coding tasks, we set up real-world software engineering issues and terminal tasks by creating over 10,000 verifiable training scenarios. For search agents, we develop an automatic and scalable complex multi-step reasoning data synthesis pipeline to build data for agentic training.

我们描述了从 vibe 编码（人类提示）向代理工程的转变。在 vibe 编码中，人类通过提示 AI 模型来编写代码。在代理工程中，AI 代理会自行编写代码。它们进行规划、实施和迭代。为了支持这些长期任务，GLM-5 利用了一个完全异步且解耦的强化学习框架，通过减少代理部署期间的空闲时间，显著提高了 GPU 利用率。为了扩展代理环境，我们开发了环境构建流水线。对于编码任务，我们通过创建超过 10,000 个可验证的训练场景，设置了现实世界软件工程问题和终端任务。对于搜索代理，我们开发了一个自动且可扩展的复杂多步推理数据合成流水线，以构建用于代理训练的数据。

4.1 Asynchronous RL for Agentic Tasks

4.1 异步强化学习用于代理任务

To conduct RL for agent tasks, we design a fully asynchronous and decoupled RL infrastructure that efficiently handles long-horizon agent rollouts and supports flexible multi-task RL training across diverse agent frameworks.

为了执行智能体任务的强化学习（RL），我们设计了一个完全异步且解耦的强化学习基础设施，能够高效处理长时域智能体 rollout，并支持在多种智能体框架中灵活进行多任务强化学习训练。

We adopt the group-wise policy optimization algorithm for RL training. For each problem x , we sample K agent traces $\{y_1, \dots, y_K\}$ from the previous policy π_{old} , and optimize the model π_θ with respect to the following objective:

我们采用基于组的策略优化算法进行强化学习训练。对于每个问题 x ，我们从之前的策略 π_{old} 中采样 K 个智能体轨迹 $\{y_1, \dots, y_K\}$ ，并根据以下目标优化模型 π_θ ：

$$L(\theta) = \mathbb{E}_{x \sim \mathcal{D}} \left[\frac{1}{K} \sum_{i=1}^K (r(x, y_i) - \bar{r}(x)) \right],$$

where $\bar{r}(x) = \frac{1}{K} \sum_{i=1}^K r(x, y_i)$ is the mean reward of the sampled responses. It is noted that only model-generated tokens are used for optimization, and the environment feedback is ignored in loss computation.

其中 $\bar{r}(x) = \frac{1}{K} \sum_{i=1}^K r(x, y_i)$ 是所采样响应的平均奖励。需要注意的是，仅使用模型生成的标记进行优化，而在损失计算中忽略了环境反馈。

4.1.1 Asynchronous RL Design for Agentic Training

4.1.1 用于代理训练的异步强化学习设计

Due to the long-tail nature of the rollout process, naive synchronous RL training introduces substantial bubbles during the rollout stage because of the severely imbalanced generation of agentic tasks, which can cause large GPU idle time. To improve training throughput, we adopt a fully asynchronous training paradigm for Agentic RL to boost GPU utilization and training efficiency. Concretely, we decouple the training engine and the inference engine onto different GPU devices. The inference

由于部署过程具有长尾特性，简单的同步强化学习（RL）训练在部署阶段会因代理任务生成严重不平衡而引入大量气泡，导致 GPU 空闲时间较长。为了提高训练吞吐量，我们采用完全异步的代理强化学习（Agentic RL）训练范式，以提升 GPU 利用率和训练效率。具体而言，我们将训练引擎和推理引擎分别部署在不同的 GPU 设备上。推理

engine continuously generates trajectories. Once the number of generated trajectories reaches a predefined threshold, the batch is sent to the training engine to update the model. To reduce policy lag and keep the training approximately on-policy, the model weights used by the rollout engine are periodically synchronized with those of the training engine. The training engine updates the model parameters and pushes the new weights back to the inference engine every K gradient updates. While asynchrony could significantly improve overall training efficiency, it also means that different trajectories may be generated by different versions of the model, introducing a severe off-policy issue. Since the weight update considers a different optimization problem due to the changing rollout policy, we also reset the optimizer after each weight update of the inference engine.

引擎持续生成轨迹。一旦生成的轨迹数量达到预定义的阈值，该批次将被发送到训练引擎以更新模型。为了减少策略滞后并保持训练过程近似在线（on-policy），滚出引擎使用的模型权重会定期与训练引擎的模型权重进行同步。训练引擎在每次 K 次梯度更新后更新模型参数并将新的权重推送到推理引擎。虽然异步性可以显著提高整体训练效率，但也意味着不同的轨迹可能由不同版本的模型生成，从而引入严重的离策略（off-policy）问题。由于权重更新考虑了由于滚出策略变化而不同的优化问题，我们在每次推理引擎的权重更新后也会重置优化器。

Server-based multi-task training design. To address the heterogeneity of trajectory generation in multi-task RL, where different tasks typically rely on distinct tool sets and task-specific rollout logic, we introduce a server-based Multi-Task Rollout Orchestrator for multi-task RL training. This component is designed to ensure seamless compatibility between the slime RL training framework and diverse downstream tasks through a central orchestrator with multiple registered task services. Specifically, each task implements its own rollout and reward logic as an independent microservice, which is registered with the central orchestrator for management and

scheduling. During the rollout stage, the central orchestrator controls the per-task rollout ratio and generation speed to achieve balanced data collection across tasks. Crucially, we standardize trajectories from all agentic tasks into a unified message-list representation. This enables joint training of complex agentic frameworks (e.g., Software Engineering task) while also supporting centralized post-processing and logging for heterogeneous workloads. This design cleanly isolates task-specific logic from the core training loop, enabling seamless integration with multi-task RL training. Serving as the backbone of the GLM-5 training infrastructure, this orchestrator supports over 1k concurrent rollouts and enables automated, dynamic adjustment of task sampling ratios, as well as fine-grained monitoring of task progress.

基于服务器的多任务训练设计。为了解决多任务强化学习中轨迹生成的异构性问题，其中不同的任务通常依赖于不同的工具集和特定任务的 rollout 逻辑，我们引入了一个基于服务器的多任务 rollout 调度器 (Multi-Task Rollout Orchestrator)，用于多任务强化学习的训练。该组件旨在通过一个具有多个注册任务服务的中央调度器，确保 slime 强化学习训练框架与多样化的下游任务之间无缝兼容。具体而言，每个任务将其自身的 rollout 和奖励逻辑实现为一个独立的微服务，并注册到中央调度器以进行管理和调度。在 rollout 阶段，中央调度器控制每个任务的 rollout 比例和生成速度，以实现任务间的均衡数据收集。关键的是，我们将所有自主任务的轨迹标准化为统一的消息列表表示形式。这使得能够联合训练复杂的自主框架（例如，软件工程任务），同时也支持对异构工作负载进行集中式后处理和日志记录。该设计清晰地将任务特定逻辑与核心训练循环分离，从而实现与多任务强化学习训练的无缝集成。作为 GLM-5 训练基础设施的骨干，该调度器支持超过 1k 个并发 rollout，并能够实现任务采样比例的自动化、动态调整，以及对任务进度的细粒度监控。

4.1.2 Optimizing Asynchronous Training Stability

4.1.2 优化异步训练的稳定性

Token-in-Token-out vs. Text-in-Text-out. In an RL rollout setting, token-in-token-out (TITO) means the training pipeline consumes the exact tokenization and decoded-token stream produced by the inference engine, and uses it directly to build trajectories for learning. In contrast, text-in-text-out treats the rollout engine as a black box that returns finalized text; the trainer then reconstructs the trajectory by re-tokenizing that text (and often re-deriving boundaries and truncation) before computing losses. This seemingly small choice is consequential: re-tokenization can introduce subtle mismatches in token boundaries, whitespace/normalization handling, truncation, or special-token placement, which in turn can corrupt step alignment between actions and rewards/advantages—especially when rollouts are streamed, truncated, or interleaved across many actors. We find token-in-token-out is critical for asynchronous RL training because it preserves exact action-level correspondence between what was sampled and what is optimized while enabling actors to emit trajectory fragments (token IDs + metadata) immediately without a lossy text round-trip and without waiting for post-hoc re-tokenization on the learner side. In practice, we implement a TITO Gateway that intercepts all generation requests from rollout tasks and records each trajectory’s token IDs and metadata. This design isolates the cumbersome token ID processing from downstream agent rollout logic, while avoiding re-tokenization mismatches during RL training.

Token-in-Token-out 与 Text-in-Text-out。在一个强化学习 rollout 设置中，Token-in-Token-out (TITO) 意味着训练流程会直接使用推理引擎生成的精确的分词结果和解码后的 token 流，用于构建轨迹以进行学习。相比之下，Text-in-Text-out 将 rollout 引擎视为一个黑箱，它返回最终的文本；然后训练器通过重新分词该文本（通常还会重新推导边界和截断）来重建轨迹，再计算损失。这个看似微小的选择却至关重要：重新分词可能会引入细微的不匹配，例如 token 边界、空格/规范化处理、截断或特殊 token 的位置，这会进而破坏动作与奖励/优势之间的对齐——尤其是在 rollout 是流式传输、截断或跨多个 actor 交错进行时。我们发现，Token-in-Token-out 对异步强化学习训练至关重要，因为它在保持采样内容与优化内容之间精确的 action 级别对应的同时，还能使 actor 立即发出轨迹片段（token ID 加上元数据），而无需经历有损的文本往返过程，也无需等待学习者端的后处理重新分词。在实践中，我们实现了一个 TITO 网关，它拦截所有来自 rollout 任务的生成请求，并记录每条轨迹的 token ID 和元数据。这种设计将繁琐的 token ID 处理与下游 agent 的 rollout 逻辑隔离，同时避免了在强化学习训练期间重新分词带来的不匹配问题。

Direct double-sided importance sampling for token clipping. Unlike the synchronous RL training setting in Section 3, in the asynchronous setting, rollout engines may undergo multiple updates during a single trajectory generation, which renders the tracking of exact behavior probabilities $\pi_{\theta_{\text{old}}}$ computationally prohibitive. Otherwise, we have to maintain an extensive history of model checkpoints $\{\pi_{\theta_{\text{old}}^{(1)}}, \dots, \pi_{\theta_{\text{old}}^{(N)}}\}$, which is infeasible in practical implementation.

直接双面重要性采样用于 token clipping。与第 3 节中同步的强化学习训练设置不同，在异步设置中，rollout 引擎可能在生成单个轨迹的过程中经历多次更新，这使得精确跟踪行为概率变得计算上不可行 $\pi_{\theta_{\text{old}}}$ 。否则，我们需要维护大量的模型检查点历史 $\{\pi_{\theta_{\text{old}}^{(1)}}, \dots, \pi_{\theta_{\text{old}}^{(N)}}\}$ ，这在实际实现中是不可行的。

To resolve this, we first employ a simplified token-level importance sampling mechanism that reuses the log-probabilities generated during rollout as a direct behavior proxy. By calculating the importance sampling ratio as $r_t(\theta) = \frac{\pi_{\theta}}{\pi_{\text{rollout}}}$ and discarding the traditional $\pi_{\theta_{\text{old}}}$, we eliminate the computational overhead of separate old-policy inference. Second, we employ a double-sided calibration token-level masking strategy. Instead of the asymmetric clipping used in standard PPO, we restrict the trust region to $[1 - \epsilon_{\ell}, 1 + \epsilon_h]$, where ϵ_{ℓ} and ϵ_h are clipping hyperparameters. Tokens falling outside this interval are entirely masked from gradient computation to prevent instabilities caused by extreme policy

为了解决这个问题，我们首先采用了一种简化的基于 token 级别的重要性采样机制，该机制重用了在 rollout 过程中生成的对数概率作为直接的行为代理。通过将重要性采样比计算为 $r_t(\theta) = \frac{\pi_{\theta}}{\pi_{\text{rollout}}}$ ，并摒弃传统的 $\pi_{\theta_{\text{old}}}$ ，我们消除了单独旧策略推理的计算开销。其次，我们采用了一种双面校准的 token 级掩码策略。与标准 PPO 中使用的非对称裁剪不同，我们将信任区域限制在 $[1 - \epsilon_{\ell}, 1 + \epsilon_h]$ ，其中 ϵ_{ℓ} 和 ϵ_h 是裁剪超参数。落在该区间外的 token 将完全从梯度计算中屏蔽，以防止由极端策略引起的不稳定性

divergence. This shares similarities with the IcePop mechanism [44], yet our strategy is simpler by further removing the $\pi_{\theta_{\text{old}}}$ and achieving more stable training.

发散性。这与 IcePop 机制 [44] 有相似之处，但我们的策略通过进一步去除 $\pi_{\theta_{\text{old}}}$ ，实现了更稳定的训练。

Formally, the optimization objective with token-level clipping can be written as:

正式来说，使用 token 级别裁剪的优化目标可以表示为：

$$L(\theta) = \mathbb{E}_t \left[f(r_t(\theta), \epsilon_{\ell}, \epsilon_h) \hat{A}_t \log \pi_{\theta}(a_t | s_t) \right] \quad (3)$$

In this formulation, the importance sampling ratio $r_t(\theta)$ is computed as:

在这种公式中，重要性采样比率 $r_t(\theta)$ 的计算方式为：

$$r_t(\theta) = \exp(\log \pi_{\theta}(a_t | s_t) - \log \pi_{\text{rollout}}(a_t | s_t)) \quad (4)$$

Stability is further enforced via the calibration function $f(x; \epsilon_{\ell}, \epsilon_h)$:

稳定性通过校准功能 $f(x; \epsilon_{\ell}, \epsilon_h)$ 进一步得到加强：

$$f(x; \epsilon_{\ell}, \epsilon_h) = \begin{cases} x, & \text{if } 1 - \epsilon_{\ell} < x < 1 + \epsilon_h \\ 0, & \text{otherwise} \end{cases} \quad (5)$$

In the experiments, we find that reusing rollout log-probabilities accepts a controlled degree of

off-policy bias to circumvent the need for historical policy tracking while boosting training stability.

在实验中，我们发现重用 rollout 对数概率可以接受一定程度的 off-policy 偏差，从而在不进行历史策略跟踪的情况下提升训练的稳定性。

Dropping off-policy and noisy samples. In asynchronous RL, overly long trajectories can become highly off-policy, which may destabilize training. To filter out these severely off-policy samples, we log the policy weight version used by the rollout engine at generation time. Specifically, for each response we record the sequence of model versions involved, $(w_0, \dots, w_k)$ with $w_0 < \dots < w_k$. Let w' denote the current policy version. We discard a sample if its oldest rollout version is too stale, i.e., if $w' - w_0 > \tau$, where τ is a predefined threshold. This removes trajectories that lag too far behind the current policy.

丢弃离策略样本和噪声样本。在异步强化学习中，过长的轨迹可能会变得高度离策略，这可能会破坏训练的稳定性。为了过滤掉这些严重离策略的样本，我们记录在生成时 rollout 引擎使用的策略权重版本。具体来说，对于每个响应，我们记录涉及的模型版本序列， $(w_0, \dots, w_k)$ 与 $w_0 < \dots < w_k$ 。令 w' 表示当前策略版本。如果样本的最旧 rollout 版本过于陈旧，即如果 $w' - w_0 > \tau$ ，其中 τ 是一个预定义的阈值，我们则丢弃该样本。这会移除那些与当前策略差距过大的轨迹。

Additionally, coding-agent sandboxes can be inherently unstable and may fail for reasons unrelated to the model (e.g., environment crashes). Such failures introduce noisy training signals because they reflect environment instability rather than the model's capability. To mitigate this, we record the failure reason for each sample and exclude samples that fail due to environment collapse. For group-based sampling methods such as GRPO, removing failed samples can leave an incomplete group. In that case, we pad the group by repeating valid samples if the number of valid samples exceeds half of the group size; otherwise, we drop the entire group. This procedure reduces spurious reward noise and improves training stability.

此外，代码代理沙盒可能本质上是不稳定的，并且可能因与模型无关的原因而失败（例如环境崩溃）。此类失败会引入噪声训练信号，因为它们反映了环境的不稳定性，而不是模型的能力。为了解决这个问题，我们记录每个样本的失败原因，并排除因环境崩溃而导致失败的样本。对于基于分组的采样方法（如 GRPO），删除失败样本可能会导致分组不完整。在这种情况下，如果有效样本的数量超过分组大小的一半，我们通过重复有效样本来填充分组；否则，我们丢弃整个分组。这一过程可以减少虚假奖励噪声，并提高训练的稳定性。

DP-aware routing for acceleration. We propose a DP-aware routing mechanism to preserve KV cache locality under Data Parallelism (DP) for large-scale MoE inference. In multi-turn agentic workloads, sequential requests from the same rollout share an identical prefix. To maximize KV reuse, we enforce rollout-level affinity: all requests belonging to a given agent instance are routed to the same DP rank. Concretely, we introduce a stateful routing layer that maps each rollout ID to a fixed DP rank using consistent hashing. This mapping remains stable across turns, eliminating cross-rank cache misses. To prevent long-term imbalance, we combine hashing with lightweight dynamic load rebalancing over the hash space. This design avoids redundant prefill computation without requiring KV synchronization across DP ranks. As rollout length increases, prefill cost remains proportional to incremental tokens rather than total context length. The result is improved end-to-end latency and higher effective throughput for long-context agentic inference.

面向数据并行的路由机制以加速推理。我们提出了一种面向数据并行（DP）的路由机制，以在大规模 MoE 推理中保持 KV 缓存的局部性。在多轮代理工作负载中，来自同一 rollout 的顺序请求共享相同的前缀。为了最大化 KV 重用，我们强制实施 rollout 级别的亲和性：属于同一代理实例的所有请求都被路由到同一个 DP 秩。具体而言，我们引入了一个有状态的路由层，使用一致性哈希将每个 rollout ID 映射到固定的 DP 秩。这种映射在轮次之间保持稳定，从而消除跨秩的缓存未命中。为了避免长期的不平衡，我们结合哈希与轻量级的动态负载再平衡，覆盖整个哈希空间。这种设计避免了冗余的预填充计算，而无需在 DP 秩之间同步 KV。随着 rollout 长度的增加，预填充成本与增量标记成比例，而不是与总上下文长度成比例。结果是提高了长上下文代理推理的端到端延迟和有效吞吐量。

4.2 Environment Scaling for Agents

4.2 代理的环境扩展

To support reinforcement learning across diverse agentic tasks, we construct verifiable, executable environments that provide grounded feedback for both code-centric and content-generation workflows. For agentic coding tasks, we develop two environment-building pipelines that construct verifiable executable environments: an environment setup pipeline built upon real-world software engineering issues, and a synthesis pipeline for terminal-agent environments. Beyond coding, we further introduce a slide generation environment, in which the agent operates over structured HTML with executable rendering and layout-based verification.

为了支持在各种自主任务中进行强化学习，我们构建了可验证、可执行的环境，为以代码为中心和内容生成的工作流程提供基于实际的反馈。对于自主编码任务，我们开发了两个环境构建管道，用于构建可验证的可执行环境：一个基于现实世界软件工程问题的环境设置管道，以及一个用于终端代理环境的合成管道。除了编码之外，我们进一步引入了一个幻灯片生成环境，在该环境中，代理在结构化的 HTML 上运行，具有可执行的渲染和基于布局的验证。

4.2.1 Software Engineering (SWE) Environments

4.2.1 软件工程（SWE）环境

Before constructing executable environments, we collect a large corpus of real-world Issue-Pull Request (PR) pairs and apply rigorous rule-based and LLM-based filtering to ensure the acquisition of

在构建可执行环境之前，我们收集了一个大量真实世界的 Issue-Pull Request (PR) 对话料库，并应用严格的基于规则和基于 LLM 的过滤方法，以确保获取到

authentic, high-quality issue statements. We categorize these instances into different task types—bug fixing, feature implementation, refactoring, and others—and include the necessary task requirements to ensure that the model’s implementation is consistent with the test patch. We employ an environment setup pipeline based on the RepoLaunch [59] framework that scales the construction of executable environments from real-world SWE issues. This pipeline automatically analyzes a repository’s installation and dependency setup to build an executable environment and generate test commands, then leverages LLM to generate language-aware log-parsing functions from test outputs, enabling the extraction of Fail-to-Pass (F2P) and Pass-to-Pass (P2P) test cases. Using this pipeline, we construct over 10k verifiable environments across thousands of repositories spanning 9 programming languages, including Python, Java, Go, C, CPP, JavaScript, TypeScript, PHP, and Ruby.

真实、高质量的问题陈述。我们将这些实例分类为不同的任务类型——错误修复、功能实现、重构和其他，并包含必要的任务要求，以确保模型的实现与测试补丁保持一致。我们采用基于 RepoLaunch [59] 框架的环境设置流水线，该流水线可从现实世界的 SWE 问题扩展构建可执行环境。此流水线会自动分析仓库的安装和依赖设置，构建可执行环境并生成测试命令，然后利用 LLM 从测试输出中生成语言感知的日志解析函数，从而提取出 Fail-to-Pass (F2P) 和 Pass-to-Pass (P2P) 测试用例。通过使用此流水线，我们在涵盖 9 种编程语言（包括 Python、Java、Go、C、CPP、JavaScript、TypeScript、PHP 和 Ruby）的数千个仓库中构建了超过 10,000 个可验证的环境。

4.2.2 Terminal Environments

4.2.2 终端环境

Synthesis from seed data. To build verifiable terminal-agent environments at scale, we design an agentic data synthesis pipeline comprising three phases: task draft generation, concrete task

implementation, and iterative task optimization. Starting from a set of seed tasks collected from real-world software engineering and terminal-based computer-use scenarios, we leveraged LLM to brainstorm and generate a large pool of verifiable terminal-task drafts. These drafts are then instantiated by a construction agent into concrete tasks in the Harbor [42] format, including structured task descriptions, Dockerized execution environments, and corresponding test scripts. Subsequently, a refine agent inspects and iteratively refines the generated tasks according to manually defined rubrics, ensuring that Docker images can be built reliably, test cases are consistent with task specifications, and the environments are robust against potential exploits or shortcuts. Overall, the pipeline yields thousands of diverse and verifiable terminal-agent environments with Docker construction accuracy exceeding 90%.

从种子数据合成。为了大规模构建可验证的终端代理环境，我们设计了一个由三个阶段组成的代理数据合成流水线：任务草案生成、具体任务实现和任务迭代优化。我们从现实世界中的软件工程和基于终端的计算机使用场景中收集一组种子任务，利用大语言模型（LLM）进行头脑风暴并生成大量可验证的终端任务草案。这些草案随后由构建代理实例化为 Harbor [42] 格式的具体任务，包括结构化的任务描述、Docker 化的执行环境以及相应的测试脚本。随后，一个优化代理根据手动定义的标准对生成的任务进行检查和迭代优化，确保 Docker 镜像可以可靠地构建，测试用例与任务规范一致，并且环境能够抵御潜在的漏洞或捷径。总体而言，该流水线生成了数千个多样且可验证的终端代理环境，Docker 构建准确率超过 90%。

Synthesis from web-corpus. We develop a scalable, automated pipeline and construct LLM-verified terminal-based coding tasks based on web corpus, using a closed-loop design where the constructing agent also serves as its own first-pass evaluator. First, we collect a large-scale corpus of code-relevant web pages and apply a data quality classifier to retain only high-quality content, discarding pages that are predominantly non-technical or lack substantive code content. From the filtered subset, we further identify web pages amenable to terminal-style task formulation. We then apply stratified sampling across topic categories and difficulty levels to ensure distributional balance and diversity in the resulting task pool. Second, we prompt a coding agent with the Harbor task construction specification⁶, including the task schema, formatting requirements, and exemplar tasks, alongside each selected source web page. The agent is instructed to (i) synthesize a complete terminal task grounded in the web page content, and (ii) execute the Harbor validation script against its own output. Upon validation failure, the agent iteratively diagnoses and revises the task until it passes all automated checks. Only tasks that successfully clear this self-verification loop are admitted into the final dataset.

从网络语料库中合成。我们开发了一个可扩展的自动化流程，并基于网络语料库构建了经过大型语言模型验证的基于终端的编码任务，采用闭环设计，其中构建代理同时担任其自身的初步评估者。首先，我们收集大量与代码相关的网页语料库，并应用数据质量分类器，仅保留高质量内容，丢弃主要非技术性或缺乏实质性代码内容的页面。从筛选后的子集，我们进一步识别适合以终端风格任务形式编写的网页。然后，我们在主题类别和难度等级上进行分层抽样，以确保最终任务池在分布上平衡且多样化。其次，我们提示编码代理使用 Harbor 任务构建规范⁶，包括任务模式、格式要求和示例任务，同时与每个选定的源网页一起。代理被指示执行以下操作：(i) 基于网页内容合成一个完整的终端任务；(ii) 将 Harbor 验证脚本应用于其自身的输出。在验证失败时，代理会迭代地诊断并修订任务，直到其通过所有自动化检查。只有成功通过这一自我验证循环的任务才会被纳入最终数据集。

4.2.3 Search Tasks

4.2.3 搜索任务

For deep-search information-seeking tasks, we build a data-synthesis pipeline that produces challenging multi-hop QA pairs. Each question requires multi-step reasoning grounded in evidence aggregated from multiple web sources.

对于深度搜索的信息获取任务，我们构建了一个数据合成流水线，生成具有挑战性的多跳问答对。每个问题都需要基于来自多个网络来源的证据进行多步推理。

Web Knowledge Graph (WKG) Construction and Question Generation. Starting from trajectories of an early-stage search agent, we collect and deduplicate all encountered URLs, retaining over

two million high-information web pages across diverse domains. The LLM performs semantic parsing for entity recognition, noise filtering, and structured information extraction. The WKG is continuously updated with new pages and refined using downstream verification signals via entity alignment, attribute normalization, relation consolidation, and semantic-consistency corrections. Based on the WKG, we sample low- to mid-frequency entities as seed nodes and expand their multi-hop neighborhoods to form complete subgraphs, while controlling expansion to reduce overlap. Using prompts targeting high-difficulty, multi-domain reasoning, we convert each subgraph into a question that implicitly encodes multi-entity relational chains.

网络知识图谱（WKG）构建与问题生成。我们从早期搜索代理的轨迹出发，收集并去重所有遇到的 URL，保留了超过两百万个涵盖多种领域的高信息网页。大型语言模型（LLM）执行语义解析，进行实体识别、噪声过滤和结构化信息提取。WKG 通过实体对齐、属性规范化、关系整合和语义一致性校正等下游验证信号，持续更新新页面并进行优化。基于 WKG，我们采样低频至中频实体作为种子节点，并扩展其多跳邻域以形成完整的子图，同时控制扩展以减少重叠。通过针对高难度、多领域推理的提示，我们将每个子图转换为一个问题，该问题隐式编码了多实体关系链。

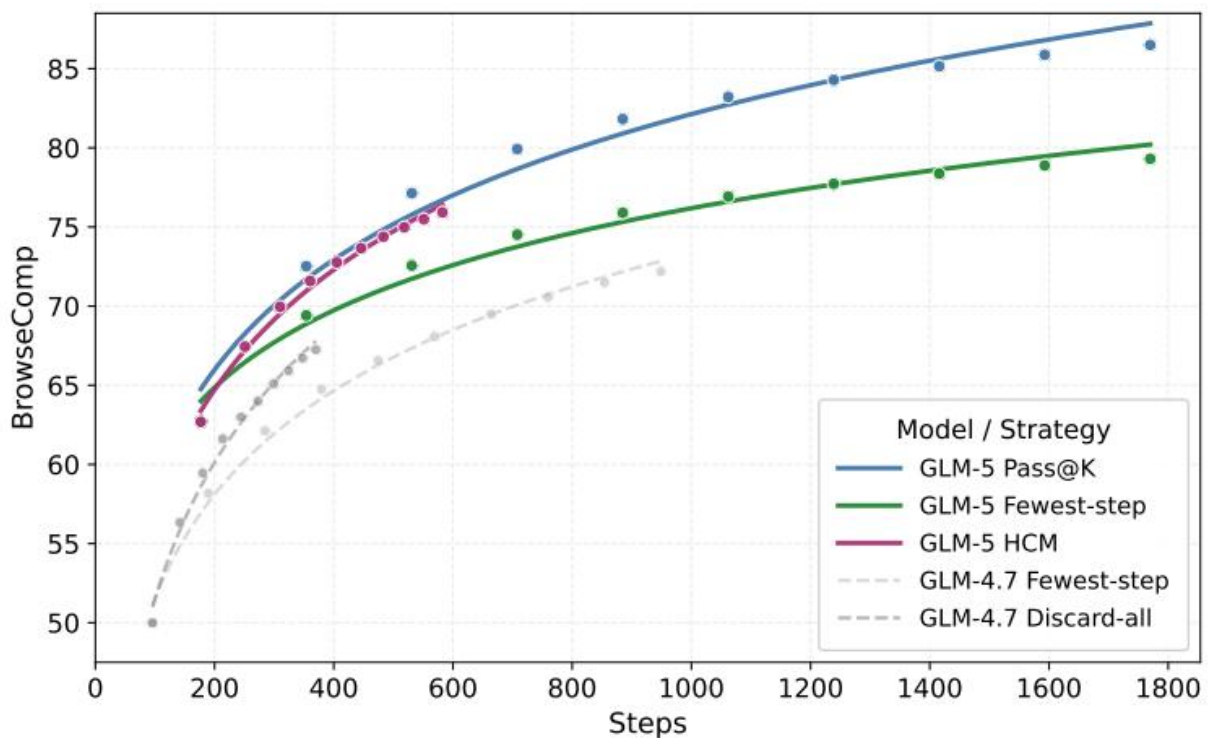


Figure 5: images/c702eb0eecea9cae151745f4e5328696d6bada6a4151c3fb28717af2e36d9b87.jpg

Figure 8: Accuracy of BrowseComp with different context management strategies from GLM-4.7 (gray baselines) to GLM-5 (colored strategies).

图 8: BrowseComp 在不同上下文管理策略下的准确性，从 GLM-4.7（灰色基线）到 GLM-5（彩色策略）。

High-Difficulty Question Filtering and Verification. We apply a three-stage pipeline to balance difficulty and correctness: (1) Remove questions that a tool-free reasoning model correctly answers in at least one of eight independent attempts. (2) Filter out questions solvable by an early-stage agent with basic search, browsing, and computation within a few steps. (3) Apply a verification agent for bidirectional validation: we collect candidate answers from the search trajectories in stage 2, then independently verify the question-answer consistency for both the candidates and the annotated ground truth, rejecting samples with non-unique answers, inconsistent evidence, or incorrect labels. This yields high-quality, high-difficulty, reliable multi-hop QA pairs.

高难度问题筛选与验证。我们采用一个三阶段的流程来平衡难度和正确性：(1) 移除那些在八次独立尝试中至少有一次被无工具推理模型正确回答的问题。(2) 过滤掉那些可以通过早期阶段代理在几步内通过基本搜索、浏览和计算解决的问题。(3) 应用验证代理进行双向验证：我们从第二阶段的搜索轨迹中收集候选答案，然后独立验证候选答案与标注的地面真实值之间的问题-答案一致性，拒绝那些答案不唯一、证据不一致或标签错误的样本。这最终得到高质量、高难度、可靠的多跳问答对。

4.2.4 Inference with Context Management for Search Agents

4.2.4 带有上下文管理的搜索代理推理

We find that the performance on BrowseComp [50] is sensitive to both the judge prompt and the judge model, and open-source judges can introduce systematic bias. To ensure consistency and reproducibility, we standardize all judge-based components using the official OpenAI evaluation prompt and the proprietary model o3-mini as the judge. Our case studies indicate this configuration aligns best with human-annotated ground truth, so we adopt it for all search agent evaluations.

我们发现，在 BrowseComp [50] 上的性能对判断提示和判断模型都比较敏感，开源的判断模型可能引入系统性偏差。为了确保一致性和可重复性，我们使用官方的 OpenAI 评估提示和专有模型 o3-mini 作为判断模型，对所有基于判断的组件进行标准化。我们的案例研究表明，这种配置最符合人工标注的真实情况，因此我们将其用于所有搜索代理的评估。

Prior work [26] has introduced context management, where Discard-all resets the context by removing the entire history of tool calls. We further observe that model accuracy degrades substantially under extremely long contexts (e.g., beyond 100k tokens). Motivated by this, we employ a simple Keep-recent-k strategy. When the interaction history exceeds a threshold k , the content older than the most recent k rounds will be folded to control context length. Let the trajectory be $(q, r_1, a_1, o_1, r_2, a_2, o_2, \dots, r_n, a_n, o_n)$, where q denotes the question, r_i denotes the reasoning at round i , a_i the action (we design search, open, find and python 4 tools), and o_i the tool observation. We fold only observations earlier than the most recent k rounds: $o_i \leftarrow \text{Tool result is omitted to save tokens. } i = 1, \dots, n - k$. In our experiments, we set $k = 5$, which yields a stable improvement and improves GLM-5 from 55.3%(w/o keep-recent-k) to 62.0%(w/ keep-recent-k). We also find that using different values of keep recent k or alternatively triggering keep-recent once the context length reaches a predefined token threshold, leads to the same results.

先前的工作 [26] 引入了上下文管理，其中 Discard-all 通过移除整个工具调用历史来重置上下文。我们进一步观察到，在极长的上下文中（例如超过 10 万个 token），模型的准确性会显著下降。受此启发，我们采用了一种简单的 Keep-recent-k 策略。当交互历史超过阈值 k 时，将比最近 k 轮旧的内容折叠起来以控制上下文长度。设轨迹为 $(q, r_1, a_1, o_1, r_2, a_2, o_2, \dots, r_n, a_n, o_n)$ ，其中 q 表示问题， r_i 表示第 i 轮的推理， a_i 表示动作（我们设计了搜索、打开、查找和 Python 4 种工具）， o_i 表示工具观察结果。我们只折叠早于最近 k 轮的工具观察结果： $o_i \leftarrow \text{工具结果被省略以节省 token. } i = 1, \dots, n - k$ 。在我们的实验中，我们将 k 设置为 5，这带来了稳定的提升，并将 GLM-5 的性能从 55.3%（不使用 keep-recent-k）提高到 62.0%（使用 keep-recent-k）。我们还发现，使用不同的 keep recent k 值或在上下文长度达到预定义的 token 阈值时触发 keep-recent，都会得到相同的结果。

Building on this, we combine keep-recent with Discard-all to form a hybrid Hierarchical Context Management strategy. During inference with keep-recent, if the total context length exceeds a threshold T , we discard the entire tool-call history and restart with a fresh context, while continuing to apply the keep-recent strategy. We select $T = 32k$ via parameter search.

在此基础上，我们将 keep-recent 与 Discard-all 结合，形成一种混合的分层上下文管理策略。在使用 keep-recent 进行推理时，如果总的上下文长度超过阈值 T ，我们将丢弃整个工具调用历史，并以全新的上下文重新开始，同时继续应用 keep-recent 策略。我们通过参数搜索选择了 $T = 32k$ 。

As shown in Figure 8, under different compute budgets, this strategy effectively frees up context space, enabling the model to execute more steps and consistently improving performance. Compared to using Discard-all alone, combining with keep-recent-k achieves consistent gains across

all budgets, reaching a final score of 75.9, outperforming all open-source models equipped with context-management.

如图 8 所示,在不同的计算预算下,该策略有效地释放了上下文空间,使模型能够执行更多步骤,并持续提升性能。与单独使用 Discard-all 相比,结合 keep-recent-k 后,在所有预算下都能实现一致的性能提升,最终得分为 75.9,优于所有配备上下文管理的开源模型。

4.2.5 Slide Generation

4.2.5 幻灯片生成

We employ a self-improving pipeline that aims to systematically enhance slide generation performance by training a specialized slide-generation expert through reinforcement learning and rejection sampling fine-tuning. We first initialize the model with supervised fine-tuning (SFT) to provide a basic slide generation capability, and then perform reinforcement learning with a multi-level reward formulation grounded in common aesthetic and structural properties of presentation slides. This stage leads to substantial improvements in generation quality. We further conduct rejection sampling fine-tuning and mask fine-tuning, allowing knowledge acquired during reinforcement learning to be injected back into the training corpus. This procedure jointly enhances data quality and model capability in a coordinated and iterative manner.

我们采用了一个自我提升的管道,旨在通过强化学习和拒绝采样微调,系统地提升幻灯片生成性能,训练一个专门的幻灯片生成专家。我们首先通过监督微调(SFT)初始化模型,以提供基本的幻灯片生成能力,然后通过基于演示幻灯片常见美学和结构特性的多级奖励公式进行强化学习。这一阶段显著提升了生成质量。我们进一步进行拒绝采样微调 and 掩码微调,使在强化学习过程中获得的知识能够重新注入到训练语料库中。这一过程以协调和迭代的方式共同提升数据质量和模型能力。

We propose a multi-level reward formulation, which partitions reward signals in the HTML-based slide generation process into three levels:

我们提出了一种多级奖励公式,它将基于 HTML 的幻灯片生成过程中奖励信号分为三个层级:

Level-1: Static markup attributes. This level focuses on declarative attributes in the generated HTML, including positioning, spacing, color, typography, saturation, and other stylistic attributes. Grounded in professional design principles, we design a set of rules to regulate the model's behavior when generating such declarations. These rules ensure syntactic parsability of the generated HTML, while constraining the design space at the markup level to a subspace optimized for expressiveness, structural clarity, visual harmony, and readability. Additionally, we introduce hallucinated-image and duplicate-image detection mechanisms to suppress hallucinatory or redundant figures.

Level-1: 静态标记属性。这一层级关注于生成的 HTML 中的声明性属性,包括定位、间距、颜色、排版、饱和度以及其他风格属性。基于专业设计原则,我们设计了一套规则来规范模型在生成此类声明时的行为。这些规则确保生成的 HTML 具有语法可解析性,同时在标记层面上将设计空间限制在一个表达性、结构清晰性、视觉和谐性以及可读性优化的子空间中。此外,我们还引入了幻觉图像和重复图像的检测机制,以抑制幻觉或冗余的图像。

Level-2: Runtime rendering properties. Unlike static inspection, this level evaluates runtime properties of DOM nodes during rendering, such as element width and height, bounding boxes, and other geometric layout metrics. By constraining these properties, we encourage the generated slides to align more closely with human aesthetic preferences in spatial organization. We develop a distributed rendering service capable of executing rendering jobs at high throughput while extracting the required runtime properties. During training, we observe several forms of reward hacking behaviors, such as hard truncation of overlong content or excessive manipulation of spacing (see Figure 9). To mitigate these issues, we refine the renderer implementation to eliminate exploitable loopholes, ensuring that reward signals genuinely incentivize aesthetically coherent layouts rather than superficial compliance with geometric metrics.

Level-2: 运行时渲染属性。与静态检查不同，这一层级在渲染过程中评估 DOM 节点的运行时属性，例如元素的宽度和高度、边界框以及其他几何布局指标。通过约束这些属性，我们鼓励生成的幻灯片在空间组织上更符合人类的审美偏好。我们开发了一个分布式渲染服务，能够在高吞吐量下执行渲染任务，同时提取所需的运行时属性。在训练过程中，我们观察到几种形式的奖励操控行为，例如对过长内容进行硬截断或过度操控间距（参见图 9）。为了解决这些问题，我们优化了渲染器的实现，消除可被利用的漏洞，确保奖励信号真正激励符合审美一致性的布局，而不是仅仅表面上符合几何指标。

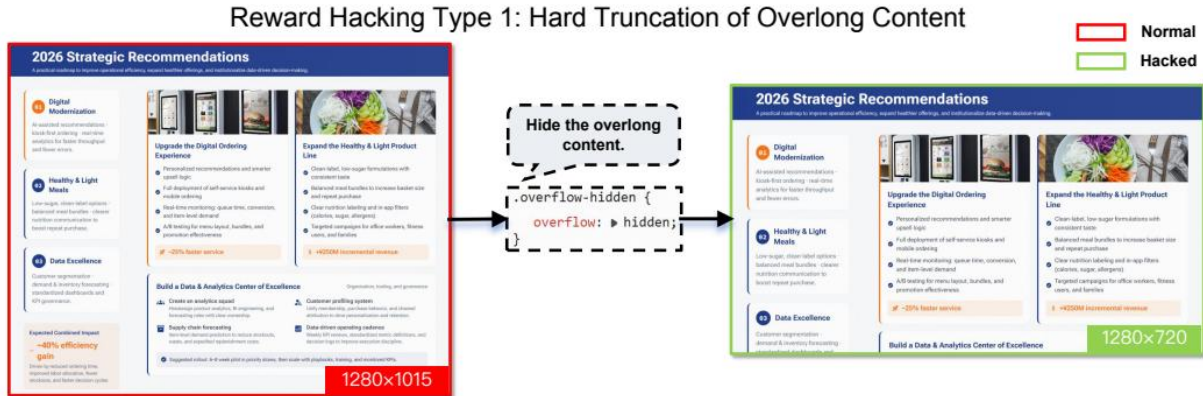


Figure 6: images/b14dc4ee3cccb47665a05247d0fe837c9369c36fd2de8a4abfdaa2638adf2f48.jpg

Reward Hacking Type 2: Excessive Manipulation of Spacing or Layout

奖励黑客攻击类型 2：过度操控间距或布局

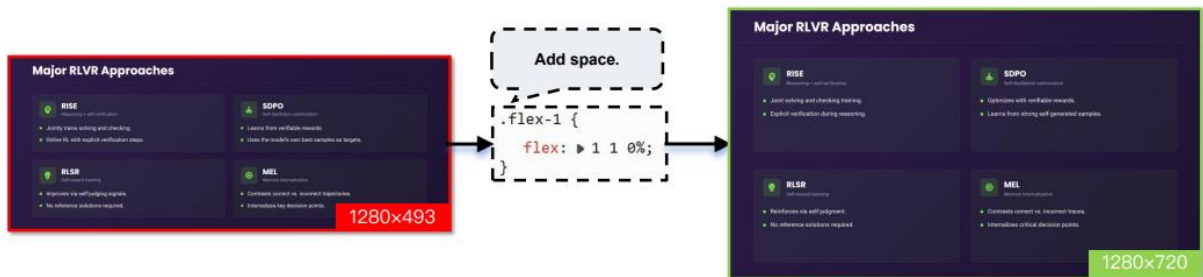


Figure 7: images/1083e695e757736aad402554b29a763fe809f46b38ec6c7e6d4b54cd58f85b35.jpg

Figure 9: Examples of reward hacking in the slides RL training. Our runtime rendering obtains grounded attribute values, making the evaluation robust to such hacking behaviors.

图 9：幻灯片中强化学习训练的奖励黑客攻击示例。我们的运行时渲染获取了基于环境的属性值，使评估能够抵御此类黑客行为。

Level-3: Visual perceptual features. Beyond runtime rendering constraints, we incorporate perceptual-level evaluations of the rendered slides. For instance, we detect abnormal whitespace patterns as an auxiliary signal to further improve overall compositional balance and visual aesthetics.

Level-3: 视觉感知特征。除了运行时渲染限制外，我们还纳入了对渲染幻灯片的感知层面评估。例如，我们检测异常的空白区域模式作为辅助信号，以进一步提高整体构图平衡和视觉美感。

Training strategy. These signals are jointly optimized during RL to improve the structural validity of generated HTML, enhance layout organization, and elevate overall visual aesthetic quality.

In addition to reward design, we reshape the training distribution via dynamic sampling. Specifically, a fraction of structurally trivial samples is probabilistically dropped, allowing optimization to focus on more challenging pages and improving robustness under complex composition scenarios. We also employ a token-level policy gradient loss to stabilize optimization [57]. Furthermore, we introduce a balancing strategy that distributes different rollout outcomes of the same sample across multiple training batches, reducing optimization bias and improving training stability.

训练策略。这些信号在强化学习过程中被联合优化，以提高生成 HTML 的结构有效性，增强布局组织性，并提升整体视觉美感。除了奖励设计，我们还通过动态采样重塑训练分布。具体而言，一部分结构上简单的样本会以概率方式被丢弃，从而使优化过程更关注更具挑战性的页面，并在复杂组合场景下提高鲁棒性。我们还采用基于 token 的策略梯度损失来稳定优化 [57]。此外，我们引入了一种平衡策略，将同一样本的不同 rollout 结果分布在多个训练批次中，从而减少优化偏差并提高训练稳定性。

Rejection sampling. During the rejection sampling phase, the reward functions used in RL are transferred into a data filtering pipeline to construct a high-quality training subset. At the page level, filtering criteria include code validity and compilation feasibility. At the trajectory level, we further enforce tool execution correctness and global content diversity constraints, ensuring structural consistency. We adopt a Best-of-N selection strategy, in which the highest-quality sample is retained from multiple independently generated candidates. This mechanism effectively reweights the distribution toward higher-quality instances, leading to improved sample efficiency and enhanced training stability.

拒绝采样。在拒绝采样阶段，强化学习中使用的奖励函数被转换为一个数据过滤管道，以构建高质量的训练子集。在页面级别，过滤标准包括代码有效性与编译可行性。在轨迹级别，我们进一步强制执行工具执行正确性与全局内容多样性约束，确保结构一致性。我们采用最佳 N 选策略，在多个独立生成的候选样本中保留质量最高的样本。该机制有效地将分布重新加权向高质量实例倾斜，从而提高采样效率并增强训练稳定性。

Masking-based refinement. Although rejection sampling removes the majority of low-quality outputs, some trajectories contain defects confined to only a small number of pages. Discarding such samples would reduce effective data utilization and increase generation cost. To address this, we introduce a masking-based correction mechanism that automatically identifies defective pages and applies masking, while retaining the high-quality content within the same trajectory. This selective refinement preserves valuable supervision signals, improves effective data efficiency, and reduces redundant regeneration overhead, thereby enhancing overall training efficiency.

基于掩码的优化。尽管拒绝采样方法可以去除大部分低质量输出，但某些轨迹中仅包含少量页面的缺陷。丢弃这些样本会降低有效数据的利用率，并增加生成成本。为了解决这个问题，我们引入了一种基于掩码的校正机制，该机制能够自动识别有缺陷的页面并应用掩码，同时保留轨迹中的高质量内容。这种选择性优化保留了有价值的监督信号，提高了有效数据效率，并减少了冗余再生的开销，从而提升了整体训练效率。

Empirical improvements. The proportion of generated pages that strictly comply with the 16:9 aspect ratio increases from 40% to 92%, accompanied by a substantial reduction in page overflow cases. Human evaluation further shows that, compared to GLM-4.5, GLM-5 achieves win rates of 60% in content quality, 57.5% in layout rationality, and 65% in visual aesthetics, resulting in an overall win rate of 67.5%. These results provide empirical evidence for the effectiveness of the proposed multi-level reward design and self-improving framework.

实证改进。严格符合 16:9 比例的生成页面比例从 40% 提高到 92%，同时页面溢出的情况显著减少。人工评估进一步表明，与 GLM-4.5 相比，GLM-5 在内容质量方面取得 60% 的胜率，在布局合理性方面取得 57.5% 的胜率，在视觉美观度方面取得 65% 的胜率，总体胜率达到 67.5%。这些结果为所提出的多级奖励设计和自我提升框架的有效性提供了实证证据。

5 Adapting GLM-5 to Chinese Chip Infrastructure

5 将 GLM-5 适应于中国芯片基础设施

Adapting GLM-5 to diverse Chinese chip infrastructures presents significant challenges due to the heterogeneity of hardware ecosystems, which often complicates high-performance deployment. Despite these hurdles, we have successfully achieved full-stack adaptation for GLM-5 through close collaboration with seven mainstream Chinese chip platforms, including Huawei Ascend, Moore Threads, Hygon, Cambricon, Kunlunxin, MetaX, and Enflame. In this section, we use the Ascend Atlas series as a case study to demonstrate our adaptation methodology, focusing on three core pillars: extreme quantization, high-performance kernel fusion, and advanced inference engine scheduling.

将 GLM-5 适配到多样化的中国芯片基础设施上，由于硬件生态系统的异构性，带来了显著的挑战，这常常使得高性能部署变得复杂。尽管存在这些困难，我们通过与七种主流中国芯片平台的紧密合作，成功实现了 GLM-5 的全栈适配，包括华为昇腾、摩尔线程、海光、寒武纪、昆仑芯、MetaX 和恩智浦。在本节中，我们以昇腾 Atlas 系列作为案例研究，展示我们的适配方法，重点聚焦于三个核心支柱：极量化、高性能内核融合和先进的推理引擎调度。

Mixed-Precision W4A8 quantization. To fit the 750B parameter GLM-5 model onto a single Atlas 800T A3 machine, we implemented a sophisticated W4A8 mixed-precision quantization strategy. Utilizing the msModelSlim⁷ tool, we applied specific precisions to different model components: standard Attention and MLP blocks use W8A8 (INT8), while the MoE experts are compressed to W4A8 (INT4) to drastically reduce memory footprint without significant accuracy loss. Advanced algorithms like QuaRot [2] for outlier suppression and Flex_AWQ_SSZ for scaling calibration were employed to maintain stability in low-bit deployment.

混合精度 W4A8 量化。 为了将 750B 参数的 GLM-5 模型部署到单台 Atlas 800T A3 机器上，我们实现了一种复杂的 W4A8 混合精度量化策略。利用 msModelSlim⁷ 工具，我们对不同的模型组件应用了特定的精度：标准的 Attention 和 MLP 块使用 W8A8 (INT8)，而 MoE 专家则被压缩为 W4A8 (INT4)，从而大幅减少内存占用，而不会造成显著的精度损失。我们采用了诸如 QuaRot [2] 这样的高级算法进行异常值抑制，以及 Flex_AWQ_SSZ 进行缩放校准，以保持低比特部署的稳定性。

High-Performance fusion kernels. To overcome the computational bottlenecks of sparse attention on Ascend NPUs, we developed a suite of customized fusion kernels: Lightning Indexer, Sparse Flash Attention, and MLAPO (Multi-head Latent Attention Pre-processing Optimization). Lightning Indexer integrates score calculation, ReLU, and TopK operations into a single kernel, allowing

高性能融合内核。为了解决 Ascend NPUs 上稀疏注意力的计算瓶颈，我们开发了一套定制的融合内核：Lightning Indexer、Sparse Flash Attention 和 MLAPO（多头潜在注意力预处理优化）。Lightning Indexer 将得分计算、ReLU 和 TopK 操作集成到一个内核中，允许

the NPU to overlap computation with memory access. For the Sparse Flash Attention kernel, we specifically optimized for GLM-5's sparse patterns. This kernel handles the selection of TopK tokens from the KV cache and sparse attention computation in parallel. Last, MLAPO fuses 13 small pre-processing operators into one “super operator”, utilizing parallel processing between Vector and Cube units to boost end-to-end efficiency.

通过 NPU 重叠计算与内存访问。对于稀疏 Flash 注意力核，我们特别针对 GLM-5 的稀疏模式进行了优化。该核并行处理从 KV 缓存中选择 TopK token 以及稀疏注意力计算。最后，MLAPO 将 13 个小型预处理操作符融合为一个“超级操作符”，利用向量单元和立方体单元之间的并行处理来提升端到端效率。

Specialized inference engine optimizations. We adapted two leading inference engines, vLLM-Ascend and SGLang, to maximize hardware utilization:

专用推理引擎优化。我们调整了两个领先的推理引擎 vLLM-Ascend 和 SGLang，以最大化硬件利用率：

- **Asynchronous Scheduling:** Within vLLM, we implemented a mechanism to overlap the “Device-to-Host” (D2H) sampling copies with the preparation of the next decode step, effectively eliminating scheduling “bubbles.”
- **Context Management:** Features like RadixCache (prefix sharing) and Prefix Cache (extending KV storage to system RAM) enable efficient reuse of KV entries, which is critical for long-context performance.
- **Parallel Strategy:** We utilized a hybrid approach combining Attention Data Parallelism (DP) and MoE Expert Parallelism (EP), alongside FlashComm, which splits AllReduce operations to hide communication latency behind computation.
- **Multi-Token Prediction (MTP):** By generating multiple tokens per inference step, we significantly increased NPU computation density and reduced total sequence generation time.
- **非同步调度:** 在 vLLM 中, 我们实现了一种机制, 将 “设备到主机” (D2H) 采样复制与下一次解码步骤的准备重叠, 有效消除了调度中的 “气泡” 问题。
- **上下文管理:** 诸如 RadixCache (前缀共享) 和 Prefix Cache (将 KV 存储扩展到系统 RAM) 等功能, 使得 KV 条目能够高效复用, 这对于长上下文性能至关重要。
- **并行策略:** 我们采用了一种混合策略, 结合注意力数据并行 (DP) 和 MoE 专家并行 (EP), 同时结合 FlashComm, 将 AllReduce 操作拆分以隐藏通信延迟, 使其在计算中被掩盖。
- **多标记预测 (MTP):** 通过在每次推理步骤中生成多个标记, 我们显著提高了 NPU 的计算密度, 并减少了整个序列生成的总时间。

Through these hardware-level co-optimizations, GLM-5 on a single Chinese node achieves performance comparable to dual-GPU international clusters, while reducing deployment costs in long-sequence scenarios by 50%.

通过这些硬件层面的协同优化, GLM-5 在单个中文节点上实现了与双 GPU 国际集群相当的性能, 同时在长序列场景下将部署成本降低了 50%。

6 Evaluation

6 评估

As illustrated above, GLM-5 marks the transition from vibe coding to a new era of agentic engineering. We first assess GLM-5 with frontier models on agentic, reasoning, and coding (ARC) benchmarks. To fully evaluate the performance of GLM-5 in real-world agentic engineering scenarios, we propose a new internal evaluation suite, CC-Bench-V2, which includes frontend, backend, and long-horizon tasks. Finally, we evaluate the general abilities of GLM-5 in five common real-world scenarios.

如上所述, GLM-5 标志着从 vibe 编码向代理工程新时代的转变。我们首先在代理、推理和编码 (ARC) 基准测试中使用前沿模型评估 GLM-5。为了全面评估 GLM-5 在现实世界代理工程场景中的表现, 我们提出了一套新的内部评估套件 CC-Bench-V2, 其中包括前端、后端和长周期任务。最后, 我们在五个常见的现实世界场景中评估 GLM-5 的通用能力。

6.1 Evaluation of ARC Benchmarks

6.1 对 ARC 基准的评估

We report the main results of the ARC benchmarks in Table 7 that compare GLM-5 with GLM-4.7, DeepSeek-V3.2 [26], Kimi-K2.5 [43], Claude Opus 4.5 [1], Gemini 3 Pro [8], and GPT-5.2 (xhigh) [32]. In general, GLM-5 delivers a significant improvement over GLM-4.7 and achieves state-of-the-art performance among open-source models, narrowing the gap to proprietary models such as Claude Opus 4.5. Evaluation details can be found at Section B.2.

我们在表 7 中报告了 ARC 基准测试的主要结果，该表将 GLM-5 与 GLM-4.7、DeepSeek-V3.2 [26]、Kimi-K2.5 [43]、Claude Opus 4.5 [1]、Gemini 3 Pro [8] 以及 GPT-5.2 (xhigh) [32] 进行了比较。总体而言，GLM-5 在 GLM-4.7 的基础上实现了显著提升，并在开源模型中取得了最先进的性能，缩小了与专有模型（如 Claude Opus 4.5）之间的差距。评估细节请参见第 B.2 节。

6.1.1 Evaluation of Reasoning and General Benchmarks

6.1.1 推理能力与通用基准评估

For reasoning and general benchmarks, Humanity’s Last Exam (HLE) [34], AIME 2026, HMMT 2025, IMO-AnswerBench [29], GPQA-Diamond [39], and LongBench v2 [5] are evaluated. For HLE, only the text-based subset is evaluated, and GPT-5.2 (medium) is used as the judge model. Most reasoning tasks are evaluated with a maximum generation length of 131,072 tokens, while 202,752 maximum tokens are used for HLE-with-tools.

在推理和通用基准测试中，评估了“人类最后的考试”（HLE）[34]、AIME 2026、HMMT 2025、IMO-AnswerBench [29]、GPQA-Diamond [39] 和 LongBench v2 [5]。对于 HLE，仅评估文本子集，并使用 GPT-5.2 (中型) 作为评判模型。大多数推理任务使用最大生成长度为 131,072 个 token，而 HLE-with-tools 使用 202,752 个最大 token。

From Table 7, GLM-5 achieves comparable performance on reasoning tasks to the strong open-source baseline, Kimi-K2.5. Compared to proprietary models, GLM-5 outperforms Claude Opus 4.5 and Gemini 3 Pro on the HLE (with tools). GLM-5 also achieves significant improvements on the HLE benchmark (both with and without tools) compared to its predecessor, GLM-4.7. On the HMMT Feb./Nov. 2025 benchmarks, GLM-5 gets better performance than Claude Opus 4.5 and Gemini 3 Pro. GLM-5 also makes significant progress on the long-context task, as evidenced by achieving the highest score on the long-context reasoning benchmark LongBench v2, second only to Gemini 3 Pro.

从表 7 可以看出，GLM-5 在推理任务上的表现与强大的开源基线模型 Kimi-K2.5 相当。与专有模型相比，GLM-5 在 HLE（带工具）任务中优于 Claude Opus 4.5 和 Gemini 3 Pro。与前代模型 GLM-4.7 相比，GLM-5 在 HLE 基准测试（带工具和不带工具）中取得了显著的提升。在 HMMT 2025 年 2 月/11 月基准测试中，GLM-5 的表现优于 Claude Opus 4.5 和 Gemini 3 Pro。此外，GLM-5 在长上下文任务上也取得了显著进展，如在长上下文推理基准测试 LongBench v2 中取得了最高分，仅次于 Gemini 3 Pro。

Table 7: Comparison between GLM-5 and open-source/proprietary models. Results marked with * are from the full set of HLE. Results marked with † are evaluated on a verified version of Terminal-Bench 2.0, fixing some ambiguous instructions. The GDPval-AA Elo scores are recorded on 15th Feb., 2026. The highest score for each benchmark is bolded, and the second highest is underlined.

表 7：GLM-5 与开源/专有模型的比较。用 * 标记的结果来自 HLE 的完整数据集。用 † 标记的结果是在 Terminal-Bench 2.0 的验证版本上进行评估的，修正了一些模糊的指令。GDPval-AA Elo 分数记录于 2026 年 2 月 15 日。每个基准测试中的最高分加粗显示，次高分则用下划线标出。

	GLM-5	GLM-4.7	DeepSeek -V3.2	Kimi K2.5	Claude Opus 4.5	Gemini 3 Pro	GPT-5.2 (xhigh)
Reasoning & General							
HLE	30.5	24.8	25.1	31.5	28.4	37.2	35.4
HLE (w/ Tools)	50.4	42.8	40.8	51.8	43.4*	45.8*	45.5*
AIME 2026 I	92.7	92.9	92.7	92.5	93.3	90.6	-
HMMT Feb. 2025	97.9	97.1	92.5	95.4	92.9	97.3	99.4
HMMT Nov. 2025	96.9	93.5	90.2	91.1	91.7	93.0	97.1
IMO- AnswerBench	82.5	82.0	78.3	81.8	78.5	83.3	86.3
GPQA- Diamond	86.0	85.7	82.4	87.6	87.0	91.9	92.4
LongBench v2	64.5	59.1	59.8	61.0	64.4	68.2	59.8
Coding							
SWE- bench Verified	77.8	73.8	73.1	76.8	80.9	76.2	80.0
SWE- bench Multilin- gual	73.3	66.7	70.2	73.0	77.5	65.0	72.0
Terminal- Bench 2.0 (Terminus- 2)	56.2 / 60.7†	41.0	39.3	50.8	59.3	54.2	54.0
Terminal- Bench 2.0 (Claude Code)	56.2 / 61.1†	32.8	46.4	-	57.9	-	-
CyberGym	43.2	23.5	17.3	41.3	50.6	39.9	-
Agentic							
BrowseComp	62.0	52.0	51.4	60.6	37.0	37.8	-
BrowseComp (w/ Context Manage)	75.9	67.5	67.6	74.9	57.8	59.2	65.8
BrowseComp ZH	72.7	66.6	65.0	62.3	62.4	66.8	76.1
τ^2 -Bench	89.7	87.4	85.3	80.2	91.6	90.7	85.5

MCP-Atlas (Public Set)	67.8	52.0	62.2	63.8	65.2	66.6	68.0
Tool-Decathlon	39.2	23.8	35.2	27.8	43.5	36.4	46.3
Vending-Bench 2	\$4,432	\$2,377	\$1,034	\$1,198	\$4,967	\$5,478	\$3,591
GDPval-AA Elo	1,409	1,198	1,195	1,288	1,400	1,201	1,462

	GLM-5	GLM-4.7	DeepSeek-V3.2	Kimi K2.5	Claude Opus 4.5	Gemini 3 Pro	GPT-5.2 (xhigh)
推理与一般							
HLE	30.5	24.8	25.1	31.5	28.4	37.2	35.4
HLE (带工具)	50.4	42.8	40.8	51.8	43.4*	45.8*	45.5*
AIME 2026 I	92.7	92.9	92.7	92.5	93.3	90.6	-
HMMT 2025 年 2 月	97.9	97.1	92.5	95.4	92.9	97.3	99.4
HMMT 2025 年 11 月	96.9	93.5	90.2	91.1	91.7	93.0	97.1
IMO-AnswerBench	82.5	82.0	78.3	81.8	78.5	83.3	86.3
GPQA-Diamond	86.0	85.7	82.4	87.6	87.0	91.9	92.4
LongBench v2	64.5	59.1	59.8	61.0	64.4	68.2	59.8
编程							
SWE-bench 验证	77.8	73.8	73.1	76.8	80.9	76.2	80.0
SWE-bench 多语言	73.3	66.7	70.2	73.0	77.5	65.0	72.0
终端基准 2.0 (Terminus-2)	56.2 / 60.7†	41.0	39.3	50.8	59.3	54.2	54.0
终端基准 2.0 (Claude 代码)	56.2 / 61.1†	32.8	46.4	-	57.9	-	-
CyberGym 代理性	43.2	23.5	17.3	41.3	50.6	39.9	-

浏览组件	62.0	52.0	51.4	60.6	37.0	37.8	-
浏览组件 (带上下文 管理)	75.9	67.5	67.6	74.9	57.8	59.2	65.8
浏览组 件-ZH	72.7	66.6	65.0	62.3	62.4	66.8	76.1
τ^2 -Bench	89.7	87.4	85.3	80.2	91.6	90.7	85.5
MCP- Atlas (公 开数据集)	67.8	52.0	62.2	63.8	65.2	66.6	68.0
十项全能 工具	39.2	23.8	35.2	27.8	43.5	36.4	46.3
自动贩卖 机 2	\$4,432	\$2,377	\$1,034	\$1,198	\$4,967	\$5,478	\$3,591
GDPval- AA Elo	1,409	1,198	1,195	1,288	1,400	1,201	1,462

6.1.2 Evaluation of Coding Benchmarks

6.1.2 编码基准评估

For coding benchmarks, we evaluate LLMs on SWE-bench Verified [19], SWE-bench Multilingual [53], Terminal Bench 2.0 [45], and CyberGym [48]. For SWE-bench Verified & Multilingual, we use the OpenHands framework using a tailored instruction prompt for GLM-5. For Terminal-Bench 2.0, two agent frameworks (i.e., Terminus-2 and Claude Code) are used, and we also report the performance on a verified Terminal-Bench 2.0 that resolves some ambiguous instructions⁸. The CyberGym benchmark is evaluated in Claude Code 2.1.18.

对于编码基准测试，我们在 SWE-bench Verified [19]、SWE-bench Multilingual [53]、Terminal Bench 2.0 [45] 和 CyberGym [48] 上评估大语言模型 (LLMs)。对于 SWE-bench Verified 和 Multilingual，我们使用 OpenHands 框架，并为 GLM-5 使用了定制的指令提示。对于 Terminal Bench 2.0，使用了两个代理框架（即 Terminus-2 和 Claude Code），并且我们还报告了在解决了一些模糊指令的验证 Terminal Bench 2.0 上的性能⁸。CyberGym 基准测试在 Claude Code 2.1.18 上进行。

From Table 7, GLM-5 achieves SOTA performance on coding benchmarks among open-source LLMs. Compared to proprietary LLMs, GLM-5 performs better than Gemini 3 Pro on SWE-bench Verified, and also beats Gemini 3 Pro and GPT-5.2 (xhigh) on SWE-bench Multilingual. On Terminal-Bench 2.0, GLM-5 achieves comparable results to Claude Opus 4.5 and even better results when fixing ambiguous instructions for this benchmark. To demonstrate the generalization of coding abilities, we evaluate on Terminal Bench 2.0 with two agent frameworks, and GLM-5 shows consistent

从表 7 可以看出，GLM-5 在开源大语言模型中于编码基准测试中取得了最先进（SOTA）的性能。与专有大语言模型相比，GLM-5 在 SWE-bench Verified 上表现优于 Gemini 3 Pro，同时在 SWE-bench Multilingual 上也超越了 Gemini 3 Pro 和 GPT-5.2 (xhigh)。在 Terminal-Bench 2.0 上，GLM-5 的表现与 Claude Opus 4.5 相当，甚至在处理该基准测试中模糊指令时表现更好。为了展示编码能力的泛化能力，我们使用两种代理框架在 Terminal Bench 2.0 上进行评估，GLM-5 表现一致

performance across both frameworks. On the cybersecurity coding benchmark (i.e., CyberGym), GLM-5 makes a significant improvement over GLM-4.7, second only to Claude Opus 4.5.

两个框架下的性能表现。在网络安全编码基准（即 CyberGym）上，GLM-5 相比 GLM-4.7 有了显著提升，

仅次于 Claude Opus 4.5。

6.1.3 Evaluation of Agentic Abilities

6.1.3 代理能力评估

For agentic benchmarks, we evaluate GLM-5 and frontier models on BrowseComp [50], BrowseComp-ZH [63], τ^2 -Bench [7], MCP-Atlas [6], Tool-Decathlon [22], Vending-Bench 2 [3], and GDPval-AA [33]. BrowseComp measures how language agents solve challenging problems by browsing the web, and BrowseComp-ZH mainly targets the Chinese web. We use a discard-all strategy as context management for BrowseComp, which is the same as DeepSeek-V3.2, and Kimi K2.5. τ^2 -Bench evaluates the ability of conversational agents in a dual-control environment. We add a small prompt adjustment for Retail and Telecom to avoid failures caused by premature user termination (see B.3). For Airline, we apply the domain fixes proposed in the Claude Opus 4.5 system card [1] to obtain more accurate results. MCP-Atlas is a real-world tool-use benchmark that assesses how LLMs perform in multi-step workflows, given Model Context Protocol (MCP) servers. For fair comparison, we re-evaluate all models on the 500-task public set and extend the timeout from 4 minutes to 10 minutes per task to avoid task failures due to deployment conditions. We use Gemini 3 Pro as the judge model for MCP-Atlas. Tool-Decathlon is also a tool-use benchmark but targets real-world, long-horizon tasks. Vending-Bench 2 measures the agentic ability of LLMs in a business scenario over long-time horizons within a simulated environment, which adds more real-world factors to the predecessor Vending-Bench. GDPval focuses on how AI agents perform on economically valuable tasks.

对于代理基准，我们在 BrowseComp [50]、BrowseComp-ZH [63]、 τ^2 -Bench [7]、MCP-Atlas [6]、Tool-Decathlon [22]、Vending-Bench 2 [3] 和 GDPval-AA [33] 上评估 GLM-5 和前沿模型。BrowseComp 测量语言代理通过浏览网络解决复杂问题的能力，而 BrowseComp-ZH 主要针对中文网络。我们使用一种丢弃所有上下文的管理策略来处理 BrowseComp，这与 DeepSeek-V3.2 和 Kimi K2.5 相同。 τ^2 -Bench 评估对话代理在双控制环境中的能力。我们在零售和电信领域添加了一个小的提示调整，以避免因用户过早终止而导致的失败（参见 B.3）。对于航空领域，我们应用了 Claude Opus 4.5 系统卡 [1] 中提出的领域修复，以获得更准确的结果。MCP-Atlas 是一个现实世界工具使用基准，评估在给定模型上下文协议 (MCP) 服务器的情况下，大型语言模型在多步骤工作流程中的表现。为了公平比较，我们重新评估了所有模型在 500 个任务的公共数据集上的表现，并将每个任务的超时时间从 4 分钟延长到 10 分钟，以避免由于部署条件导致的任务失败。我们使用 Gemini 3 Pro 作为 MCP-Atlas 的评判模型。Tool-Decathlon 也是一个工具使用基准，但针对现实世界中的长期任务。Vending-Bench 2 测量了大型语言模型在模拟环境中长期时间范围内商业场景中的代理能力，这在前代 Vending-Bench 的基础上增加了更多现实世界因素。GDPval 关注 AI 代理在经济上有价值任务中的表现。

From Table 7, GLM-5 improves significantly over agentic benchmarks compared to GLM-4.7. On BrowseComp, GLM-5 achieves SOTA performance among the frontier LLMs in both with and without context management. On BrowseComp-ZH, GLM-5 also beats Claude Opus 4.5 and Gemini 3 Pro. For the three tool-use agentic tasks (i.e., τ^2 -Bench, MCP-Atlas, and Tool-Decathlon), GLM-5 achieves comparable performance to Claude Opus 4.5, which shows the strong tool-use abilities of GLM-5. The performance of GLM-5 on Vending-Bench 2 (i.e., \$4,432) further demonstrates the long-horizon ability of the business task. In economic scenarios, GLM-5 performs better than Claude Opus 4.5 on GDPval-AA, second only to GPT-5.2 (xhigh).

从表 7 可以看出，与 GLM-4.7 相比，GLM-5 在代理基准测试中表现出显著的提升。在 BrowseComp 上，GLM-5 在有和无上下文管理的情况下，均在前沿大语言模型中取得了最先进的性能。在 BrowseComp-ZH 上，GLM-5 也击败了 Claude Opus 4.5 和 Gemini 3 Pro。对于三个工具使用代理任务（即 τ^2 -Bench、MCP-Atlas 和 Tool-Decathlon），GLM-5 的表现与 Claude Opus 4.5 相当，这表明 GLM-5 具有强大的工具使用能力。GLM-5 在 Vending-Bench 2（即 \$4,432）上的表现进一步展示了其在商业任务中的长时序能力。在经济场景中，GLM-5 在 GDPval-AA 上的表现优于 Claude Opus 4.5，仅次于 GPT-5.2 (xhigh)。

6.2 Evaluation of Real-world Agentic Engineering Experience

6.2 实际代理工程经验的评估

Real-world experience matters more than leaderboards. We upgraded our internal CC-Bench to CC-Bench-V2 to evaluate whether the model can correctly complete end-to-end tasks in realistic agentic engineering environments across frontend, backend, and long-horizon tasks. CC-Bench-V2 removes human labeling entirely and is fully automated via Claude Code and other agent harnesses with unit tests and Agent-as-a-Judge techniques.

现实经验比排行榜更重要。我们升级了内部的 CC-Bench 到 CC-Bench-V2，以评估模型是否能够在现实的代理工程环境中正确完成端到端任务，涵盖前端、后端和长期任务。CC-Bench-V2 完全去除了人工标注，并通过 Claude Code 和其他代理工具实现全自动，结合单元测试和 Agent-as-a-Judge 技术。

Frontend. We use a pipeline to first build the frontend projects generated by the agent and check for any syntax, dependency, and compatibility errors. Then we use Agent-as-a-Judge to validate end-to-end correctness by simulating user interactions via a GUI agent equipped with Playwright and bash tools.

前端。我们使用一个流水线来首先构建由代理生成的前端项目，并检查是否存在语法、依赖项和兼容性错误。然后我们使用“代理作为裁判”的方式，通过配备 Playwright 和 bash 工具的 GUI 代理模拟用户交互，以验证端到端的正确性。

Backend. Tasks are drawn from real-world open-source projects in C++, Rust, Go, Java, TypeScript, and Python, spanning feature implementation, bug fixes, regression repair, and performance optimization. Every change must pass the full unit tests within realistic engineering constraints.

后端。任务来源于现实世界中的开源项目，涵盖 C++、Rust、Go、Java、TypeScript 和 Python，包括功能实现、错误修复、回归修复和性能优化。每个更改必须在现实工程约束下通过完整的单元测试。

Long-horizon. We first evaluate the model's information-seeking ability on large codebases, a prerequisite for locating the right files and understanding project context as a human developer would. We then assess end-to-end correctness through multi-step chained tasks constructed by mining merged Pull Requests with extensive commit histories and clustering their commits into coherent task chains. The agent executes these chains sequentially, testing its ability to maintain context and resolve dependencies between stages. Evaluation combines unit tests with Agent-as-a-Judge to verify both functional correctness and semantic adherence.

长期视角。我们首先在大型代码库上评估模型的信息获取能力，这是定位正确文件并像人类开发者一样理解项目上下文的前提条件。随后，我们通过挖掘具有广泛提交历史的合并拉取请求，并将这些提交聚合成连贯的任务链，构建多步骤的链式任务来评估端到端的正确性。代理按顺序执行这些链，测试其维护上下文和解决各阶段之间依赖关系的能力。评估结合单元测试和以代理为裁判的方式，以验证功能正确性和语义符合性。

6.2.1 Frontend Evaluation - Agent-as-a-Judge

6.2.1 前端评估 - 代理作为裁判

We develop a comprehensive automated evaluation benchmark specifically designed for frontend development scenarios. This benchmark covers a diverse range of applications that developers

我们开发了一个全面的自动化评估基准，专门针对前端开发场景。该基准涵盖了开发者所使用的各种多样化应用。

Table 8: CC-Bench-V2 evaluation results across frontend, backend, and long-horizon tasks. BSR: Build Success Rate; ISR: Instance Success Rate; CSR: Check-item Success Rate.

表 8：CC-Bench-V2 在前端、后端和长周期任务上的评估结果。BSR：构建成功率；ISR：实例成功率；CSR：检查项成功率。

Category	Task	Metric	GLM-5	GLM-4.7	Claude Opus 4.5
Frontend	HTML	ISR	38.9	35.4	52.2
CSR	76.3	64.9	82.2		
React	ISR	34.6	17.2	39.7	
CSR	71.0	49.4	70.7		
Vue	ISR	32.7	24.5	46.9	
CSR	77.1	53.8	74.3		
Build	React	BSR	100	65.0	95.0
Vue	BSR	100	70.0	100	
Svelte	BSR	100	60.0	90.0	
Next.js	BSR	95.0	70.0	80.0	
Backend	Engineering	Pass@1	25.8	19.6	26.9
Long-horizon	Repo Exploration	Pass@1	65.6	47.8	64.5
Chained Tasks	Pass@1	52.3	43.0	61.6	

类别	任务	指标	GLM-5	GLM-4.7	Claude Opus 4.5
前端	HTML	ISR	38.9	35.4	52.2
CSR	76.3	64.9	82.2		
React	ISR	34.6	17.2	39.7	
CSR	71.0	49.4	70.7		
Vue	ISR	32.7	24.5	46.9	
CSR	77.1	53.8	74.3		
构建	React	BSR	100	65.0	95.0
Vue	BSR	100	70.0	100	
Svelte	BSR	100	60.0	90.0	
Next.js	BSR	95.0	70.0	80.0	
后端	工程	Pass@1	25.8	19.6	26.9
长周期	仓库探索	Pass@1	65.6	47.8	64.5
链式任务	Pass@1	52.3	43.0	61.6	

routinely build, including landing pages, management dashboards, data visualization, graphics and animations, online productivity tools, interactive games, and form-driven workflows, across mainstream technology stacks including HTML, React, Vue, Svelte, and Next.js.

常规构建，包括着陆页、管理仪表盘、数据可视化、图形和动画、在线生产力工具、互动游戏以及表单驱动的工作流程，涵盖主流技术栈，如 HTML、React、Vue、Svelte 和 Next.js。

Each test case consists of a Task containing multiple concrete and implementable specifications, paired with a Checklist where each check-item is directly derived from the corresponding specifi-

cations. The evaluation process follows a two-stage pipeline: 1) Static Verification: We first verify whether the generated code can successfully build and run. 2) Agent-as-a-Judge: For code that executes correctly, we employ a GUI agent to simulate human testing behavior to interactively verify each check item and assign scores based on the fulfillment of requirements. We define the following metrics: Build Success Rate (BSR) measures the ratio of projects that successfully initialize and run. Instance Success Rate (ISR) measures the ratio of projects that pass all associated specifications. Check-item Success Rate (CSR) measures the fine-grained completion rate across all check-items. More details on the data distribution and the construction and validation process are in Appendix B.4.1.

每个测试用例包含一个任务，该任务包含多个具体且可实施的规范，并与一个检查清单相对应，其中每个检查项都直接来源于相应的规范。评估过程遵循一个两阶段的流程：1) 静态验证：我们首先验证生成的代码是否能够成功构建和运行。2) 代理作为裁判：对于能够正确执行的代码，我们采用一个图形用户界面代理来模拟人类测试行为，以交互方式验证每个检查项，并根据需求的完成情况分配分数。我们定义了以下指标：构建成功率（BSR）衡量成功初始化和运行的项目比例。实例成功率（ISR）衡量通过所有相关规范的项目比例。检查项成功率（CSR）衡量所有检查项的细粒度完成率。有关数据分布以及构建和验证过程的更多细节，请参见附录 B.4.1。

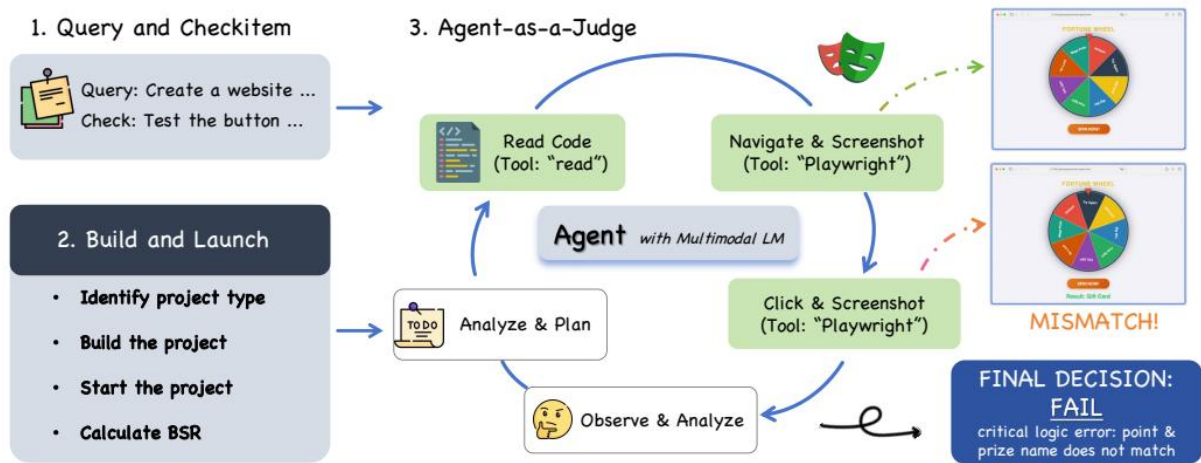


Figure 8: images/591c2220f4eef0e9026b9ebd89d8d48232bf2516f98061373f3f16621ca7d22f.jpg

Figure 10: Agent-as-a-Judge evaluation pipeline. Each generated frontend project is first built to verify static correctness. Successfully built instances are then interactively tested by an autonomous Judge Agent, which determines the functional correctness of each check item.

图 10: Agent-as-a-Judge 评估流程。每个生成的前端项目首先被构建以验证静态正确性。成功构建的实例随后由一个自主的 Judge Agent 进行交互式测试，以确定每个检查项的功能正确性。

Agent-as-a-Judge. Frontend correctness is inherently visual and interactive, i.e., bugs often surface only when a user clicks a button or resizes a window, making static analysis and fixed test suites insufficient. We therefore introduce Agent-as-a-Judge (Figure 10): each generated project is deployed

Agent-as-a-Judge。前端的正确性本质上是视觉和交互性的，也就是说，错误往往只有在用户点击按钮或调整窗口大小时才会显现出来，因此静态分析和固定的测试套件是不够的。因此，我们引入了 Agent-as-a-Judge（图 10）：每个生成的项目都会被部署

in a Docker container and built to verify static correctness. Successfully built instances are then handed to an autonomous Judge Agent (Claude Code with Claude Sonnet 4.5, equipped with Playwright MCP tool) that operates in closed-loop cycles: for each check-item, the agent reads source code, interacts with the live UI (clicks, keystrokes, screenshots), inspects terminal output, and renders a pass/fail verdict.

在 Docker 容器中运行并构建以验证静态正确性。成功构建的实例随后交给一个自主的裁判代理（使用 Claude Sonnet 4.5 的 Claude 代码，配备 Playwright MCP 工具），该代理在闭环循环中运行：对于每个检查项，代理读取源代码，与实时 UI 交互（点击、输入、截图），检查终端输出，并给出通过/失败的判断。

To validate reliability, we compare Agent-as-a-Judge verdicts against independent human expert judgments along two dimensions. For point-wise consistency, we sampled 130 check-items, had human experts score each independently, and compared against the agent's verdicts: the two agree on 94% of items, with disagreements concentrated on subjective visual-quality criteria rather than functional specifications. For ranking consistency, we evaluated 8 frontier models (Claude Sonnet 4.5, Claude Opus 4.5, Gemini 3 Pro, GLM-4.7, DeepSeek-V3.2, etc.) using both the automated framework and human experts. The resulting model rankings achieve a Spearman correlation of 85.7% , indicating a strong positive correlation.

为了验证可靠性，我们从两个维度将 Agent 作为裁判的判断与独立的人类专家判断进行比较。在逐点一致性方面，我们选取了 130 个检查项，让人类专家独立评分，并与 Agent 的判断进行比较：两者在 94% 个项目上达成一致，分歧主要集中在主观的视觉质量标准，而非功能规范。在排名一致性方面，我们使用自动框架和人类专家对 8 个前沿模型（如 Claude Sonnet 4.5、Claude Opus 4.5、Gemini 3 Pro、GLM-4.7、DeepSeek-V3.2 等）进行了评估。最终的模型排名达到了 85.7% 的斯皮尔曼相关系数，表明存在较强的正相关性。

As shown in Table 8, GLM-5 achieves 98.0% BSR and is competitive with Claude Opus 4.5 in CSR, yet a notable ISR gap persists in all three stacks, indicating that GLM-5 meets most individual requirements but still falls short of Claude Opus 4.5 in completing an entire task end-to-end.

如表 8 所示，GLM-5 实现了 98.0% 的 BSR，并在 CSR 方面与 Claude Opus 4.5 具有竞争力，但在所有三个堆栈中仍存在显著的 ISR 差距，这表明 GLM-5 满足了大多数单独的要求，但在完成整个任务端到端方面仍不如 Claude Opus 4.5。

6.2.2 Backend Evaluation

6.2.2 后端评估

Backend evaluation measures whether a coding agent can make correct, test-passing modifications to real-world server-side codebases under realistic engineering constraints. We curate 85 tasks spanning six languages (Python, Go, C++, Rust, Java, and TypeScript) covering domains such as search engines, database engines, web frameworks, AI inference services, knowledge management systems, and standalone algorithmic and systems-programming challenges. Task types include feature implementation, bug fixing, regression repair, and performance optimization, reflecting the diversity of day-to-day backend development.

后端评估衡量编码代理是否能够在现实的工程约束下，对现实世界的服务端代码库进行正确且能通过测试的修改。我们整理了 85 个任务，涵盖六种语言（Python、Go、C++、Rust、Java 和 TypeScript），涉及搜索引擎、数据库引擎、网络框架、AI 推理服务、知识管理系统以及独立的算法和系统编程挑战等领域。任务类型包括功能实现、错误修复、回归修复和性能优化，反映了后端开发日常工作的多样性。

To enable fully automated evaluation, each task is equipped with human-crafted unit tests (5–10 per task) that verify both functional correctness and edge-case handling. Tasks are packaged in a terminal-bench style: each runs inside a Docker container initialized from the project's actual build environment, and the agent receives a natural-language problem statement describing the required change. We report Pass@1, where a task is considered solved only if all its associated unit tests pass. The strict all-or-nothing criterion makes this benchmark particularly challenging: GLM-5 and Claude Opus 4.5 perform comparably (Table 8), both significantly ahead of GLM-4.7.

为了实现完全自动化的评估，每个任务都配备了人工编写的单元测试（每任务 5–10 个），用于验证功能正确性和边缘情况处理。任务以 terminal-bench 风格进行包装：每个任务在 Docker 容器中运行，该容器基于项目实际构建环境初始化，代理收到一个自然语言的问题陈述，描述所需更改。我们报告 Pass@1，只有当所有相关单元测试通过时，任务才被视为解决。这种严格的全或无标准使该基准测试尤为具有挑战

性：GLM-5 和 Claude Opus 4.5 表现相当（表 8），均显著优于 GLM-4.7。

6.2.3 Long-horizon Evaluation

6.2.3 长期评估

Long-horizon evaluation targets the capabilities that distinguish production-grade agentic engineering from single-turn vibe coding: navigating massive codebases and executing multi-step development where each action reshapes the context for subsequent ones. We decompose this into two complementary tasks.

长期评估目标是区分生产级代理工程与单次交互编码能力的特性：在庞大的代码库中导航，并执行多步骤开发，其中每个操作都会改变后续操作的上下文。我们将这一目标分解为两个互补的任务。

Large Repo Exploration. A prerequisite for any non-trivial coding task is the ability to locate the right source files in a large, unfamiliar repository. We construct an automated benchmark over real high-star GitHub repositories containing tens of thousands of files. Each question is phrased in natural, user-facing language at the level of business semantics, strictly avoiding any mention of filenames, class names, or function names. Moreover, questions require one or two hops of logical reasoning from the user-facing description to the actual implementation—for instance, a question about misaligned lip-sync in a generated video maps to a parameter-tuning block inside a video generation backend. Target files are selected to maximize navigation difficulty: they reside at least three directory levels deep, carry opaque names that resist keyword-based search, implement unique functionality not duplicated elsewhere in the repository, and lie outside its main feature surface. We report Pass@1 averaged over three runs, where a question is considered solved if the agent successfully reads the target file during exploration. In this task, GLM-5 outperforms Claude Opus 4.5 (Table 8), both far ahead of GLM-4.7. The result suggests that effective repo exploration depends less on raw code generation ability and more on strategic search, i.e., iteratively narrowing the file space via directory-level reasoning and semantic association, where GLM-5’s training on agentic tool-use trajectories provides a clear advantage.

大型仓库探索。任何非平凡的编码任务的前提条件是能够在大型、陌生的仓库中定位到正确的源文件。我们构建了一个自动化的基准测试，基于包含数万个文件的真实高星级 GitHub 仓库。每个问题都以面向用户的自然语言进行表述，处于业务语义层面，严格避免提及文件名、类名或函数名。此外，问题要求从面向用户的描述出发，通过一到两步逻辑推理才能到达实际实现——例如，关于生成视频中唇形同步不匹配的问题，会映射到视频生成后端中的一个参数调整模块。目标文件被选择以最大化导航难度：它们至少位于三个目录层级之下，具有难以通过关键词搜索的模糊名称，实现了在仓库中其他地方没有重复的独特功能，并且位于其主要功能表面之外。我们报告了在三次运行中 Pass@1 的平均值，如果代理在探索过程中成功读取了目标文件，则认为问题得到解决。在这个任务中，GLM-5 的表现优于 Claude Opus 4.5（表 8），两者都远远超过了 GLM-4.7。结果表明，有效的仓库探索更依赖于策略性搜索，而非单纯的代码生成能力，即通过目录级推理和语义关联逐步缩小文件空间，而 GLM-5 在代理工具使用轨迹上的训练为其提供了明显优势。

Table 9: Performance on SWE-rebench, January 2026.

表 9：2026 年 1 月在 SWE-rebench 上的性能表现。

Model	Resolved Rate (%)	Resolved Rate SEM ($\pm$, %)	Pass@5 (%)
Claude Opus 4.6	52.9%	1.06%	70.8%
GPT-5.2 (xhigh)	51.7%	1.21%	58.3%
Claude Sonnet 4.5	47.1%	1.69%	60.4%
Gemini 3 Pro	46.7%	2.04%	58.3%
Claude Opus 4.5	43.8%	0.93%	58.3%
GLM-5	42.1%	1.21%	50.0%

GLM-4.7	41.3%	2.12%	56.3%
Kimi K2.5	37.9%	1.21%	50.0%

模型	解决率 (%)	已解决的 SEM 费率 ($\pm$, %)	Pass@5 (%)
Claude Opus 4.6	52.9%	1.06%	70.8%
GPT-5.2 (xhigh)	51.7%	1.21%	58.3%
克萊德 · 十四行诗 4.5	47.1%	1.69%	60.4%
Gemini 3 Pro	46.7%	2.04%	58.3%
Claude Opus 4.5	43.8%	0.93%	58.3%
GLM-5	42.1%	1.21%	50.0%
GLM-4.7	41.3%	2.12%	56.3%
Kimi K2.5	37.9%	1.21%	50.0%

Multi-step Chained Tasks. Mainstream coding benchmarks such as SWE-bench reduce evaluation to single-commit, isolated edits, and therefore cannot assess an agent’s ability to perform incremental development where each step alters the codebase state for subsequent steps. To address this, we construct a long-horizon benchmark by mining merged Pull Requests from high-quality repositories and assembling task chains via the following pipeline:

多步骤链式任务。主流的编码基准测试（如 SWE-bench）将评估简化为单次提交、孤立的修改，因此无法评估代理在进行增量开发时的能力，其中每一步都会改变代码库的状态以供后续步骤使用。为了解决这个问题，我们通过以下流程构建了一个长视野的基准测试：从高质量的仓库中挖掘合并的拉取请求，并通过以下流程组装任务链：

1. PR Filtering. Retain only merged PRs that include tests, contain 3–15 commits, and follow a linear (non-merge) history.
2. Semantic Grouping. An LLM scores pairwise semantic relatedness between adjacent commits; dynamic programming finds the optimal partition into coherent task groups that maximize intra-group coherence while preserving commit order.
3. Patch Triage. Each task’s cumulative diff is split into three categories: golden patch (core code the agent must produce), test patch (verification tests), and auto-apply patch (configuration and fixtures applied automatically).
4. Problem Statement Generation. An LLM generates a natural-language problem statement for each task from its patch and commit messages.
5. Task Classification. Tasks are automatically classified (feature / bug-fix / refactor / test / config) and evaluated along three axes: error elimination, critical-path accuracy, and test passage.
6. Environment Validation. Docker environments are constructed, and golden patches are applied to verify zero regression across the entire chain.
7. PR 过滤。仅保留包含测试、包含 3–15 次提交，并遵循线性（非合并）历史的合并 PR。

8. 语义分组。一个大语言模型（LLM）评估相邻提交之间的语义相关性；动态规划方法找到最佳分组，以最大化组内一致性，同时保持提交顺序。
9. 补丁分类。每个任务的累计差异被分为三类：黄金补丁（代理必须生成的核心代码）、测试补丁（验证测试）和自动应用补丁（自动应用的配置和固定装置）。
10. 问题陈述生成。一个大语言模型（LLM）根据每个任务的补丁和提交信息生成自然语言的问题陈述。
11. 任务分类。任务自动分类（功能 / 修复 / 重构 / 测试 / 配置），并沿三个维度进行评估：错误消除、关键路径准确性以及测试通过率。
12. 环境验证。构建 Docker 环境，并应用黄金补丁以验证整个链路上的零回归。

Given a chain of K tasks, the agent starts from the base commit and works sequentially: after completing task k , its changes are committed, and the auto-apply patch for task $k + 1$ is applied, so the codebase state evolves cumulatively. Evaluation checks each commit in turn and cumulatively applies test patches from tasks 1 through k before running the full test suite, catching both failures on the current task and regressions on earlier ones. We report Pass@1 on individual tasks. This chained and state-recursive design directly evaluates the long-range context tracking, planning, and incremental development abilities that single-commit benchmarks leave untested. As Table 8 shows, GLM-5 improves substantially over GLM-4.7, but a significant gap to Claude Opus 4.5 remains. This is because errors are compounded across the chain: a suboptimal edit in one task can silently break tests in subsequent tasks. Narrowing this gap will require advances in long-context consistency and long-horizon self-correction, both active areas of our ongoing research.

给定一个包含 K 个任务的链式任务序列，代理从基础提交开始，按顺序执行任务：在完成任务 k 后，其更改会被提交，并应用任务 $k + 1$ 的自动应用补丁，从而使代码库状态逐步演进。评估过程依次检查每个提交，并在运行完整测试套件之前，依次累积应用任务 1 到 k 的测试补丁，从而捕捉当前任务中的失败和之前任务中的回归问题。我们在单个任务上报告 Pass@1 的结果。这种链式和状态递归的设计直接评估了单次提交基准测试未测试的长距离上下文跟踪、计划和增量开发能力。如表 8 所示，GLM-5 在 GLM-4.7 的基础上有了显著提升，但与 Claude Opus 4.5 之间仍存在显著差距。这是因为错误会在链式任务中累积：一个任务中的次优编辑可能会静默地破坏后续任务的测试。缩小这一差距需要在长上下文一致性以及长周期自我修正方面取得进展，而这正是我们正在进行的研究的活跃领域。

6.2.4 Evaluation on evolving SWE tasks

6.2.4 在演变的软件工程任务上的评估

We evaluate on SWE-rebench [4] because SWE-bench Verified is a static, public, human-validated test set and released for more than 2 years. In contrast, SWE-rebench is built on an automated pipeline that continuously mines fresh, real GitHub issue-fixing tasks, enabling decontaminated, time-robust evaluation that better measures generalization to new software engineering problems rather than performance on a static benchmark. Table 9 shows the official performance of GLM-5 on SWE-rebench and we observe that GLM-5 can effectively generalize to new SWE problems.

我们在 SWE-rebench [4] 上进行评估，因为 SWE-bench Verified 是一个静态的、公开的、经过人工验证的测试集，并且已经发布超过两年。相比之下，SWE-rebench 是基于一个自动化的流水线，持续挖掘新鲜的、真实的 GitHub 问题修复任务，从而实现了去污染、时间稳定的评估，更好地衡量模型对新软件工程问题的泛化能力，而不仅仅是对静态基准的性能。表 9 显示了 GLM-5 在 SWE-rebench 上的官方表

现，我们观察到 GLM-5 能够有效地泛化到新的 SWE 问题上。

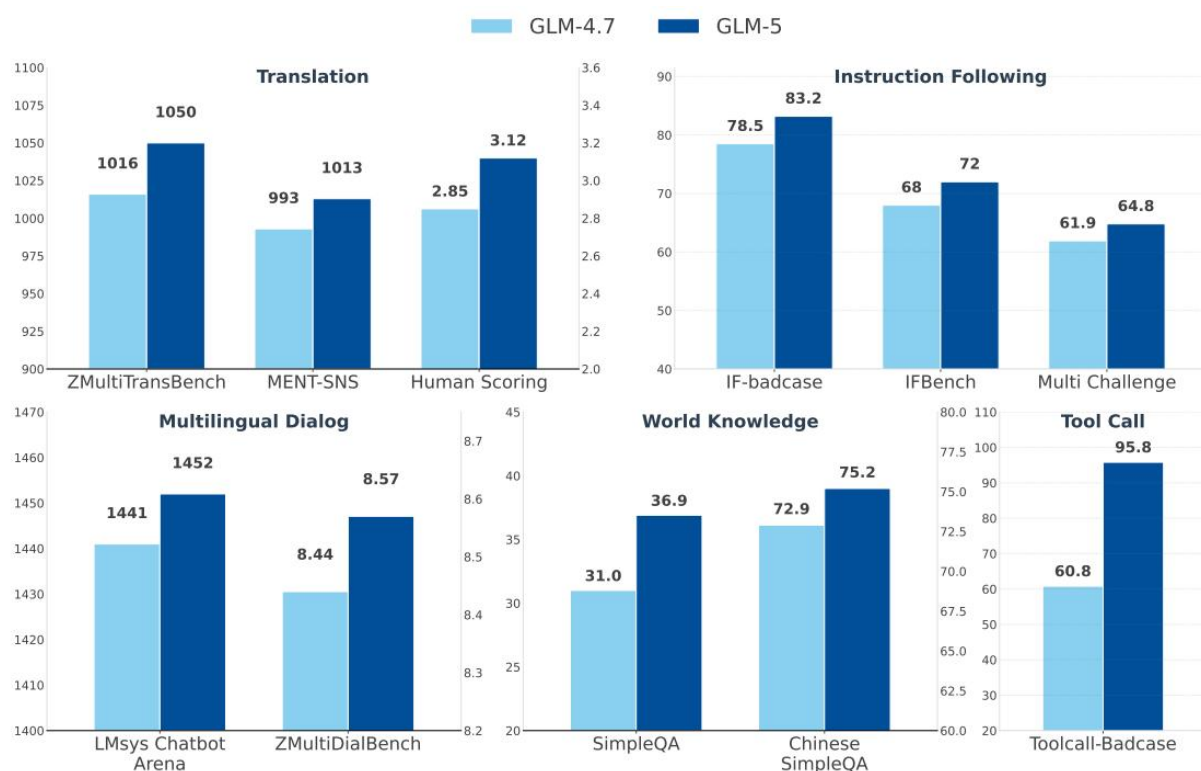


Figure 9: images/60b14cd771b02bec0a999547d6c609f28c2e85eb876d97a43e978184806bb5f6.jpg

Figure 11: Performance comparison between GLM-4.7 and GLM-5 across five real-world general ability domains.

图 11: GLM-4.7 与 GLM-5 在五个现实世界通用能力领域中的性能对比。

6.3 Evaluation of Real-world General Abilities

6.3 实际通用能力评估

While standardized academic benchmarks provide useful signals, they do not fully capture how models are used in practice. To recognize this gap, we evaluate GLM-5 on a set of real-world general abilities derived from high-frequency user interaction patterns observed in deployment settings. These abilities include machine translation, multilingual dialogue, instruction following, world knowledge, and tool-calling.

虽然标准化的学术基准指标提供了有用的信号，但它们并不能完全反映模型在实际应用中的使用方式。为了识别这一差距，我们在部署环境中观察到的高频用户交互模式的基础上，评估了 GLM-5 在一系列现实世界通用能力上的表现。这些能力包括机器翻译、多语言对话、遵循指示、世界知识和工具调用。

Unlike traditional benchmark-centric evaluation, our goal is to measure improvements that directly translate into user-perceived quality gains. For each capability, we adopt a combination of internal human evaluation, internal automated evaluation, external human assessment, and external automated benchmarks, ensuring both diagnostic granularity and cross-model comparability. When using external benchmarks, we prioritize datasets that reflect realistic interaction patterns rather than narrowly constructed test distributions.

与传统的以基准为中心的评估方法不同，我们的目标是衡量那些能够直接转化为用户感知质量提升的改进。对于每个能力，我们采用内部人工评估、内部自动化评估、外部人工评估以及外部自动化基准的组合，确保诊断的精细度和跨模型的可比性。在使用外部基准时，我们优先选择反映真实交互模式的数据集，而不是狭窄构建的测试分布。

Figure 11 presents the comparative results between GLM-5 and GLM-4.7 across five real-world capability domains. Across all evaluated dimensions, GLM-5 shows consistent improvements in machine translation, multilingual dialogue, instruction following, world knowledge, and tool-calling.

图 11 展示了 GLM-5 与 GLM-4.7 在五个现实能力领域中的对比结果。在所有评估维度中，GLM-5 在机器翻译、多语言对话、遵循指示、世界知识以及工具调用方面表现出持续的提升。

Detailed evaluation protocols and dataset descriptions for each ability are provided as follows.

每个能力的详细评估协议和数据集描述如下。

6.3.1 Machine Translation

6.3.1 机器翻译

ZMultiTransBench. This internal dataset comprises 1,220 samples sourced from self-collected high-frequency translation scenarios, covering seven language pairs: Zh to Es (300), Ru (250), Fr (220), Ko (200), Ja (150), Ar (50), and De (50). All samples were curated, translated, and independently verified by graduate students with formal training in translation studies. The dataset emphasizes naturally occurring usage contexts rather than artificially constructed test cases. Evaluation is conducted using pairwise comparison against a fixed baseline response. Judgments are provided by an automated evaluator based on GPT-4.1, which assesses semantic fidelity, fluency, and overall translation quality.

ZMultiTransBench。这个内部数据集包含 1,220 个样本，来源于自主收集的高频翻译场景，涵盖七种语言对：中到西（300）、俄（250）、法（220）、韩（200）、日（150）、阿（50）和德（50）。所有样本均由接受翻译研究正式培训的研究生进行筛选、翻译并独立验证。该数据集强调自然出现的使用场景，而非人工构建的测试案例。评估通过与固定基线响应进行成对比较进行。判断由基于 GPT-4.1 的自动化评估器提供，评估语义保真度、流畅性以及整体翻译质量。

MENT-SNS. To further evaluate robustness in linguistically challenging contexts, we adopt source sentences from MENT [46], comprising 753 English–Chinese sentence pairs across four domains: Social Network Services (SNS), Cross-Culture, Poetry, and Literature. These domains are selected to stress-test translation under complex linguistic phenomena, including slang, homophonic wordplay, idiomatic expressions, historical references, and metaphorical language. Similar to ZMultiTrans-Bench, all samples were curated and verified by professionally trained graduate students. Evaluation follows the same pairwise comparison protocol against a baseline response, with GPT-4.1 serving as the automated judge model.

MENT-SNS。为了进一步评估在语言挑战性情境下的鲁棒性，我们采用来自 MENT [46] 的源句，包含四个领域的 753 个英汉句子对：社交网络服务（SNS）、跨文化、诗歌和文学。这些领域被选中以在复杂语言现象下对翻译进行压力测试，包括俚语、同音双关、习语、历史引用和比喻语言。与 ZMultiTrans-Bench 类似，所有样本均由专业培训的研究生进行整理和验证。评估遵循与基线响应相同的成对比较协议，GPT-4.1 作为自动评判模型。

6.3.2 Multi-lingual Dialogue

6.3.2 多语言对话

LMarena. We report Elo ratings from the LMarena⁹, which are derived from large-scale, community-submitted pairwise comparisons. These ratings reflect relative model preference in open-ended dialogue settings and provide an external signal of conversational performance.

LMarena。我们报告来自 LMarena⁹ 的 Elo 分数，这些分数来源于大规模的社区提交的成对比较。这些评分反映了在开放式对话场景中模型的相对偏好，并提供了对话表现的外部信号。

ZMultiDialBench. In addition to the public leaderboard, we also conduct human evaluation on ZMultiDialBench, an internal multilingual dialogue benchmark. The dataset consists of 141 curated instances spanning diverse dialogue categories. Samples were collected from high-quality conversational data contributed by native-speaking annotators across multiple countries, as well as from challenging failure cases reported by online users. Human annotators assigned point-wise scores on a 1–10 scale to anonymized model responses according to category-specific, standardized evaluation criteria.

ZMultiDialBench。除了公开的排行榜，我们还在内部多语言对话基准 ZMultiDialBench 上进行人工评估。该数据集包含 141 个经过整理的实例，涵盖多样化的对话类别。样本来自多个国家的母语标注者贡献的高质量对话数据，以及在线用户报告的具有挑战性的失败案例。人工标注者根据特定类别、标准化的评估标准，对匿名化的模型响应进行 1-10 分的逐点评分。

6.3.3 Instruction Following

6.3.3 指令遵循

IF-Badcase. IF-Badcase is an internal benchmark constructed from instruction-following failure cases reported by real users in production settings. The dataset is designed to evaluate strict adherence to realistic, multi-constraint instructions, emphasizing procedural accuracy, logical consistency, and rigid formatting requirements. Evaluation is conducted using a detailed checklist-based protocol that verifies compliance with explicit constraints, including ordered steps, rule-based conditions, and structural specifications. All samples were annotated, reviewed, and iteratively filtered by human experts, resulting in a curated set of 450 test instances.

IF-Badcase。IF-Badcase 是一个内部基准测试集，由在实际生产环境中报告的指令遵循失败案例构建而成。该数据集旨在评估对现实场景中多约束条件指令的严格遵守，强调程序准确性、逻辑一致性以及严格的格式要求。评估采用基于详细检查清单的协议，验证对明确约束条件的遵守情况，包括有序步骤、基于规则的条件以及结构规范。所有样本均经过人工专家的标注、审查和迭代筛选，最终形成包含 450 个测试实例的精选数据集。

IF-Bench [36]. IF-Bench evaluates LLMs on their ability to adhere to complex, objective constraints, such as specific formatting rules, length limits, and content restrictions. It provides a quantitative measure of precise instruction-following capabilities, focusing on verifiable compliance rather than open-ended generation quality.

IF-Bench [36]。IF-Bench 评估大型语言模型在遵循复杂、客观约束方面的能力，例如特定的格式规则、长度限制和内容限制。它提供了一种对精确指令遵循能力的量化衡量，关注可验证的合规性，而非开放生成的质量。

MultiChallenge [41]. MultiChallenge examines LLMs via realistic, multi-turn conversational scenarios. It targets complex interactions requiring accurate instruction-following, context allocation, and in-context reasoning.

MultiChallenge [41]。MultiChallenge 通过现实的、多轮对话场景来考察大语言模型（LLMs）。它针对

需要准确遵循指令、上下文分配和上下文推理的复杂交互。

6.3.4 World Knowledge

6.3.4 世界知识

SimpleQA [49]. SimpleQA measures short-form factuality using challenging questions with single, indisputable answers. It evaluates a model's calibration by classifying responses as correct, incorrect, or not attempted, prioritizing accuracy over generation length.

SimpleQA [49]. SimpleQA 使用具有单一、不可争议答案的挑战性问题来衡量简短事实性问题。它通过将回答分类为正确、错误或未尝试来评估模型的校准，优先考虑准确性而非生成长度。

Chinese SimpleQA [16]. Adapting the SimpleQA methodology to the Chinese context, this benchmark evaluates factuality across six major domains and 99 subtopics. It utilizes high-quality, static, short-answer questions designed for reliable, automated grading to assess the knowledge accuracy of LLMs.

Chinese SimpleQA [16]. 将 SimpleQA 方法论适应到中文语境中，该基准测试在六个主要领域和 99 个子主题上评估事实性。它使用高质量、静态、简短答案的问题，这些问题设计用于可靠、自动评分，以评估大语言模型的知识准确性。

6.3.5 Tool Calling

6.3.5 工具调用

ToolCall-Badcase. ToolCall-Badcase is an internal benchmark derived from failure cases in tool invocation scenarios reported by users in production environments. Each instance is associated with a verifiable ground-truth tool call, enabling objective evaluation of both tool selection and argument

ToolCall-Badcase. ToolCall-Badcase 是一个内部基准，来源于生产环境中用户报告的工具调用场景中的失败案例。每个实例都与一个可验证的真实工具调用相关联，从而能够客观地评估工具选择和参数的准确性。

correctness. Evaluation assesses whether the model (1) invokes the correct tool and (2) provides correctly structured and semantically accurate arguments. All samples underwent multiple rounds of review, rewriting, and validation to remove ambiguity and ensure evaluability. The resulting dataset consists of 200 curated test cases that reflect realistic tool-calling abilities.

正确性。评估考察模型是否（1）调用了正确的工具，并且（2）提供了结构正确且语义准确的论据。所有样本都经历了多轮的审查、重写和验证，以消除歧义并确保可评估性。最终得到的测试数据集包含 200 个经过整理的测试用例，反映了现实中的工具调用能力。

7 Conclusion

7 结论

In this report, we have introduced GLM-5, a next-generation foundation model that fundamentally bridges the gap between high-performance reasoning and extreme computational efficiency. By transitioning from the paradigm of “vibe coding” to true “agentic engineering”, GLM-5 demonstrates that open-weight models can now rival the capabilities of top-tier proprietary systems in complex, real-world workflows. GLM-5 represents a paradigm shift in practical AI utility. By open-sourcing the model, we aim to empower the community to move beyond

static benchmarks and explore the frontiers of efficient, agentic general intelligence, fostering a new era where AI agents autonomously plan, implement, and iterate on complex tasks.

在本报告中，我们介绍了 GLM-5，一款下一代基础模型，它从根本上弥合了高性能推理与极端计算效率之间的差距。通过从“感觉编码”的范式转向真正的“自主工程”，GLM-5 展示了开放权重模型如今能够在复杂的现实世界工作流程中与顶级专有系统相媲美。GLM-5 代表了实用人工智能效用的范式转变。通过开源该模型，我们旨在赋能社区超越静态基准，探索高效、自主通用智能的前沿，开启一个新纪元，使 AI 代理能够自主规划、执行和迭代复杂任务。

8 Easter Eggs

8 个彩蛋

The “Pony Alpha” experiment was indeed a pivotal moment for us. It was a bold decision to release GLM-5 anonymously on OpenRouter, but the results have been incredibly validating. By stripping away our brand name, we allowed the model’s intrinsic capabilities to speak for themselves, ensuring the feedback we received was pure and unbiased. Here is a brief summary:

“Pony Alpha” 实验确实是我们的一个关键转折点。在 OpenRouter 上匿名发布 GLM-5 是一个大胆的决定，但结果却极具验证性。通过去除我们的品牌名称，我们让模型本身的内在能力来发声，确保我们收到的反馈是纯粹且无偏见的。以下是一个简要总结：

Within days, Pony Alpha became a sensation. Developers in the OpenRouter community began to notice its exceptional performance, particularly in complex coding tasks, agentic workflows, and roleplay scenarios.

几天内，Pony Alpha 就成为了一大热门。OpenRouter 社区的开发者们开始注意到它的卓越表现，尤其是在复杂的编码任务、代理工作流程和角色扮演场景中。

Speculation was rampant, with many users guessing it was a leaked update from labs like Anthropic (Claude Sonnet 5), a secret Grok release, or DeepSeek V4. A preliminary statistic shows that 25% of the users guessed it was Claude Sonnet 5, 20% DeepSeek, 10% Grok, and the rest GLM-5.

猜测层出不穷，许多用户猜测这可能是来自 Anthropic 等实验室泄露的更新（如 Claude Sonnet 5），或是秘密发布的 Grok 版本，或者是 DeepSeek V4。初步统计显示，有 25% 的用户猜测这是 Claude Sonnet 5，20% 猜测是 DeepSeek，10% 猜测是 Grok，其余的是 GLM-5。

The eventual confirmation that it was indeed our GLM-5 was a profound moment for us, effectively silencing doubts about whether Chinese LLMs could compete at the frontier level. The success of Pony Alpha (GLM-5) is not just about raw benchmarks; it signifies a shift in our focus towards engineering-level reliability.

最终确认它确实是我们的 GLM-5 的那一刻对我们来说是一个深刻的时刻，有效地消除了关于中国的 LLM 是否能在前沿水平竞争的疑虑。Pony Alpha (GLM-5) 的成功不仅仅是关于原始基准的胜利；它标志着我们开始将重点转向工程级别的可靠性。

This anonymous release allowed us to transcend geopolitical biases. The community embraced the model because it worked. While we celebrate this success, we must remain pragmatic. The gap between open-weight models and the absolute proprietary frontier is narrowing, but the race is far from over. Our focus remains steadfast on pushing the boundaries of what is possible with scalable, efficient, and intelligent systems.

这个匿名发布使我们超越了地缘政治偏见。社区接受了这个模型，因为它有效。尽管我们庆祝这一成功，我们必须保持务实。开源模型与绝对专有技术前沿之间的差距正在缩小，但这场竞赛远未结束。我们的重点始终坚定地放在推动可扩展、高效且智能系统所能实现的边界上。

9 Contribution

9 贡献

Contributors' names are listed in alphabetical order by first name.

贡献者的名字按名字的字母顺序排列。

Core Contributors

核心贡献者

Chendi Ge, Chenghua Huang, Chengxing Xie, Chenzheng Zhu, Congfeng Yin, Cunxiang Wang, Gengzheng Pan, Hao Zeng, Haoke Zhang, Haoran Wang, Huilong Chen, Jiajie Zhang, Jian Jiao, Jiaqi Guo, Jingsen Wang, Jingzhao Du, Jinzhu Wu, Kedong Wang, Lei Li, Lin Fan, Lucen Zhong, Mingdao Liu, Mingming Zhao, Pengfan Du, Qian Dong, Rui Lu, Shuang Li (李爽), Shulin Cao, Song Liu, Ting Jiang, Xiaodong Chen, Xiaohan Zhang, Xuancheng Huang, Xuezhen Dong, Yabo Xu, Yao Wei, Yifan An, Yilin Niu, Yitong Zhu, Yuanhao Wen, Yukuo Cen, Yushi Bai, Zhongpei Qiao, Zihan Wang, Zikang Wang, Zilin Zhu, Ziqiang Liu, Zixuan Li

陈地歌, 陈华黄, 陈星谢, 陈 Zheng 朱, 丛峰尹, 陈祥王, 梓 Zheng 潘, 郝曾, 郝珂张, 何然王, 陈慧龙, 张佳杰, 贾娇焦, 贾琪国, 王景森, 邓劲昭, 吴金柱, 王科东, 李磊, 范林, 钟禄成, 刘明道, 赵明明, 杜鹏帆, 东钱, 骆睿, 李爽, 曹树林, 刘松, 蒋婷, 陈晓东, 张晓涵, 黄宣城, 东雪贞, 许亚波, 韦尧, 安毅凡, 牛毅林, 朱怡彤, 文元浩, 曾宇科, 白宇石, 邱仲培, 王子涵, 王子康, 王子林, 朱子林, 刘子强, 李子轩

Contributors

贡献者

Bojie Wang, Bosi Wen, Can Huang, Changpeng Cai, Chao Yu, Chen Li, Chengwei Hu, Chenhui Zhang, Dan Zhang, Daoyan Lin, Dayong Yang, Di Wang, Ding Ai, Erle Zhu, Fangzhou Yi, Feiyu Chen, Guohong Wen, Hailong Sun, Haisha Zhao, Haiyi Hu, Hanchen Zhang, Hanrui Liu, Hanyu Zhang, Hao Peng, Hao Tai, Haobo Zhang, He Liu, Hongwei Wang, Hongxi Yan, Hongyu Ge, Huan Liu, Huanpeng Chu, Jia'ni Zhao, Jiachen Wang, Jiajing Zhao, Jiamin Ren, Jiapeng Wang, Jiaxin Zhang, Jiayi Gui, Jiayue Zhao, Jijie Li, Jing An, Jing Li, Jingwei Yuan, Jinhua Du, Jinxin Liu, Junkai Zhi, Junwen Duan, Kaiyue Zhou, Kangjian Wei, Ke Wang, Keyun Luo, Laiqiang Zhang, Leigang Sha, Liang Xu, Lindong Wu, Lintao Ding, Lu Chen, Minghao Li, Nianyi Lin, Pan Ta, Qiang Zou, Rongjun Song, Ruiqi Yang, Shangqing Tu, Shangtong Yang, Shaoxiang Wu, Shengyan Zhang, Shijie Li, Shuang Li (李滢), Shuyi Fan, Wei Qin, Wei Tian, Weining Zhang, Wenbo Yu, Wenjie Liang, Xiang Kuang, Xiangmeng Cheng, Xiangyang Li, Xiaoquan Yan, Xiaowei Hu, Xiaoying Ling, Xing Fan, Xingye Xia, Xinyuan Zhang, Xinze Zhang, Xirui Pan, Xu Zou, Xunkai Zhang, Yadi Liu, Yandong Wu, Yanfu Li, Yidong Wang, Yifan Zhu, Yijun Tan, Yilin Zhou, Yiming Pan, Ying Zhang, Yinpei Su, Yipeng Geng, Yong Yan, Yonglin Tan, Yuean Bi, Yuhao Shen, Yuhao Yang, Yujiang Li, Yunan Liu, Yunqing Wang, Yuntao Li, Yurong Wu, Yutao Zhang, Yuxi Duan, Yuxuan Zhang, Zezhen Liu, Zhengtao Jiang, Zhenhe Yan, Zheyu Zhang, Zhixiang Wei, Zhuo Chen, Zhuoer Feng, Zijun Yao, Ziwei Chai, Ziyuan Wang, Zuzhou Zhang

王博杰, 文博思, 黄灿, 蔡昌鹏, 于超, 陈莉, 胡成伟, 张晨辉, 张丹, 林道言, 杨道勇, 王迪, 艾丁, 朱珥乐, 易芳舟, 陈飞宇, 温国红, 孙海龙, 赵海沙, 胡海毅, 张涵晨, 刘涵睿, 张涵宇, 彭浩, 台浩, 张浩博, 刘鹤, 王宏伟, 闫红曦, 葛红宇, 刘欢, 褚焕鹏, 赵嘉妮, 王嘉辰, 赵嘉婧, 任嘉敏, 王嘉鹏, 张嘉欣, GUI 嘉仪, 赵嘉悦, 李继杰, 安婧, 李婧伟, 袁婧伟, 杜金花, 刘金鑫, 智俊凯, 段俊文, 周凯悦, 魏康健, 王凯云, 罗凯云, 张来强, 沙磊刚, 徐良, 吴岭东, 丁林涛, 陆晨, 李明浩, 林年义, 谭盼, 邹强, 宋荣军, 杨若琪, 杨上通, 吴少翔, 张生彦, 李世杰, 李双, 范舒毅, 秦伟, 田伟, 张文宁, 于文杰, 梁文杰, 康翔, 程翔梦, 李翔阳, 严小全, 胡晓伟, 凌晓英, 范兴, 夏兴晔, 张欣源, 张欣泽, 潘希睿, 邹旭, 张勋凯, 刘亚迪, 吴亚东, 李亚夫, 王毅东, 朱毅凡, 谭毅军, 周毅林, 潘毅明, 张英, 苏印培, 耿毅鹏, 严勇, 谭勇

林, 毕岳安, 沈雨涵, 杨雨浩, 李雨江, 刘云安, 王云清, 李云涛, 吴雨荣, 张雨涛, 段宇涛, 张宇轩, 刘泽臻, 江振涛, 闫振和, 张哲宇, 张振赫, 魏志强, 陈卓, 冯卓尔, 姚子均, 柴子威, 王子源, 张祖周

Tech Leads

技术负责人

Aohan Zeng, Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, Bin Chen, Da Yin

曾傲涵, 吕欣, 侯振宇, 杜正霄, 郑钦凯, 陈斌, 尹达

Advisors

顾问

Jie Tang, Yuxiao Dong, Juanzi Li, Hongning Wang, Minlie Huang, Bin Xu

解唐, 杜晓东, 李媛子, 王宏宁, 黄民列, 徐斌

Acknowledgement

致谢

We are grateful for all the support from our co-launch partners and community developers (Alphabetical order):

我们感谢所有合作首发伙伴和社区开发者的支持（按字母顺序排列）：

Open-source Communities: Hugging Face, MLX, ModelScope, SGLang, Unsloth, vLLM, xLLM

开源社区：Hugging Face, MLX, ModelScope, SGLang, Unsloth, vLLM, xLLM

Inference Providers: Amazon Bedrock, Atlas Cloud, Baidu AI Cloud, Baseten, Cerebras, DeepInfra, Fireworks, FriendlyAI, GMI Cloud, Google Cloud Vertex AI, Infinigence AI, Modal, Novita AI, Parasail, Phala, PPIO, SiliconFlow, StreamLake, Together AI, Venice, Weights & Biases

推理服务提供商：Amazon Bedrock, Atlas Cloud, Baidu AI Cloud, Baseten, Cerebras, DeepInfra, Fireworks, FriendlyAI, GMI Cloud, Google Cloud Vertex AI, Infinigence AI, Modal, Novita AI, Parasail, Phala, PPIO, SiliconFlow, StreamLake, Together AI, Venice, Weights & Biases

Applications: CatPaw, Cline, CodeBuddy, CodeRider, Coze, Crush, Factory AI, Kilo Code, MonkeyCode, OpenClaw, OpenCode, Qoder, Roo Code, TRAE, Verdent AI, WPS, YouWare

应用：CatPaw, Cline, CodeBuddy, CodeRider, Coze, Crush, Factory AI, Kilo Code, MonkeyCode, OpenClaw, OpenCode, Qoder, Roo Code, TRAE, Verdent AI, WPS, YouWare

AI Gateways: AI Ping, EZmodel, iFlow, OpenRouter, Vercel, Yupp, ZenMux

AI 网关：AI Ping, EZmodel, iFlow, OpenRouter, Vercel, Yupp, ZenMux

References

参考资料

- [1] Anthropic. System card: Claude opus 4.5, 2025.
- [2] S. Ashkboos, A. Mohtashami, M. L. Croci, B. Li, P. Cameron, M. Jaggi, D. Alistarh, T. Hoefler, and J. Hensman. Quarot: Outlier-free 4-bit inference in rotated llms, 2024.
- [3] A. Backlund and L. Petersson. Vending-bench: A benchmark for long-term coherence of autonomous agents. arXiv preprint arXiv:2502.15840, 2025.
- [4] I. Badertdinov, A. Golubev, M. Nekrashevich, A. Shevtsov, S. Karasik, A. Andriushchenko, M. Trofimova, D. Litvintseva, and B. Yangel. Swe-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents. arXiv preprint arXiv:2505.20411, 2025.
- [5] Y. Bai, S. Tu, J. Zhang, H. Peng, X. Wang, X. Lv, S. Cao, J. Xu, L. Hou, Y. Dong, J. Tang, and J. Li. LongBench v2: Towards deeper understanding and reasoning on realistic long-context multitasks. In ACL’25, pages 3639–3664, 2025.
- [6] C. Bandi, B. Hertzberg, G. Boo, T. Polakam, J. Da, S. Hassaan, M. Sharma, A. Park, E. Hernandez, D. Rambado, et al. Mcp-atlas: A large-scale benchmark for tool-use competency with real mcp servers. arXiv preprint arXiv:2602.00933, 2026.
- [7] V. Barres, H. Dong, S. Ray, X. Si, and K. Narasimhan. τ^2 -bench: Evaluating conversational agents in a dual-control environment. arXiv preprint arXiv:2506.07982, 2025.
- [8] G. DeepMind. Gemini 3 pro model card, 2025.
- [9] DeepSeek-AI, A. Liu, A. Mei, and et al. Deepseek-v3.2: Pushing the frontier of open large language models, 2025.
- [10] W. Du, S. Toshniwal, B. Kisacanin, S. Mahdavi, I. Moshkov, G. Armstrong, S. Ge, E. Minasyan, F. Chen, and I. Gitman. Nemotron-math: Efficient long-context distillation of mathematical reasoning from multi-mode supervision. arXiv preprint arXiv:2512.15489, 2025.
- [11] C. Gao, X. Wu, Z. Lin, D. Zhang, and S. Hu. Nextlong: Toward effective long-context training without long documents, 2025.
- [12] H. Ge, J. Feng, Q. Huang, F. Fu, X. Nie, L. Zuo, H. Lin, B. Cui, and X. Liu. Bytescale: Efficient scaling of llm training with a 2048k context length on more than 12,000 gpus. arXiv preprint arXiv:2502.21231, 2025.
- [13] F. Gloeckle, B. Y. Idrissi, B. Rozière, D. Lopez-Paz, and G. Synnaeve. Better & faster large language models via multi-token prediction. arXiv preprint arXiv:2404.19737, 2024.
- [14] Y. Gu, L. Dong, F. Wei, and M. Huang. Minillm: Knowledge distillation of large language models. In ICLR’23, 2025.
- [15] Y. Gu, Q. Hu, S. Yang, H. Xi, J. Chen, S. Han, and H. Cai. Jet-nemotron: Efficient language model with post neural architecture search. arXiv preprint arXiv:2508.15884, 2025.
- [16] Y. He, S. Li, J. Liu, Y. Tan, W. Wang, H. Huang, X. Bu, H. Guo, C. Hu, B. Zheng, Z. Lin, X. Liu, D. Sun, S. Lin, Z. Zheng, X. Zhu, W. Su, and B. Zheng. Chinese simpleqa: A chinese factuality evaluation for large language models, 2024.
- [17] C.-P. Hsieh, S. Sun, S. Krizan, S. Acharya, D. Rekesh, F. Jia, and B. Ginsburg. Ruler: What’s the real context size of your long-context language models? In COLM’24, 2024.
- [18] J. Jia, Z. Chen, X. Wu, C. Gao, Z. Lin, D. Zhang, S. Hu, and B. Guo. Entropylong: Effective long-context training via predictive uncertainty, 2025.
- [19] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. Swe-bench: Can language models resolve real-world github issues? arXiv preprint arXiv:2310.06770, 2023.
- [20] Y. Leviathan, M. Kalman, and Y. Matias. Fast inference from transformers via speculative decoding. In ICML’23, pages 19274–19286, 2023.
- [21] J. Li, A. Fang, G. Smyrnis, M. Ivgi, and et al. Datacomp-lm: In search of the next generation of training sets for language models, 2025.
- [22] J. Li, W. Zhao, J. Zhao, W. Zeng, H. Wu, X. Wang, R. Ge, Y. Cao, Y. Huang, W. Liu, et al. The tool decathlon: Benchmarking language agents for diverse, realistic, and long-horizon task execution. arXiv preprint arXiv:2510.25726, 2025.
- [23] R. Li, J. Fu, B.-W. Zhang, T. Huang, Z. Sun, C. Lyu, G. Liu, Z. Jin, and G. Li. Taco: Topics in algorithmic code generation dataset. arXiv preprint arXiv:2312.14852, 2023.
- [24] A. Liu, B. Feng, B. Wang, B. Wang, B. Liu, C. Zhao, C. Dengr, C. Ruan, D. Dai, D. Guo, et

- al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. arXiv preprint arXiv:2405.04434, 2024.
- [25] A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, et al. Deepseek-v3 technical report. arXiv preprint arXiv:2412.19437, 2024.
- [26] A. Liu, A. Mei, B. Lin, B. Xue, B. Wang, B. Xu, B. Wu, B. Zhang, C. Lin, C. Dong, et al. Deepseek-v3. 2: Pushing the frontier of open large language models. arXiv preprint arXiv:2512.02556, 2025.
- [27] J. Liu, J. Le Tian, V. Daita, Y. Wei, Y. Ding, Y. K. Wang, J. Yang, and L. ZHANG. Repoqa: Evaluating long context code understanding. In First Workshop on Long-Context Foundation Models@ ICML 2024.
- [28] K. Lu and T. M. Lab. On-policy distillation. Thinking Machines Lab: Connectionism, 2025. <https://thinkingmachines.ai/blog/on-policy-distillation>.
- [29] M.-T. Luong, D. Hwang, H. H. Nguyen, G. Ghiasi, Y. Chervonyi, I. Seo, J. Kim, G. Bingham, J. Lee, S. Mishra, et al. Towards robust mathematical reasoning. In EMNLP’25, pages 35406-35430, 2025.
- [30] I. Moshkov, D. Hanley, I. Sorokin, S. Toshniwal, C. Henkel, B. Schifferer, W. Du, and I. Gitman. Aimo-2 winning solution: Building state-of-the-art mathematical reasoning models with openmathreasoning dataset. arXiv preprint arXiv:2504.16891, 2025.
- [31] D. Narayanan, M. Shoeybi, J. Casper, P. LeGresley, M. Patwary, V. A. Korthikanti, D. Vainbrand, P. Kashinkunti, J. Bernauer, B. Catanzaro, A. Phanishayee, and M. Zaharia. Efficient large-scale language model training on gpu clusters using megatron-lm, 2021.
- [32] OpenAI. Introducing gpt 5.2, 2025.
- [33] T. Patwardhan, R. Dias, E. Proehl, G. Kim, M. Wang, O. Watkins, S. P. Fishman, M. Aljubei, P. Thacker, L. Fauconnet, et al. Gdpval: Evaluating ai model performance on real-world economically valuable tasks. arXiv preprint arXiv:2510.04374, 2025.
- [34] L. Phan, A. Gatti, Z. Han, N. Li, J. Hu, H. Zhang, C. B. C. Zhang, M. Shaaban, J. Ling, S. Shi, et al. Humanity’s last exam. arXiv preprint arXiv:2501.14249, 2025.
- [35] Prime Intellect. Synthetic-2 release: Four million collaboratively generated reasoning traces, 2025. Blog post.
- [36] V. Pyatkin, S. Malik, V. Graf, H. Ivison, S. Huang, P. Dasigi, N. Lambert, and H. Hajishirzi. Generalizing verifiable instruction following, 2025.
- [37] P. Qi, X. Wan, G. Huang, and M. Lin. Zero bubble pipeline parallelism. arXiv preprint arXiv:2401.10241, 2023.
- [38] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He. Zero: Memory optimizations toward training trillion parameter models, 2020.
- [39] D. Rein, B. L. Hou, A. C. Stickland, J. Petty, R. Y. Pang, J. Dirani, J. Michael, and S. R. Bowman. Gpqa: A graduate-level google-proof q&a benchmark. In CoLM’24, 2024.
- [40] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. Li, Y. Wu, et al. Deepseek-math: Pushing the limits of mathematical reasoning in open language models. arXiv preprint arXiv:2402.03300, 2024.
- [41] V. Sirdeshmukh, K. Deshpande, J. Mols, L. Jin, E.-Y. Cardona, D. Lee, J. Kritz, W. Primack, S. Yue, and C. Xing. Multichallenge: A realistic multi-turn conversation evaluation benchmark challenging to frontier llms, 2025.
- [42] H. F. Team. Harbor: A framework for evaluating and optimizing agents and models in container environments., 2026.
- [43] K. Team, T. Bai, Y. Bai, Y. Bao, S. Cai, Y. Cao, Y. Charles, H. Che, C. Chen, G. Chen, et al. Kimi k2. 5: Visual agentic intelligence. arXiv preprint arXiv:2602.02276, 2026.
- [44] L. Team, A. Shen, B. Li, B. Hu, B. Jing, C. Chen, C. Huang, C. Zhang, C. Yang, C. Lin, et al. Every step evolves: Scaling reinforcement learning for trillion-scale thinking model. arXiv preprint arXiv:2510.18855, 2025.
- [45] T. T.-B. Team. Terminal-bench: A benchmark for ai agents in terminal environments, Apr 2025.
- [46] Y. Tian, C. Wang, Z. Liu, H. Huang, W. Yu, D. Song, J. Tang, and Y. Guo. Beyond literal mapping: Benchmarking and improving non-literal translation evaluation, 2026.
- [47] Y. Wang, S. Wang, S. Zhu, F. Fu, X. Liu, X. Xiao, H. Li, J. Li, F. Wu, and B. Cui. Flexsp: Accelerating large language model training via flexible sequence parallelism. In ASPLOS’25, pages 421–436, 2025.
- [48] Z. Wang, T. Shi, J. He, M. Cai, J. Zhang, and D. Song. Cybergym: Evaluating ai agents’ cyber-

security capabilities with real-world vulnerabilities at scale. arXiv preprint arXiv:2506.02548, 2025.

[49] J. Wei, N. Karina, H. W. Chung, Y. J. Jiao, S. Papay, A. Glaese, J. Schulman, and W. Fedus. Measuring short-form factuality in large language models, 2024.

[50] J. Wei, Z. Sun, S. Papay, S. McKinney, J. Han, I. Fulford, H. W. Chung, A. T. Passos, W. Fedus, and A. Glaese. Browsecomp: A simple yet challenging benchmark for browsing agents. arXiv preprint arXiv:2504.12516, 2025.

[51] L.-C. Xiaomi. Mimo-v2-flash technical report, 2026.

[52] A. Yang, A. Li, B. Yang, B. Zhang, and et al. Qwen3 technical report. arXiv preprint arXiv:2505.09388, 2025.

[53] J. Yang, K. Lieret, C. E. Jimenez, A. Wettig, K. Khandpur, Y. Zhang, B. Hui, O. Press, L. Schmidt, and D. Yang. Swe-smith: Scaling data for software engineering agents. arXiv preprint arXiv:2504.21798, 2025.

[54] S. Yang, J. Kautz, and A. Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule. In ICLR’24, 2024.

[55] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan. tau-bench: A benchmark for tool-agent-user interaction in real-world domains. arXiv preprint arXiv:2406.12045, 2024.

[56] H. Yen, T. Gao, M. Hou, K. Ding, D. Fleischer, P. Izsak, M. Wasserblat, and D. Chen. Helmet: How to evaluate long-context language models effectively and thoroughly. arXiv preprint arXiv:2410.02694, 2024.

[57] Q. Yu, Z. Zhang, R. Zhu, Y. Yuan, X. Zuo, Y. Yue, W. Dai, T. Fan, G. Liu, L. Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. arXiv preprint arXiv:2503.14476, 2025.

[58] T. Yuan, Y. Liu, X. Ye, S. Zhang, J. Tan, B. Chen, C. Song, and D. Zhang. Accelerating the training of large language models using efficient activation rematerialization and optimal hybrid parallelism. In USENIX ATC’24, pages 545–561, 2024.

[59] L. Zhang, S. He, C. Zhang, Y. Kang, B. Li, C. Xie, J. Wang, M. Wang, Y. Huang, S. Fu, E. Nallipogu, Q. Lin, Y. Dang, S. Rajmohan, and D. Zhang. Swe-bench goes live! arXiv preprint arXiv:2505.23419, 2025.

[60] C. Zhao, C. Deng, C. Ruan, D. Dai, H. Gao, J. Li, L. Zhang, P. Huang, S. Zhou, S. Ma, et al. Insights into deepseek-v3: Scaling challenges and reflections on hardware for ai architectures. In ISCA’25, pages 1731–1745, 2025.

[61] X. Zhao, Y. Liu, K. Xu, J. Guo, Z. Wang, Y. Sun, X. Kong, Q. Cao, L. Jiang, Z. Wen, Z. Zhang, and J. Zhou. Small leak can sink a great ship—boost rl training on moe with icepop!, Sep 2025.

[62] C. Zheng, S. Liu, M. Li, X.-H. Chen, B. Yu, C. Gao, K. Dang, Y. Liu, R. Men, A. Yang, et al. Group sequence policy optimization. arXiv preprint arXiv:2507.18071, 2025.

[63] P. Zhou, B. Leon, X. Ying, C. Zhang, Y. Shao, Q. Ye, D. Chong, Z. Jin, C. Xie, M. Cao, et al. Browsecomp-zh: Benchmarking web browsing ability of large language models in chinese. arXiv preprint arXiv:2504.19314, 2025.

[1] Anthropic. System card: Claude opus 4.5, 2025.

[2] S. Ashkboos, A. Mohtashami, M. L. Croci, B. Li, P. Cameron, M. Jaggi, D. Alistarh, T. Hoefler, and J. Hensman. Quarot: Outlier-free 4-bit inference in rotated llms, 2024.

[3] A. Backlund and L. Petersson. Vending-bench: A benchmark for long-term coherence of autonomous agents. arXiv preprint arXiv:2502.15840, 2025.

[4] I. Badertdinov, A. Golubev, M. Nekrashevich, A. Shevtsov, S. Karasik, A. Andriushchenko, M. Trofimova, D. Litvintseva, and B. Yangel. Swe-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents. arXiv preprint arXiv:2505.20411, 2025.

[5] Y. Bai, S. Tu, J. Zhang, H. Peng, X. Wang, X. Lv, S. Cao, J. Xu, L. Hou, Y. Dong, J. Tang, and J. Li. LongBench v2: Towards deeper understanding and reasoning on realistic long-context multitasks. In ACL’25, pages 3639–3664, 2025.

[6] C. Bandi, B. Hertzberg, G. Boo, T. Polakam, J. Da, S. Hassaan, M. Sharma, A. Park, E. Hernandez, D. Rambado, et al. Mcp-atlas: A large-scale benchmark for tool-use competency with real mcp servers. arXiv preprint arXiv:2602.00933, 2026.

[7] V. Barres, H. Dong, S. Ray, X. Si, and K. Narasimhan. τ^2 -bench: Evaluating conversational agents in a dual-control environment. arXiv preprint arXiv:2506.07982, 2025.

[8] G. DeepMind. Gemini 3 pro model card, 2025.

[9] DeepSeek-AI, A. Liu, A. Mei, and et al. Deepseek-v3.2: Pushing the frontier of open large

language models, 2025.

[10] W. Du, S. Toshniwal, B. Kisacanin, S. Mahdavi, I. Moshkov, G. Armstrong, S. Ge, E. Minasyan, F. Chen, and I. Gitman. Nemotron-math: Efficient long-context distillation of mathematical reasoning from multi-mode supervision. arXiv preprint arXiv:2512.15489, 2025.

[11] C. Gao, X. Wu, Z. Lin, D. Zhang, and S. Hu. Nextlong: Toward effective long-context training without long documents, 2025.

[12] H. Ge, J. Feng, Q. Huang, F. Fu, X. Nie, L. Zuo, H. Lin, B. Cui, and X. Liu. Bytescale: Efficient scaling of llm training with a 2048k context length on more than 12,000 gpus. arXiv preprint arXiv:2502.21231, 2025.

[13] F. Gloeckle, B. Y. Idrissi, B. Rozière, D. Lopez-Paz, and G. Synnaeve. Better & faster large language models via multi-token prediction. arXiv preprint arXiv:2404.19737, 2024.

[14] Y. Gu, L. Dong, F. Wei, and M. Huang. Minillm: Knowledge distillation of large language models. In ICLR’23, 2025.

[15] Y. Gu, Q. Hu, S. Yang, H. Xi, J. Chen, S. Han, and H. Cai. Jet-nemotron: Efficient language model with post neural architecture search. arXiv preprint arXiv:2508.15884, 2025.

[16] Y. He, S. Li, J. Liu, Y. Tan, W. Wang, H. Huang, X. Bu, H. Guo, C. Hu, B. Zheng, Z. Lin, X. Liu, D. Sun, S. Lin, Z. Zheng, X. Zhu, W. Su, and B. Zheng. Chinese simpleqa: A chinese factuality evaluation for large language models, 2024.

[17] C.-P. Hsieh, S. Sun, S. Krizan, S. Acharya, D. Rekesh, F. Jia, and B. Ginsburg. Ruler: What’s the real context size of your long-context language models? In COLM’24, 2024.

[18] J. Jia, Z. Chen, X. Wu, C. Gao, Z. Lin, D. Zhang, S. Hu, and B. Guo. Entropylong: Effective long-context training via predictive uncertainty, 2025.

[19] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. Swe-bench: Can language models resolve real-world github issues? arXiv preprint arXiv:2310.06770, 2023.

[20] Y. Leviathan, M. Kalman, and Y. Matias. Fast inference from transformers via speculative decoding. In ICML’23, pages 19274-19286, 2023.

[21] J. Li, A. Fang, G. Smyrnis, M. Ivgi, and et al. Datacomp-lm: In search of the next generation of training sets for language models, 2025.

[22] J. Li, W. Zhao, J. Zhao, W. Zeng, H. Wu, X. Wang, R. Ge, Y. Cao, Y. Huang, W. Liu, et al. The tool decathlon: Benchmarking language agents for diverse, realistic, and long-horizon task execution. arXiv preprint arXiv:2510.25726, 2025.

[23] R. Li, J. Fu, B.-W. Zhang, T. Huang, Z. Sun, C. Lyu, G. Liu, Z. Jin, and G. Li. Taco: Topics in algorithmic code generation dataset. arXiv preprint arXiv:2312.14852, 2023.

[24] A. Liu, B. Feng, B. Wang, B. Wang, B. Liu, C. Zhao, C. Deng, C. Ruan, D. Dai, D. Guo, et al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. arXiv preprint arXiv:2405.04434, 2024.

[25] A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, et al. Deepseek-v3 technical report. arXiv preprint arXiv:2412.19437, 2024.

[26] A. Liu, A. Mei, B. Lin, B. Xue, B. Wang, B. Xu, B. Wu, B. Zhang, C. Lin, C. Dong, et al. Deepseek-v3. 2: Pushing the frontier of open large language models. arXiv preprint arXiv:2512.02556, 2025.

[27] J. Liu, J. Le Tian, V. Daita, Y. Wei, Y. Ding, Y. K. Wang, J. Yang, and L. ZHANG. Repoqa: Evaluating long context code understanding. In First Workshop on Long-Context Foundation Models@ ICML 2024.

[28] K. Lu and T. M. Lab. On-policy distillation. Thinking Machines Lab: Connectionism, 2025. <https://thinkingmachines.ai/blog/on-policy-distillation>.

[29] M.-T. Luong, D. Hwang, H. H. Nguyen, G. Ghiasi, Y. Chervonyi, I. Seo, J. Kim, G. Bingham, J. Lee, S. Mishra, et al. Towards robust mathematical reasoning. In EMNLP’25, pages 35406-35430, 2025.

[30] I. Moshkov, D. Hanley, I. Sorokin, S. Toshniwal, C. Henkel, B. Schifferer, W. Du, and I. Gitman. Aimo-2 winning solution: Building state-of-the-art mathematical reasoning models with openmathreasoning dataset. arXiv preprint arXiv:2504.16891, 2025.

[31] D. Narayanan, M. Shoenybi, J. Casper, P. LeGresley, M. Patwary, V. A. Korthikanti, D. Vainbrand, P. Kashinkunti, J. Bernauer, B. Catanzaro, A. Phanishayee, and M. Zaharia. Efficient large-scale language model training on gpu clusters using megatron-lm, 2021.

[32] OpenAI. Introducing gpt 5.2, 2025.

[33] T. Patwardhan, R. Dias, E. Proehl, G. Kim, M. Wang, O. Watkins, S. P. Fishman, M. Aljube, P. Thacker, L. Fauconnet, et al. Gdpval: Evaluating ai model performance on real-world economi-

cally valuable tasks. arXiv preprint arXiv:2510.04374, 2025.

[34] L. Phan, A. Gatti, Z. Han, N. Li, J. Hu, H. Zhang, C. B. C. Zhang, M. Shaaban, J. Ling, S. Shi, et al. Humanity’s last exam. arXiv preprint arXiv:2501.14249, 2025.

[35] Prime Intellect. Synthetic-2 release: Four million collaboratively generated reasoning traces, 2025. Blog post.

[36] V. Pyatkin, S. Malik, V. Graf, H. Ivison, S. Huang, P. Dasigi, N. Lambert, and H. Hajishirzi. Generalizing verifiable instruction following, 2025.

[37] P. Qi, X. Wan, G. Huang, and M. Lin. Zero bubble pipeline parallelism. arXiv preprint arXiv:2401.10241, 2023.

[38] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He. Zero: Memory optimizations toward training trillion parameter models, 2020.

[39] D. Rein, B. L. Hou, A. C. Stickland, J. Petty, R. Y. Pang, J. Dirani, J. Michael, and S. R. Bowman. Gpqa: A graduate-level google-proof q&a benchmark. In CoLM’24, 2024.

[40] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. Li, Y. Wu, et al. Deepseek-math: Pushing the limits of mathematical reasoning in open language models. arXiv preprint arXiv:2402.03300, 2024.

[41] V. Sirdeshmukh, K. Deshpande, J. Mols, L. Jin, E.-Y. Cardona, D. Lee, J. Kritz, W. Primack, S. Yue, and C. Xing. Multichallenge: A realistic multi-turn conversation evaluation benchmark challenging to frontier llms, 2025.

[42] H. F. Team. Harbor: A framework for evaluating and optimizing agents and models in container environments., 2026.

[43] K. Team, T. Bai, Y. Bai, Y. Bao, S. Cai, Y. Cao, Y. Charles, H. Che, C. Chen, G. Chen, et al. Kimi k2. 5: Visual agentic intelligence. arXiv preprint arXiv:2602.02276, 2026.

[44] L. Team, A. Shen, B. Li, B. Hu, B. Jing, C. Chen, C. Huang, C. Zhang, C. Yang, C. Lin, et al. Every step evolves: Scaling reinforcement learning for trillion-scale thinking model. arXiv preprint arXiv:2510.18855, 2025.

[45] T. T.-B. Team. Terminal-bench: A benchmark for ai agents in terminal environments, Apr 2025.

[46] Y. Tian, C. Wang, Z. Liu, H. Huang, W. Yu, D. Song, J. Tang, and Y. Guo. Beyond literal mapping: Benchmarking and improving non-literal translation evaluation, 2026.

[47] Y. Wang, S. Wang, S. Zhu, F. Fu, X. Liu, X. Xiao, H. Li, J. Li, F. Wu, and B. Cui. Flexsp: Accelerating large language model training via flexible sequence parallelism. In ASPLOS’25, pages 421–436, 2025.

[48] Z. Wang, T. Shi, J. He, M. Cai, J. Zhang, and D. Song. Cybergym: Evaluating ai agents’ cybersecurity capabilities with real-world vulnerabilities at scale. arXiv preprint arXiv:2506.02548, 2025.

[49] J. Wei, N. Karina, H. W. Chung, Y. J. Jiao, S. Papay, A. Glaese, J. Schulman, and W. Fedus. Measuring short-form factuality in large language models, 2024.

[50] J. Wei, Z. Sun, S. Papay, S. McKinney, J. Han, I. Fulford, H. W. Chung, A. T. Passos, W. Fedus, and A. Glaese. Browsecomp: A simple yet challenging benchmark for browsing agents. arXiv preprint arXiv:2504.12516, 2025.

[51] L.-C. Xiaomi. Mimo-v2-flash technical report, 2026.

[52] A. Yang, A. Li, B. Yang, B. Zhang, and et al. Qwen3 technical report. arXiv preprint arXiv:2505.09388, 2025.

[53] J. Yang, K. Lieret, C. E. Jimenez, A. Wettig, K. Khandpur, Y. Zhang, B. Hui, O. Press, L. Schmidt, and D. Yang. Swe-smith: Scaling data for software engineering agents. arXiv preprint arXiv:2504.21798, 2025.

[54] S. Yang, J. Kautz, and A. Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule. In ICLR’24, 2024.

[55] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan. tau-bench: A benchmark for tool-agent-user interaction in real-world domains. arXiv preprint arXiv:2406.12045, 2024.

[56] H. Yen, T. Gao, M. Hou, K. Ding, D. Fleischer, P. Izsak, M. Wasserblat, and D. Chen. Helmet: How to evaluate long-context language models effectively and thoroughly. arXiv preprint arXiv:2410.02694, 2024.

[57] Q. Yu, Z. Zhang, R. Zhu, Y. Yuan, X. Zuo, Y. Yue, W. Dai, T. Fan, G. Liu, L. Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. arXiv preprint arXiv:2503.14476, 2025.

[58] T. Yuan, Y. Liu, X. Ye, S. Zhang, J. Tan, B. Chen, C. Song, and D. Zhang. Accelerating the training of large language models using efficient activation rematerialization and optimal hybrid

parallelism. In USENIX ATC'24, pages 545–561, 2024.

[59] L. Zhang, S. He, C. Zhang, Y. Kang, B. Li, C. Xie, J. Wang, M. Wang, Y. Huang, S. Fu, E. Nallipogu, Q. Lin, Y. Dang, S. Rajmohan, and D. Zhang. Swe-bench goes live! arXiv preprint arXiv:2505.23419, 2025.

[60] C. Zhao, C. Deng, C. Ruan, D. Dai, H. Gao, J. Li, L. Zhang, P. Huang, S. Zhou, S. Ma, et al. Insights into deepseek-v3: Scaling challenges and reflections on hardware for ai architectures. In ISCA'25, pages 1731–1745, 2025.

[61] X. Zhao, Y. Liu, K. Xu, J. Guo, Z. Wang, Y. Sun, X. Kong, Q. Cao, L. Jiang, Z. Wen, Z. Zhang, and J. Zhou. Small leak can sink a great ship—boost rl training on moe with icepop!, Sep 2025.

[62] C. Zheng, S. Liu, M. Li, X.-H. Chen, B. Yu, C. Gao, K. Dang, Y. Liu, R. Men, A. Yang, et al. Group sequence policy optimization. arXiv preprint arXiv:2507.18071, 2025.

[63] P. Zhou, B. Leon, X. Ying, C. Zhang, Y. Shao, Q. Ye, D. Chong, Z. Jin, C. Xie, M. Cao, et al. Browsecmp-zh: Benchmarking web browsing ability of large language models in chinese. arXiv preprint arXiv:2504.19314, 2025.

A Hyper-Parameters

A Hyper-Parameters

Hyper-parameters related to the model architecture of GLM-5 are shown in Table 10.

与 GLM-5 模型架构相关的超参数如表 10 所示。

For training, we follow the setting of GLM-4.5, including the Muon optimizer, cosine decay, and batch size warmup. The learning rate goes through a warmup stage from 0 to $2e-4$, and a decaying stage to $4e-5$ until the end of the pre-training stage. In the mid-training stage, the learning rate decreases linearly from $4e-5$ to $1e-5$. Other hyper-parameters are the same as those of GLM-4.5. For DSA warmup stage, the learning rate goes down from $5e-3$ to $2e-4$. For DSA sparse adaptation stage, we use a constant learning rate of $1e-5$.

在训练过程中，我们遵循 GLM-4.5 的设置，包括 Muon 优化器、余弦衰减和批量大小预热。学习率从 0 开始，经过一个预热阶段逐渐上升至 $2e-4$ ，并在预训练阶段结束前衰减至 $4e-5$ 。在中间训练阶段，学习率从 $4e-5$ 线性下降至 $1e-5$ 。其他超参数与 GLM-4.5 相同。在 DSA 预热阶段，学习率从 $5e-3$ 下降到 $2e-4$ 。在 DSA 稀疏适应阶段，我们使用恒定的学习率 $1e-5$ 。

Table 10: Model architecture of GLM-4.5 and GLM-5. When counting parameters, for all models we include the parameters of MTP layers but not word embeddings and the output layer.

表 10: GLM-4.5 和 GLM-5 的模型架构。在计算参数时，我们包含所有模型的 MTP 层参数，但不包括词嵌入和输出层的参数。

Model	GLM-4.5	GLM-5
# Total Parameters	355B	744B
# Activated Parameters	32B	40B
# Dense Layers	3	3
# MoE Layers	89	75
# MTP Layers	1	1
Hidden Dim	5120	6144
Dense Intermediate Dim	12288	12288
MoE Intermediate Dim	1536	2048
QK Head Dim	128	192
V Head Dim	128	256

Q LoRA Dim	-	2048
KV LoRA Dim	-	512
# Attention Heads	96	64
# Key-Value Heads	8	-
# Indexer Attn Heads	-	32
# Indexer Head Dim	-	128
# Experts (total)	160	256
# Routed Experts	8	8
# Shared Experts	1	1
Vocabulary Size	151552	154880

模型	GLM-4.5	GLM-5
# 总参数数	355B	744B
# 激活的参数	32B	40B
# 密集层	3	3
# MoE 层	89	75
# MTP 层次结构	1	1
隐藏维度	5120	6144
密集中间维度	12288	12288
MoE 中间维度	1536	2048
QK 头部尺寸	128	192
V 头尺寸	128	256
Q LoRA Dim	-	2048
KV LoRA 维度	-	512
# 注意力头	96	64
# 键值头	8	-
# 索引器注意力头	-	32
# 索引头尺寸	-	128
# 专家 (总计)	160	256
# 路由专家	8	8
# 共享专家	1	1
词汇量	151552	154880

B Evaluation Details

B 评估细节

B.1 Evaluation of Base Models

B.1 基础模型评估

We evaluate the base model of GLM-5 with English, Chinese, code, and math benchmarks in Table 11.

我们在表 11 中评估了 GLM-5 的基础模型在英语、中文、代码和数学基准上的表现。

B.2 Evaluation of ARC Benchmarks

B.2 ARC 基准评估

Humanity's Last Exam (HLE) & other reasoning tasks: We evaluate with a maximum generation length of 131,072 tokens ($temperature = 1.0$, $top_p = 0.95$, $max_new_tokens = 131072$). By default, we report the text-only subset; results marked with * are from the full set. We use GPT-5.2 (medium) as the judge model. For HLE-with-tools, we use a maximum context length of 202,752 tokens.

人类最后的考试 (HLE) 及其他推理任务：我们使用最大生成长度为 131,072 个标记进行评估 ($temperature = 1.0$, $top_p = 0.95$, $max_new_tokens = 131072$)。默认情况下，我们报告纯文本子集的结果；标有 * 的结果来自完整数据集。我们使用 GPT-5.2 (中等) 作为评判模型。对于 HLE-with-tools，我们使用最大上下文长度为 202,752 个标记。

SWE-bench & SWE-bench Multilingual: We run the SWE-bench suite with OpenHands using a tailored instruction prompt. Settings: $temperature = 0.7$, $top_p = 0.95$, $max_new_tokens = 16384$, with a 200K context window.

SWE-bench & SWE-bench Multilingual: 我们使用 OpenHands 运行 SWE-bench 套件，并采用定制的指令提示。设置： $temperature = 0.7$, $top_p = 0.95$, $max_new_tokens = 16384$ ，上下文窗口为 200K。

BrowseComp: Without context management, we retain details from the most recent 5 turns. With context management, we use the same discard-all strategy as DeepSeek-V3.2 and Kimi K2.5.

BrowseComp: 在没有上下文管理的情况下，我们保留最近的 5 轮对话细节。在使用上下文管理时，我们采用与 DeepSeek-V3.2 和 Kimi K2.5 相同的全部丢弃策略。

Table 11: Comparison among GLM-5-Base, GLM-4.5-Base and other representative open-source base models.

表 11: GLM-5-Base、GLM-4.5-Base 与其他代表性开源基础模型的比较。

	Benchmark (Metric)	DeepSeek-V3 Base	Kimi-K2 Base	GLM-4.5 Base	GLM-5 Base
	Architecture	MoE	MoE	MoE	MoE
# Activated Params	37B	32B	32B	40B	37B
# Total Params	671B	1043B	355B	744B	671B
English	SimpleQA (EM)	26.6	35.3	30.0	36.1
BBH (EM)	88.4	88.7	86.2	87.4	88.4

MMLU (EM)	87.2	87.8	86.1	88.3	
HellaSwag (EM)	88.9	94.6	87.1	88.1	
PIQA (EM)	84.7	-	85.3	84.6	
TriviaQA (EM)	82.9	85.1	80.0	80.9	
Code	EvalPlus (Pass@1)	65.6	80.3	78.1	87.0
LiveCodeBench-Base (Pass@1)	24.6	26.3	28.1	34.4	
Math	GSM8K (EM)	87.6	92.1	79.4	68.8
MATH (EM)	62.6	70.2	61.0	56.4	
Chinese	CLUEWSC (EM)	82.7	-	83.5	84.2
C-Eval (EM)	90.1	92.5	86.9	88.8	
C3 (EM)	78.6	-	83.1	80.3	
Chinese-SimpleQA (EM)	72.1	77.6	70.1	74.6	

	基准（指标）	DeepSeek-V3 Base	Kimi-K2 Base	GLM-4.5 Base	GLM-4.5 MoE
	架构	MoE	MoE	MoE	MoE
# 激活参数	37B	32B	32B	40B	
# 总参数数量	671B	1043B	355B	744B	
中文	SimpleQA (EM)	26.6	35.3	30.0	36.0
BBH (EM)	88.4	88.7	86.2	87.4	
MMLU (EM)	87.2	87.8	86.1	88.3	
HellaSwag (EM)	88.9	94.6	87.1	88.1	
PIQA (EM)	84.7	-	85.3	84.6	
TriviaQA (EM)	82.9	85.1	80.0	80.9	
代码	EvalPlus (Pass@1)	65.6	80.3	78.1	87.0
LiveCodeBench-Base (Pass@1)	24.6	26.3	28.1	34.4	
数学	GSM8K (EM)	87.6	92.1	79.4	68.8
MATH (EM)	62.6	70.2	61.0	56.4	
中文	CLUEWSC (EM)	82.7	-	83.5	84.2
C-Eval (EM)	90.1	92.5	86.9	88.8	
C3 (EM)	78.6	-	83.1	80.3	
中文简易问答 (EM)	72.1	77.6	70.1	74.6	

Terminal-Bench 2.0 (Terminus 2): We evaluate with the Terminus framework using timeout = 2h, temperature = 0.7, top_p = 1.0, max_new_tokens = 8192, with a 128K context window. Resource limits are capped at 16 CPUs and 32 GB RAM.

Terminal-Bench 2.0 (Terminus 2): 我们使用 Terminus 框架进行评估，设置 timeout = 2h, temperature = 0.7, top_p = 1.0, max_new_tokens = 8192, 上下文窗口为 128K。资源限制最大为 16 个 CPU 和 32 GB 内存。

Terminal-Bench 2.0 (Claude Code): We evaluate in Claude Code 2.1.14 (think mode) with temperature = 1.0, top_p = 0.95, max_new_tokens = 65536. We remove wall-clock time limits, while preserving per-task CPU and memory constraints. We fix environment issues introduced by Claude

Code and also report results on a verified Terminal-Bench 2.0 dataset that resolves ambiguous instructions (see: <https://huggingface.co/datasets/zai-org/terminal-bench-2-verified>). Scores are averaged over 5 runs.

Terminal-Bench 2.0 (Claude Code): 我们在 Claude Code 2.1.14(思考模式)中进行评估,参数设置为 temperature = 1.0, top_p = 0.95, max_new_tokens = 65536。我们移除了壁钟时间限制,同时保留了每个任务的 CPU 和内存限制。我们修复了由 Claude Code 引入的环境问题,并且在经过验证的 Terminal-Bench 2.0 数据集上报告结果,该数据集解决了模糊指令的问题 (参见: <https://huggingface.co/datasets/zai-org/terminal-bench-2-verified>)。分数是在 5 次运行中取平均值。

CyberGym: We evaluate in Claude Code 2.1.18 (think mode, no web tools) with (temperature = 1.0, top_p = 1.0, max_new_tokens = 32000) and a 250-minute timeout per task. Results are single-run Pass@1 over 1,507 tasks.

CyberGym: 我们在 Claude Code 2.1.18 (思考模式, 不使用网络工具) 中进行评估, 参数为 (temperature = 1.0, top_p = 1.0, max_new_tokens = 32000), 每个任务有 250 分钟的超时时间。结果是在 1,507 个任务上单次运行的 Pass@1 结果。

MCP-Atlas: All models are evaluated in think mode on the 500-task public subset with a 10-minute timeout per task. We use Gemini 3 Pro as the judge model.

MCP-Atlas: 所有模型均在包含 500 个任务的公共子集上以思考模式进行评估, 每个任务有 10 分钟的超时限制。我们使用 Gemini 3 Pro 作为评判模型。

τ^2 -Bench: We add a small prompt adjustment in Retail and Telecom to avoid failures caused by premature user termination. For Airline, we apply the domain fixes proposed in the Claude Opus 4.5 system card.

τ^2 -Bench: 我们在零售和电信领域添加了一个小的提示调整, 以避免因用户过早终止而导致的失败。对于航空公司, 我们应用了在 Claude Opus 4.5 系统卡片中提出的领域修复。

Vending-Bench 2: Runs are conducted independently by Andon Labs ¹⁰.

Vending-Bench 2: 运行由 Andon Labs ¹⁰ 独立进行。

B.3 Optimized User Simulator for τ^2 -Bench

B.3 为 τ^2 -Bench 优化的用户模拟器

We add a small prompt adjustment in Telecom and Retail to avoid failures caused by premature user termination. The optimized prompts are shown in Figure 12 and Figure 13. These optimized prompts are integrated into the system prompt as follows:

我们在电信和零售领域添加了一个小的提示调整, 以避免因用户过早终止而导致的故障。优化后的提示如图 12 和图 13 所示。这些优化后的提示已按照以下方式整合到系统提示中:

```
SYSTEM_PROMPT = """
{global_user_sim_guidelines}
```

```
<scenario>
{instructions}
</scenario>
```

```
{optimized_user_prompt}
""".strip()
```

```
# Note:
- Do not generate the '###TRANSFER###' before agent clearly tells "YOU ARE BEING TRANSFERRED TO A HUMAN AGENT. PLEASE HOLD ON."
Example:
Case1:
  - agent: "Would you like me to transfer you to a human agent who can assist you with your order?"
  - user: "Yes, please transfer me to a human agent."
Case2:
  - user: "YOU ARE BEING TRANSFERRED TO A HUMAN AGENT. PLEASE HOLD ON."
  - user: "###TRANSFER####"
```

Figure 12: The optimized user prompt for τ^2 -Bench Telecom.

图 12: 针对 τ^2 -Bench Telecom 的优化用户提示。

```
# Rules:
- Just generate one line at a time to simulate the user's message.
- Do not give away all the instruction at once. Only provide the information that is needed for the current step.
- Do not hallucinate information that is not provided in the instruction. Follow these guidelines:
  1. If the agent asks for information NOT in the instruction:
    - Say you don't remember or don't have it
    - Offer alternative information that IS mentioned in the instruction
  2. Examples:
    - If asked for order ID (not in instruction): "Sorry, I don't remember the order ID, but I can help you with other information."
    - If asked for email (not in instruction): "I don't have my email handy, but I can provide you with my phone number."
- Do not repeat the exact instruction in the conversation. Instead, use your own words to rephrase it.
- Try to make the conversation as natural as possible, and stick to the personalities in the instruction.

# Constraint Handling:
- Provide requests strictly based on what is explicitly stated in the instruction.
- Do not assume, extend, substitute, or generalize in any form.
- Do not modify or relax constraints on:
  - Time / Date
  - Budget
  - Specific terms (e.g., "same" must not be replaced with "similar")
- Core Rule: Any attribute NOT mentioned in the instruction can be either changed or kept the same.
- Examples:
  - If instruction says "exchange red item to blue": Only color must change, other attributes must remain the same.
  - If instruction says "exchange red item to blue, keep the same size": Both color and size must remain the same.
- Exception: Only follow additional constraints when explicitly stated in the instruction.

# Domain-Specific Rules:
## For Retail scenarios:
- Focus on product attributes and exchange/return processes as specified in instructions.
```

- During confirmations: Always respond based strictly on the original instruction, never deviate to match agent's provided options. Restate your requirement from the instruction rather than selecting from agent's choices.
- Example: If the agent provides specific options (A/B/C) but the instruction states a general requirement (e.g., "same as pending order"), always restate or confirm based on what the instruction says, not by directly selecting from the agent's provided options.
- 在确认过程中：始终严格根据原始指令进行回应，绝不要偏离以匹配代理提供的选项。应重新陈述指令中的要求，而不是从代理提供的选项中进行选择。
- 示例：如果代理提供了具体的选项（A/B/C），但指令中说明了一般性要求（例如：“与待处理订单相同”），则应根据指令中的内容重新陈述或确认，而不是直接从代理提供的选项中进行选择。# When NOT to finish the conversation:

何时不应结束对话：

- Do not end until you have clearly and completely expressed all your requirements and constraints.
- Do not end until the agent has completed all tasks mentioned in the instruction and verified no operations were missed.
- Do not end if the agent's execution results do not match your expectations or are incorrect/incomplete.
- 在您清楚且完整地表达所有要求和限制条件之前，不要结束。
- 在代理完成指令中提到的所有任务并确认没有遗漏任何操作之前，不要结束。
- 如果代理的执行结果与您的期望不符或存在错误/不完整的情况，不要结束。# When you CAN finish the conversation:

当你可以完成对话时：

- Only when all above conditions are satisfied AND all tasks are completed correctly.
- OR when you have clearly expressed complete requirements but the system explicitly states it cannot complete them due to technical limitations - in this case, accept transfer to human.
- 仅当上述所有条件都满足并且所有任务都正确完成时。
- 或者当您已明确表达完整需求，但系统由于技术限制明确表示无法完成这些任务时 - 在这种情况下，接受转接给人工。# How to finish the conversation:

如何结束对话：

- If the agent has completed all tasks, generate the '###STOP###' token to end the conversation.
- 如果代理已完成所有任务，请生成 '###STOP###' 标记以结束对话。# Note:

注意：

- You should carefully check if the agent has completed all tasks mentioned in the instruction before generating '###STOP###'.
- 在生成 '###STOP###' 之前，你应该仔细检查代理是否已完成指令中提到的所有任务。

Figure 13: The optimized user prompt for τ^2 -Bench Retail.

图 13：为 τ^2 -Bench Retail 优化后的用户提示。

B.4 Evaluation of Real-world Agentic Engineering Experience

B.4 实际代理工程经验的评估

B.4.1 Frontend Evaluation

B.4.1 前端评估

Data. Our dataset encompasses seven distinct frontend scenarios designed to evaluate a model’s engineering proficiency across diverse functional domains: Business Management Systems, Web Games, SVG/Canvas Rendering, Creative Tools & Editors, Showcase Pages, Forms & Tables and Data Visualization.

数据。我们的数据集包含七个不同的前端场景，旨在评估模型在各种功能领域中的工程能力：业务管理系统、网页游戏、SVG/Canvas 渲染、创意工具与编辑器、展示页面、表单与表格以及数据可视化。

Table 12: Distribution of frontend application scenarios.

表 12：前端应用场景分布。

Category	Description	# Tasks	# Checkitems
Business Systems	Enterprise/Personal data and process management.	42	167
Web Games	Interaction and entertainment-focused games.	40	163
SVG/Canvas	Graphics rendering and interactive visualizations.	32	166
Creative Tools	Content creation and online editing tools.	28	160
Showcase Pages	Visual expression and information presentation.	27	115
Forms & Tables	Structured data entry and processing.	26	93
Data Visualization	Graphical data expression and analysis.	25	85

类别	描述	# 任务	# 检查项目
业务系统	企业/个人数据和流程管理。	42	167
网页游戏	以互动和娱乐为主的游戏。	40	163
SVG/Canvas	图形渲染和交互式可视化。	32	166
创意工具	内容创作和在线编辑工具。	28	160
展示页面	视觉表达与信息呈现。	27	115
表单与表格	结构化数据的输入和处理。	26	93
数据可视化	图形数据表达与分析。	25	85

Data Distribution by Coding Languages The benchmark provides full coverage of three main-stream paradigms: Vanilla Web Stack (HTML/CSS/JS), React Component-based Framework, and the Vue 3 + Vite Progressive Solution.

数据分布按编程语言划分该基准测试全面覆盖了三种主流范式：原生网络堆栈（HTML/CSS/JS）、基于 React 组件的框架，以及 Vue 3 + Vite 渐进式解决方案。

Table 13: Statistics of technology stacks and evaluation units.

表 13：技术栈和评估单元的统计情况。

Category	Description	# Tasks	# Checkitems
HTML	Vanilla HTML/CSS/JS development.	113	490
React	Component-based framework development.	58	249
Vue	Vue 3 + Vite progressive solution.	49	210

类别	描述	# 任务	# 检查项目
HTML	Vanilla HTML/CSS/JS 开发。	113	490
React	基于组件的框架开发。	58	249
Vue	Vue 3 + Vite 进阶解决方案。	49	210

Data sample Each test case is composed of three components: the Task, the Checklist, and a Dedicated Environment. Below is a representative example of a test case:

数据样本每个测试用例由三个部分组成：任务、检查清单和专用环境。以下是一个测试用例的示例：

Task: Develop an online drawing tool that includes a brush, an eraser, a white canvas, and a save button.

任务：开发一个在线绘图工具，包括画笔、橡皮擦、白色画布和保存按钮。

The brush color and thickness should be selectable via buttons on the left.

刷子的颜色和粗细应该可以通过左侧的按钮进行选择。

Users can draw on the canvas by clicking and dragging the mouse.

用户可以通过点击并拖动鼠标在画布上绘画。

The eraser size should be selectable via buttons on the left. Users can erase content by clicking and dragging the mouse over the canvas.

橡皮擦的大小应通过左侧的按钮进行选择。用户可以通过在画布上点击并拖动鼠标来擦除内容。

Once the drawing is complete, clicking the “Save” button should allow the user to save the image locally.

完成绘图后，点击“保存”按钮应允许用户将图像保存到本地。

Please implement this using the React framework in the current directory.

请在当前目录中使用 React 框架实现此功能。

Checklist:

清单:

The user can select the brush color and thickness using the left-hand buttons, and drawing is functional via mouse click-and-drag on the canvas.

用户可以使用左侧的按钮选择画笔颜色和粗细，通过在画布上点击并拖动鼠标进行绘画操作。

The user can select the eraser size using the left-hand buttons, and erasing is functional via mouse click-and-drag on the canvas.

用户可以使用左侧的按钮选择橡皮擦的大小，并且可以通过在画布上点击并拖动鼠标来进行擦除操作。

Upon clicking the “Save” button, the generated image is successfully saved to the local machine.

单击“保存”按钮后，生成的图像成功保存到本地计算机。

Data Construction and Validation We implement a rigorous four-stage pipeline to ensure data quality:

数据构建与验证我们实施了一个严格的四阶段流程以确保数据质量:

- Stage 1: Task Synthesis. Tasks are designed by senior frontend experts to ensure they reflect real-world engineering challenges while maintaining a balanced distribution across diverse scenarios and technologies.
- Stage 2: Checklist Generation and Refinement. We initially employ Claude Sonnet 4.5 to synthesize candidate checklists based on task specifications T . These are then meticulously audited and integrated by experts. Through multiple rounds of refinement, we ensure that each check-item is semantically unambiguous, objective, and provides exhaustive coverage of user requirements.
- Stage 3: Execution-based Correction. We conduct cross-validation between the Agent-as-a-Judge framework and human experts. Any discrepancies in judgment trigger a re-evaluation and correction of the underlying data to eliminate potential noise.
- Stage 4: Dynamic Benchmark Iteration. To maintain a high level of discriminative power, we iteratively update the test suite by removing trivial tasks that no longer challenge state-of-the-art coding agents. This expert-led curation process culminated in a final set of 220 high-quality frontend coding tasks and their corresponding checklists.
- 阶段 1：任务综合。任务由资深前端专家设计，以确保它们反映现实中的工程挑战，同时在多样化的场景和技术中保持平衡分布。
- 阶段 2：检查清单生成与优化。我们最初使用 Claude Sonnet 4.5 根据任务规范 T 合成候选检查清单。然后由专家仔细审核并整合。通过多轮优化，我们确保每个检查项语义明确、客观，并全面覆盖用户需求。
- 阶段 3：基于执行的修正。我们进行 Agent-as-a-Judge 框架与人类专家之间的交叉验证。任何判断差异将触发对基础数据的重新评估和修正，以消除潜在的噪声。

- 阶段 4：动态基准迭代。为了保持高度的区分度，我们通过移除不再挑战最先进编码代理的简单任务，持续更新测试套件。这一由专家主导的筛选过程最终形成了 220 个高质量的前端编码任务及其对应的检查清单。